

Componentes/Interface/STIReceCon.prw

562 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

[illegible]

[illegible]

```

207         Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
208         .T.          ) //[14] Indica se o campo e virtual
209
210     oStruMst:AddField( "Cpf"          ,; //[01] Titulo do campo
211         "Cpf"          ,; //[02] Desc do campo
212         "AB_CPF"       ,; //[03] Id do Field
213         "C"            ,; //[04] Tipo do campo
214         nTamCnpj       ,; //[05] Tamanho do campo
215         0              ,; //[06] Decimal do campo
216         Nil            ,; //[07] Code-block de validacao do campo
217         Nil            ,; //[08] Code-block de validacao When do campo
218         Nil            ,; //[09] Lista de valores permitido do campo
219         Nil            ,; //[10] Indica se o campo tem preenchimento obrigatorio
220         Nil            ,; //[11] Code-block de inicializacao do campo
221         Nil            ,; //[12] Indica se trata-se de um campo chave
222         Nil            ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
223         .T.            ) //[14] Indica se o campo e virtual
224
225     oStruMst:AddField( "Prefixo"      ,; //[01] Titulo do campo
226         "Prefixo"      ,; //[02] Desc do campo
227         "AB_PREFIXO"   ,; //[03] Id do Field
228         "C"            ,; //[04] Tipo do campo
229         nTamPref       ,; //[05] Tamanho do campo
230         0              ,; //[06] Decimal do campo
231         Nil            ,; //[07] Code-block de validacao do campo
232         Nil            ,; //[08] Code-block de validacao When do campo
233         Nil            ,; //[09] Lista de valores permitido do campo
234         Nil            ,; //[10] Indica se o campo tem preenchimento obrigatorio
235         Nil            ,; //[11] Code-block de inicializacao do campo
236         Nil            ,; //[12] Indica se trata-se de um campo chave
237         Nil            ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
238         .T.            ) //[14] Indica se o campo e virtual
239
240     oStruMst:AddField( "Numero"        ,; //[01] Titulo do campo
241         "Numero"        ,; //[02] Desc do campo
242         "AB_NUMERO"    ,; //[03] Id do Field
243         "C"            ,; //[04] Tipo do campo
244         nTamNume       ,; //[05] Tamanho do campo
245         0              ,; //[06] Decimal do campo
246         Nil            ,; //[07] Code-block de validacao do campo
247         Nil            ,; //[08] Code-block de validacao When do campo
248         Nil            ,; //[09] Lista de valores permitido do campo
249         Nil            ,; //[10] Indica se o campo tem preenchimento obrigatorio
250         Nil            ,; //[11] Code-block de inicializacao do campo
251         Nil            ,; //[12] Indica se trata-se de um campo chave
252         Nil            ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
253         .T.            ) //[14] Indica se o campo e virtual
254
255     oStruMst:AddField( "Parcela"       ,; //[01] Titulo do campo
256         "Parcela"       ,; //[02] Desc do campo
257         "AB_PARCELA"   ,; //[03] Id do Field
258         "C"            ,; //[04] Tipo do campo

```

[illegible]

[illegible]

[illegible]

```

415
416 @param CPF ou CNPJ do cliente
417 /*/
418 //-----
419 Function STISCnpjRec( cCpfCnpj )
420 cCnpjReceb := cCpfCnpj
421 Return Nil
422
423 //-----
424 /*/{Protheus.doc} STIPanReceCancel
425 Contexto Cancelar recebimento de titulo
426
427 @param
428 @return
429 @obs
430 @sample
431 /*/
432 //-----
433 Function STIPanReceCancel()
434
435 Local oPanSaleCancel := Nil // PaineL Cancelamento de Venda
436 Local oPanelMVC := STIGetPanel() // PaineL Principal
437 Local oBtnConfirm := Nil // Botao de confirmação "Cancelar Recebimento"
438 Local oBtnCancel := Nil // Botao de confirmação "Cancelar"
439
440 oPanSaleCancel := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2)
441
442 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{|| STR0013 },oPanSaleCancel,,,,,T,,,,) //"Confirma o cancelamento do recebimento?"
443 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
444
445
446 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0014,oPanSaleCancel,{ || (STFRestart(), STIRegItemInterface()) }, ; //"Cancelar Recebimento"
447 LARGBTN,ALTURABTN,,,,.T.)
448
449 oBtnCancel := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0015,oPanSaleCancel,{ || STIBaixaTit() }, ; //"Não Cancelar"
450 LARGBTN,ALTURABTN,,,,.T.)
451
452 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL ))
453 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel), CSS_BTN_ATIVO ))
454
455 Return oPanSaleCancel
456
457 //-----
458 /*/{Protheus.doc} STIReCliMdl
459 Configura na Cesta os Dados do Cliente
460
461 @param cCPF // CPF do Cliente
462 @param oModelCli //Modelo do Cliente
463 @return
464 @obs
465 @sample
466 /*/

```

[illegible]

```

518 Local nMVLJPEATU      := SuperGetMV("MV_LJPEATU",,1)    // Pesquisa retaguarda 0=Desabilitado, faz pesquisa Local. 1=Pesquisa retaguarda se falhar local. 2=Pesquisa somente
    retaguarda
519 Local lPesqFindTit    := .F. //Pesquisar no STIFindTit()
520
521 If !Empty(oCpf:cText)
522     lRet := .T.
523 EndIf
524
525 If lRet .Or. (lRet .And. !(Empty(oPrefixo:cText) .AND. Empty(oNumber:cText) .AND. Empty(oParcela:cText)))
526
527     oCpf:cText := StrTran(oCpf:cText, ".", "")
528     oCpf:cText := StrTran(oCpf:cText, "-", "")
529     oCpf:cText := StrTran(oCpf:cText, "/", "")
530
531     If lSelectCust
532         aCliLoja := STDRSelCus(oCpf:cText)
533
534         // Se o CNPJ/CPF informado tiver mais de um cadastro chama tela de selecao de cliente
535         If Len(aCliLoja) > 1
536             STIPnlSelCust(aCliLoja)
537         Else
538             If Len(aCliLoja) > 0
539                 aCliLoja := STDGCustTt(aCliLoja[1])
540             EndIf
541             lPesqFindTit := .T.
542         EndIf
543     Else
544         lPesqFindTit := .T.
545     EndIf
546
547     If lPesqFindTit
548         If nMVLJPEATU <> 0
549             STIFindTit(aCliLoja)
550         Else
551             MsgStop(STR0022)    //"CPF/CNPJ não encontrado na base local! Caso queira acessar a retaguarda, utilize MV_LJPEATU = 1."
552             oCpf:SetFocus()
553         EndIf
554     EndIf
555
556 Else
557     STFMessage(ProcName(),"STOP",STR0012) //"Por favor, informe o CPF/CNPJ, ou Prefixo/Número/Parcela!"
558     STFShowMessage(ProcName())
559     oCpf:SetFocus()
560 EndIf
561
562 Return lRet

```

Componentes/Interface/STIReceSel.prw

879 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'PROTHEUS.CH'
2 #INCLUDE "POSCSS.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STIRECESEL.CH"
5
6 Static oLst          := Nil      //Listbox dos titulos
7 Static aLst          := {}      //Lista dos titulos
8 Static lIsRec        := .F.     //Variavel que informa se e ou nao recebimento
9 Static nLargura      := 0       //Largura
10 Static oMulta        := Nil     //Get Multa
11 Static oJuros        := Nil     //Get Juros
12 Static oDesconto     := Nil     //Get Desconto
13 Static oVlrTot       := Nil     //Get Total
14 Static oQtdTitSel    := Nil     //Get Quant. Titulos Seleccionados
15 Static oVlrTotAPagar := Nil     //Get Total SELECCIONADOS
16 Static oTotOrig      := Nil     //Get Valor
17 Static nGetMulta     := 0       //Variavel get da multa
18 Static nGetJuros     := 0       //Variavel get de juros
19 Static nGetDesconto  := 0       //Variavel get de desconto
20 Static nGetVlrTot    := 0       //Variavel get de valor total
21 Static nGetQtdTitSel := 0       //Variavel get de quant. de titulos seleccionados
22 Static nGetVlrAPagar := 0       //Variavel get de valor total dos SELECCIONADOS
23 Static nGetTotOrig   := 0       //Variavel get da valor
24 Static nGetVlrAcres  := 0       //Variavel get de valor de acrescimo
25 Static oBtnOk        := Nil     //Botao de baixar as parcelas
26 Static oObjRec       := Nil     //Retorna o objeto da classe do recebimento de titulo
27 Static aLista        := {}      //Lista dos Titulos
28 Static aSeleccionado := {}      //Lista dos Titulos SELECCIONADOS
29 Static lAtuValores   := .F.     //Habilita a edição dos campos de multa/juros e desconto (Editavel = .T. / Bloqueado = .F.)
30 Static lAtuValMulta  := .F.     //Habilita a edição dos campos de multao (Editavel = .T. / Bloqueado = .F.)
31 Static lAtuValJuros  := .F.     //Habilita a edição dos campos de jurosos (Editavel = .T. / Bloqueado = .F.)
32 Static lAtuValDesconto := .F.   //Habilita a edição dos campos de desconto (Editavel = .T. / Bloqueado = .F.)
33 Static lAtuValtotal  := .F.     //Habilita a edição dos campos de Total (Editavel = .T. / Bloqueado = .F.)
34
35
36 //-----
37 /*{Protheus.doc} STIPnlSelRece
38 Abre o painel para selecao dos recebimento
39
40 @param
41 @return
42 */
43 //-----
44 Function STIPnlSelRece()
45
46 STIBtnDeActivate()
47 STIChangeCssBtn()
48 STIExchangePanel( { || STIReceSel() } )
49
50 Return .T.
51
52 //-----

```

```

53 /*/{Protheus.doc} STIReceSel
54 Monta painel com opcoes para a selecao dos recebimentos
55
56 @param
57 @return
58 */
59 //-----
60 Function STIReceSel()
61
62 Local oPanelMVC      := STIGetPanel()                                //Painel principal
63 Local oPanGetRece     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Objeto do painel
64 Local nTamAltListBox := oPanelMVC:nHeight * 0.11 //Tamanho: Altura do ListBox
65 Local cTpOp          := STIGetTpOp() //Tipo de operacao
66 Local nList          := 1 //Para o TListBox
67 Local bValid         := {|| .T.} //Valida baixa de titulo
68 Local bButon        := {|| iIf(Eval(bValid),STIBaixaTit(),Nil)} //Botão Baixa/Estorno
69 Local aSalesMan      := IIF(FindFunction('STDFindSMan'),STDFindSMan(),{}) //Carrega o nome do vendedor
70 Local aCustomer      := IIF(FindFunction('STDFindCust'),STDFindCust(),{}) //Carrega informacoes do cliente Nome/Cpf
71 Local oSay15         := Nil
72 Local oSay16         := Nil //Objeto TSay = Tipo
73
74 /* Sempre iniciar a operação de um Recebimento de Titulo com as informações zeradas */
75 nGetTotOrig := 0
76 nGetMulta  := 0
77 nGetJuros  := 0
78 nGetDesconto := 0
79 nGetVlrTot := 0
80 nGetQtdTitSel := 0
81 nGetVlrAPagar := 0
82
83 STFRestart()
84
85 If FindFunction("STWValidRecep")
86     bValid := {|| STWValidRecep(cTpOp,aLista,aSelecioneado)}
87 EndIf
88
89 oObjRec := STIRetObjTit()
90 oLst := Nil
91 nLargura := (oPanGetRece:nWidth / 2) - (2 * ((oPanGetRece:nWidth / 2) * 0.04)) //Largura
92
93 /* oSay1 - Recebimentos de títulos > Selecionar Títulos */
94 If cTpOp == 'R'
95     oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanGetRece,,,,,T,,,,) //"Recebimentos de títulos > Selecionar Títulos"
96 Else
97     oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0002},oPanGetRece,,,,,T,,,,) //"Estorno de títulos > Selecionar Títulos"
98 EndIf
99 oSay1:SetCSS( POSCSS (GetClassName(oSay1),CSS_BREADCUMB ))
100
101 /* oSay14 - Filial */
102 oSay14 := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0023},oPanGetRece,,,,,T,,,,) //"Filial"
103 oSay14:SetCSS( POSCSS (GetClassName(oSay14),CSS_BREADCUMB))
104

```

```

105 /* oSay2 - Prefixo */
106 oSay2 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 4,{||STR0003},oPanGetRece,,,,,.T.,,,) //"Prefixo"
107 oSay2:SetCSS( POSCSS (GetClassName(oSay2),CSS_BREADCUMB))
108
109 /* oSay9 - Titulo */
110 oSay9 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 7,{||STR0004},oPanGetRece,,,,,.T.,,,) //"Titulo"
111 oSay9:SetCSS( POSCSS (GetClassName(oSay9),CSS_BREADCUMB))
112
113 /* oSay10 - Parcela */
114 oSay10 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 10.0,{||STR0005},oPanGetRece,,,,,.T.,,,) //"Parcela"
115 oSay10:SetCSS( POSCSS (GetClassName(oSay10),CSS_BREADCUMB))
116
117 /* oSay3 - Data */
118 oSay3 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 12.7,{||STR0006},oPanGetRece,,,,,.T.,,,) //"Vencimento"
119 oSay3:SetCSS( POSCSS (GetClassName(oSay3),CSS_BREADCUMB))
120
121 /* oSay4 - Valor */
122 oSay4 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 16.3,{||STR0007},oPanGetRece,,,,,.T.,,,) //"Valor"
123 oSay4:SetCSS( POSCSS (GetClassName(oSay4),CSS_BREADCUMB ))
124
125 /* oSay15 - Acrescimo */
126 oSay15 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 18.5,{||STR0025},oPanGetRece,,,,,.T.,,,) //"Acrescimo"
127 oSay15:SetCSS( POSCSS (GetClassName(oSay4),CSS_BREADCUMB ))
128
129 /* oSay16 - Tipo */
130 oSay16 := TSay():New(POSVERT_LABEL1,POSHOR_1 * 21.5,{||STR0026},oPanGetRece,,,,,.T.,,,) //"Tipo"
131 oSay16:SetCSS( POSCSS (GetClassName(oSay16),CSS_BREADCUMB ))
132
133 /* oLst - Listbox com os titulos */
134 oLst := TListBox():Create(oPanGetRece, POSVERT_GET1, POSHOR_1, {||u|if(Pcount()>0,nList:=u,nList)},;
135     STIReceLst(), nLargura, nTamAltListBox,{|| STITitAtualiza()});
136     ,,,.T.,,,;
137     {|| (STIGetValues(),lAtuValores := .F.)},,,,;
138     ,,,,)
139 oLst:SetCSS( POSCSS (GetClassName(oLst),CSS_LISTBOX ))
140
141 If cTpOp == 'R'
142
143     /* Botao atualizar valores */
144     oButton := TButton():New(POSVERT_GET2 * 1.65,POSHOR_BTNFOCAL * 0.91,STR0012,oPanGetRece,{ || Iif(lAtuValores,(STIUpdTit(),lAtuValores := .F.),Nil) },LARGBTN *
145     0.90,ALTURABTN * 0.75,,,,.T.,,,) //"Atualizar Valores"
146     oButton:SetCSS( POSCSS (GetClassName(oButton),POSVERT_BTNFOCAL ))
147
148     /* Botao Editar valores */
149     oEdtButton := TButton():New(POSVERT_GET2 * 2.00,POSHOR_BTNFOCAL * 0.91,STR0020,oPanGetRece,{ || (lAtuValores := STIVldFldRec()) },LARGBTN * 0.90,ALTURABTN *
150     0.75,,,,.T.,,,) //"Editar Valores"
151     oEdtButton:SetCSS( POSCSS (GetClassName(oEdtButton),POSVERT_BTNFOCAL ))
152
153     /* oSay13 - Valor */
154     oSay13 := TSay():New(POSVERT_LABEL2 * 1.51,POSHOR_1,{||STR0022},oPanGetRece,,,,,.T.,,,) //"Valor Original"
155     oSay13:SetCSS( POSCSS (GetClassName(oSay13),CSS_BREADCUMB ))

```

```

155
156 /* Valor Original */
157 oTotOrig := TGet():New(POSVERT_GET2 * 1.40,POSHOR_1,{|u| If(PCount()>0,nGetTotOrig:=u,nGetTotOrig)},oPanGetRece,(oPanelMVC:nWidth / 2) * 0.190,ALTURAGET,"@E
99,999,999.99" ,,,,,,T,,,,,T,,,,,"nGetTotOrig")
158 oTotOrig:SetCSS( POSCSS (GetClassName(oTotOrig),CSS_GET_FOCAL ))
159
160 /* oSay5 - Multa */
161 oSay5 := TSay():New(POSVERT_LABEL2 * 1.51,POSHOR_1 * 6.3,{||STR0008},oPanGetRece,,,,,T,,,,) //"Multa"
162
163 oSay5:SetCSS( POSCSS (GetClassName(oSay5),CSS_BREADCUMB ))
164
165 /* oMulta */
166 oMulta := TGet():New(POSVERT_GET2 * 1.40,POSHOR_1 * 6.3,{|u| If(PCount()>0,nGetMulta:=u,nGetMulta)},oPanGetRece,(oPanelMVC:nWidth / 2) * 0.14,ALTURAGET,"@E 999.99",{||
STIAtuVlFinal()},,,,,T,,,,{|| lAtuValMulta },,,,,"nGetMulta")
167 oMulta:SetCSS( POSCSS (GetClassName(oMulta),CSS_GET_FOCAL ))
168
169 /* oMulta - Atualiza valores*/
170 oMulta:bLostFocus := { || STIAtlValView("MULTA") }
171
172 /* oSay6 - Juros */
173 oSay6 := TSay():New(POSVERT_LABEL2 * 1.51,POSHOR_1 * 10.5,{||STR0009},oPanGetRece,,,,,T,,,,) //"Juros"
174 oSay6:SetCSS( POSCSS (GetClassName(oSay6),CSS_BREADCUMB ))
175
176 /* oJuros */
177 oJuros := TGet():New(POSVERT_GET2 * 1.40,POSHOR_1 * 10.5 ,{|u| If(PCount()>0,nGetJuros:=u,nGetJuros)},oPanGetRece,(oPanelMVC:nWidth / 2) * 0.14,ALTURAGET,"@E
999.99",{|| STIAtuVlFinal()},,,,,T,,,,{|| lAtuValJuros },,,,,"nGetJuros")
178 oJuros:SetCSS( POSCSS (GetClassName(oJuros),CSS_GET_FOCAL ))
179
180 /* oJuros - Atualiza valores*/
181 oJuros:bLostFocus := { || STIAtlValView("JUROS") }
182
183 /* oSay7 - Desconto */
184 oSay7 := TSay():New(POSVERT_LABEL2 * 1.51,POSHOR_1 * 14.6,{||STR0010},oPanGetRece,,,,,T,,,,) //"Desconto"
185 oSay7:SetCSS( POSCSS (GetClassName(oSay7),CSS_BREADCUMB ))
186
187 /* oDesconto */
188 oDesconto := TGet():New(POSVERT_GET2 * 1.40,POSHOR_1 * 14.6,{|u| If(PCount()>0,nGetDesconto:=u,nGetDesconto)},oPanGetRece,(oPanelMVC:nWidth / 2) * 0.14,ALTURAGET,"@E
999.99",{|| STIAtuVlFinal()},,,,,T,,,,{|| lAtuValDesconto },,,,,"nGetDesconto")
189 oDesconto:SetCSS( POSCSS (GetClassName(oDesconto),CSS_GET_FOCAL ))
190
191 /* oDesconto - Atualiza valores*/
192 oDesconto:bLostFocus := { || STIAtlValView("DESCONTO") }
193
194 /* oSay8 - Valor Final */
195 oSay8 := TSay():New(POSVERT_LABEL2 * 1.51,POSHOR_1 * 18.6,{||STR0011},oPanGetRece,,,,,T,,,,) //"Valor Final"
196 oSay8:SetCSS( POSCSS (GetClassName(oSay8),CSS_BREADCUMB ))
197
198 /* oVlrTot */
199 oVlrTot := TGet():New(POSVERT_GET2 * 1.40,POSHOR_1 * 18.6,{|u| If(PCount()>0,nGetVlrTot:=u,nGetVlrTot)},oPanGetRece,(oPanelMVC:nWidth / 2) * 0.225,ALTURAGET,
PesqPict("SE1","E1_VALOR") ,{|| STIVldVlFinal()},,,,,T,,,,{|| lAtuValTotal },,,,,"nGetVlrTot")
200 oVlrTot:SetCSS( POSCSS (GetClassName(oVlrTot),CSS_GET_FOCAL ))
201

```

```

202      /* oDesconto - Atualiza valores*/
203      oVlrTot:bLostFocus := { || STIAtlValView("VLRTOT") }
204
205 EndIf
206
207 /* oSay12 - Qtd. Tit. Sel. */
208 oSay12 := TSay():New(POSVERT_LABEL2 * iif(cTpOp == "R", 1.95, 1.51) ,POSHOR_1 * 1.0,{||STR0017},oPanGetRece,,,,,.T.,,,) //"Qtd. Tit. Sel."
209 oSay12:SetCSS( POSCSS (GetClassName(oSay12),CSS_BREADCUMB ))
210
211 /* oQtdTitSel */
212 oQtdTitSel := TGet():New(POSVERT_GET2 * iif(cTpOp == "R", 1.75, 1.40),POSHOR_1 * 1.0,{|u| If(PCount())>0,nGetQtdTitSel:=u,nGetQtdTitSel}},oPanGetRece,(oPanelMVC:nWidth / 2) *
    0.14,ALTURAGET, "9999" ,,,,,,.T.,,,,,.T.,,,,"nGetQtdTitSel")
213 oQtdTitSel:SetCSS( POSCSS (GetClassName(oQtdTitSel),CSS_GET_FOCAL ))
214
215 /* oSay11 - Total a Pagar */
216 oSay11 := TSay():New(POSVERT_LABEL2 * iif(cTpOp == "R", 1.95, 1.51),POSHOR_1 * 6.4,{||STR0018},oPanGetRece,,,,,.T.,,,) //"Total a Pagar"
217 oSay11:SetCSS( POSCSS (GetClassName(oSay11),CSS_BREADCUMB ))
218
219 /* oVlrTotAPagar */
220 oVlrTotAPagar := TGet():New(POSVERT_GET2 * iif(cTpOp == "R", 1.75, 1.40),POSHOR_1 * 6.4,{|u| If(PCount())>0,nGetVlrAPagar:=u,nGetVlrAPagar}},oPanGetRece,(oPanelMVC:nWidth / 2)
    * 0.225,ALTURAGET, PesqPict("SE1","E1_VALOR") ,,,,,,.T.,,,,,.T.,,,,"nGetVlrAPagar")
221 oVlrTotAPagar:SetCSS( POSCSS (GetClassName(oVlrTotAPagar),CSS_GET_FOCAL ))
222
223 /* Identificacao do vendedor */
224 If Len(aSalesMan) > 0
225     oLblSalesMan := TSay():New(POSVERT_LABEL7,POSHOR_1,{||STR0021 + " " + AllTrim(aSalesMan[1])},oPanGetRece,,,,,.T.,,,100) //"Vendedor(a):"
226     oLblSalesMan:SetCSS( POSCSS (GetClassName(oLblSalesMan), CSS_BREADCUMB ))
227
228     LjGrvLog( /*cNumControl*/, "Nome do Vendedor", aSalesMan[1] )//Grava Log =====
229 EndIf
230
231 /* Identificacao do cliente */
232 If Len(aCustomer) > 0
233     oLblCustomer := TSay():New(POSVERT_LABEL5,POSHOR_1,{||AllTrim(aCustomer[1])},oPanGetRece,,,,,.T.,,,100)
234     oLblCustomer:SetCSS( POSCSS (GetClassName(oLblCustomer), CSS_BREADCUMB ))
235
236     oLblCpf := TSay():New(POSVERT_LABEL6,POSHOR_1,{||AllTrim(aCustomer[2])},oPanGetRece,,,,,.T.,,,100)
237     oLblCpf:SetCSS( POSCSS (GetClassName(oLblCpf), CSS_BREADCUMB ))
238
239     LjGrvLog( /*cNumControl*/, "Nome do Cliente", aCustomer[1] )//Grava Log =====
240     LjGrvLog( /*cNumControl*/, "CPF/CNPJ do Cliente", aCustomer[2] )//Grava Log =====
241 EndIf
242
243 // Inicializa Variáveis
244 //STITitIni()
245 //
246 //STITitAtualiza()
247 //oLst:Select(1)
248 //oLst:SetFocus()
249
250 /* Botao baixar titulo */
251 If cTpOp == 'R'

```

```

252     oBtnOk := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0013+CRLF+"(CTRL+B)",oPanGetRece,bButon,LARGBTN,ALTURABTN,,,,.T.) //"Baixar Titulo"
253     SetKey(2,{|| STIBaixaTit() })
254 Else
255     oBtnOk := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0014+CRLF+"(CTRL+E)",oPanGetRece,bButon,LARGBTN,ALTURABTN,,,,.T.) //"Estornar Titulo"
256     SetKey(5,{|| STIBaixaTit() })
257 EndIf
258 oBtnOk:SetCSS( POSCSS (GetClassName(oBtnOk),CSS_BTN_FOCAL ))
259
260 Return oPanGetRece
261
262 //-----
263 /*/{Protheus.doc} STIReceLst
264 Formata a lista de titulos para exibir no listBox
265
266 @param
267 @return
268 */
269 //-----
270 Function STIReceLst(lUpdate)
271
272 Local aRet      := {}                // Array de Retorno
273 Local nX        := 0                // Contador
274 Local nGetPos   := 0                // Posição do TListBox
275
276 Default lUpdate := .F.
277
278 aRet := {}
279 aLista := STIGetTitles()
280 If ValType(oLst) = "0"
281     nGetPos := oLst:GetPos()
282 Else
283     STISetTitSelecioneado(0)
284 EndIf
285
286 For nX := 1 To Len(aLista)
287     aAdd( aRet, STITitDescricao(nX) )
288 Next nX
289
290 If lUpdate
291     If oLst:Len() = 0
292         oLst:Reset()
293         oLst:SetArray(aRet)
294     Else
295         oLst:Modify(aRet[nGetPos],nGetPos)
296     EndIf
297 EndIf
298
299 Return aRet
300
301 //-----
302 /*/{Protheus.doc} STIUpdTit
303 Atualização de juros, multa e desconto no valor dos titulos

```

```

304
305 @param
306 @return
307 */
308 //-----
309 Function STIUpdTit(lAtualiza ,nLine ,nDescTit ,nVlrTit,;
310                 _nGetMulta ,_nGetJuros)
311
312 DEFAULT nLine      := oLst:GetPos()      // Linha selecionada
313 DEFAULT nDescTit   := 0
314 DEFAULT nVlrTit    := 0
315 DEFAULT lAtualiza  := .T.
316 DEFAULT _nGetMulta := 0
317 DEFAULT _nGetJuros := 0
318
319 /* Caso os valores abaixo estejam preenchidos, eh por que vieram do PE STIVALIDREC e ignoraram a informacao
320    do padrao que esta na tela.
321 */
322
323 If nDescTit > 0
324     nGetDesconto := nDescTit
325 EndIf
326
327 If nVlrTit > 0
328     nGetVlrTot := nVlrTit
329 EndIf
330
331 If _nGetMulta > 0
332     nGetMulta := _nGetMulta
333 EndIf
334
335 If _nGetJuros > 0
336     nGetJuros := _nGetJuros
337 EndIf
338
339 oObjRec:SetParcMulct(nLine, nGetMulta)
340 oObjRec:SetParcInterest(nLine, nGetJuros)
341 oObjRec:SetParcDiscount(nLine, nGetDesconto)
342 oObjRec:SetParcTotal(nLine, nGetVlrTot)
343
344 // Atualiza títulos selecionados na tela
345 STITitAtualiza(nLine)
346 oLst:Select(nLine)
347 oLst:Refresh()
348
349 Return .T.
350
351 //-----
352 /*{Protheus.doc} STIBaixaTit
353 Realiza a baixa dos titulos
354
355 @param

```

```

356 @return
357 /*/
358 //-----
359 Function STIBaixaTit()
360
361 Local nX          := 0          //Variavel de loop
362 Local lRet        := .T.        //Variavel de retorno
363 Local cTpOp       := STIGetTpOp() //Tipo de operacao
364 Local nParcTotal  := 0          // Total dos títulos
365 Local aNCCs       := STDGetNCCs("1") //NCCs do cliente
366 Local aNCCCli     := {}
367 Local lSelNcc     := ExistBlock("STSelNcc") //Existe ponto de entrada para selecao da NCC?
368 Local lSTConfEst  := ExistBlock("STConfEst") //Existe ponto de entrada para o estorno de título
369 Local oEstTit     := Nil        //Recebe o objeto dos titulos selecionados para o estorno
370
371 // Soma dos títulos selecionados
372 For nX := 1 to Len(aLista)
373     If STIGetTitSelecionado(nX)
374         nParcTotal := nParcTotal + oObjRec:GetParcTotal(nX)
375         oObjRec:SetParcSelected(nX, .T.)
376         AADD(aNCCCli,{aLista[nX][13],aLista[nX][14]}) ///Codigo de Cliente e Loja do Titulo
377     EndIf
378 Next
379
380 STFSetTot( 'L1_VLRTOT', nParcTotal )
381
382 If nParcTotal = 0
383     lRet := .F.
384     STFMessage(ProcName(0), "STOP", STR0019) //"Favor selecionar pelo menos um título!"
385     STFShowMessage(ProcName(0))
386 Else
387     If cTpOp == "R" .AND. !lSelNcc
388         STFCleanMessage(ProcName(0))
389         STBRemoteExecute("STDFindNCCModel", {aNCCCli[1][1],aNCCCli[1][2]},,, @aNCCs) // Busca NCCs do cliente na Retaguarda
390         STDSetNCCs("1",aNCCs)
391     EndIf
392 EndIf
393
394 If !Empty(nParcTotal) .AND. cTpOp == 'R' .AND. lRet
395
396     lIsRec := .T.
397     STIChangeCssBtn()
398     If Len(aNCCs) > 0
399         STIExchangePanel( { || STINCCSelection() } )
400     Else
401         STIExchangePanel( { || STIPayment() } )
402     EndIf
403
404 ElseIf oLst:Len() > 0 .AND. !Empty(nParcTotal) .AND. cTpOp == 'E' .AND. lRet
405
406     STFMessage(ProcName(0), "ALERT", STR0024 ) //"Aguarde. Realizando o estorno do título."
407     STFShowMessage(ProcName(0))

```

```

408   If oObjRec:ReverseDropTitles()
409       oObjRec:Print()
410
411       If lSTConfEst
412           oEstTit := STIRetObjTit()
413           ExecBlock("STConfEst",.F.,.F.,{oEstTit})
414       EndIf
415
416       //Excluo o movimento da tabela MHK
417       If ChkFile("MHJ") .AND. ChkFile("MHK")
418           STIExcMHX(oObjRec)
419       EndIf
420       STIRegItemInterface()
421       STFMessage(ProcName(0), "STOP", STR0016) //"Estorno realizado com sucesso."
422       STFShowMessage(ProcName(0))
423       STFRestart()
424   Else
425       //Necessario zerar o total para poder efetuar outras operações no caixa.
426       STFSetTot( 'L1_VLRTOT', 0 )
427       lRet := .F.
428   EndIf
429 EndIf
430
431 Return lRet
432
433
434 //-----
435 /*{Protheus.doc} STIGetRecTit
436 Retornar a variavel logica dizendo se e um recebimento de
437 titulo ou nao
438
439 @param
440 @return
441 */
442 //-----
443 Function STIGetRecTit()
444 Return lIsRec
445
446
447 //-----
448 /*{Protheus.doc} STISetRecTit
449 Alimenta a variavel lIsRec se .T. ou .F.
450
451 @param
452 @return
453 */
454 //-----
455 Function STISetRecTit(lRec)
456 lIsRec := lRec
457 Return .T.
458
459

```

```

460 //-----
461 /*{Protheus.doc} STIGetValues
462 Alimenta os gets com os valores de multa, juros e desconto
463
464 @param
465 @return
466 */
467 //-----
468 Static Function STIGetValues()
469
470 Local nPosAnt := oLst:GetPos()      // Linha anterior
471
472 STITitDbClick()
473 oLst:Select(nPosAnt)
474 oLst:SetFocus()
475
476 Return .T.
477
478 //-----
479 /*{Protheus.doc} STIAtlValView
480 Atualiza o valor total considerando juros, multa e desconto
481 somente em tela
482
483 @param
484 @return
485 */
486 //-----
487 Static Function STIAtlValView(cField)
488
489 Local nTotAPagar      := 0           // Calculo das parcelas
490 Local nX              := 0           // Contador
491 Local nPosAnt         := oLst:GetPos() // Linha anterior
492 Local nNewValue       := 0           // Valor a ser somado ou subtraido
493 Local lAterado        := .F.         // Verifica se foi alterado o valor
494
495 Default cField        := ""          // Campo alterado (juros/multa/desconto)
496
497 If AllTrim(Upper(cField)) == "MULTA"
498     nNewValue := nGetMulta - IIf(ValType(oObjRec:GetParcMulct(oLst:GetPos())) == "N", oObjRec:GetParcMulct(oLst:GetPos()), 0)
499 ElseIf AllTrim(Upper(cField)) == "JUR0S"
500     nNewValue := nGetJuros - IIf(ValType(oObjRec:GetParcInterest(oLst:GetPos())) == "N", oObjRec:GetParcInterest(oLst:GetPos()), 0)
501 ElseIf AllTrim(Upper(cField)) == "VLRT0T"
502     nNewValue := nGetVlrTot - IIf(ValType(oObjRec:GetParcTotal(oLst:GetPos())) == "N", oObjRec:GetParcTotal(oLst:GetPos()), 0)
503     nGetDesconto := 0
504 Else
505     nNewValue := IIf(ValType(oObjRec:GetParcDiscount(oLst:GetPos())) == "N", oObjRec:GetParcDiscount(oLst:GetPos()), 0) - nGetDesconto
506 EndIf
507
508 If nNewValue <> 0
509     lAterado := .T.
510 EndIf
511

```

```

512 If lAterado
513
514     nGetVlrTot := oObjRec:GetParcTotal(oLst:GetPos()) + nNewValue
515
516     STIUpdTit(.F.)
517
518     oVlrTot:CtrlRefresh()
519
520     For nX := 1 to Len(aLista)
521         If STIGetTitSelecioneado(nX)
522             nTotAPagar := nTotAPagar + oObjRec:GetParcTotal(nX)    +;
523                                     oObjRec:GetParcMulct(nX)      +;
524                                     oObjRec:GetParcInterest(nX)   +;
525                                     oObjRec:GetParcDiscount(nX)
526         EndIf
527     Next
528     nGetVlrAPagar := nTotAPagar
529     oVlrTotAPagar:CtrlRefresh()
530
531 EndIf
532 oLst:Select(nPosAnt)
533
534 Return .T.
535
536
537 //-----
538 /*{Protheus.doc} STISetTitSelecioneado
539 Gravo o conteúdo do array a partir do aLista
540 Determino se uma conta a receber está selecionada ou não
541 Se nPos = 0, limpo o conteúdo do array
542
543 @param
544 @return
545 */
546 //-----
547 Function STISetTitSelecioneado(nPos,lSel)
548
549 Local nX      := 0           // Contador
550 Local nRet    := .F.        // Retorno
551
552 Default nPos      := 0
553 Default lSel      := .F.
554
555 If nPos = 0
556     aSelecioneado := {}
557     For nX := 1 to Len(aLista)
558         aAdd(aSelecioneado,.F.)
559     Next
560     nRet := .T.
561 Elseif nPos <= Len(aSelecioneado)
562     aSelecioneado[nPos] := lSel
563     nRet := .T.

```

```

564 EndIf
565
566 Return nRet
567
568
569 //-----
570 /*{Protheus.doc} STIGetTitSelecionado
571 Leio o conteúdo do array a partir da posição do alista
572 Retorno se uma conta a receber está selecionado ou não
573
574 @param
575 @return
576 */
577 //-----
578 Function STIGetTitSelecionado(nPos)
579
580 Local      lRet      := .F.      // Atribuição
581 Default nPos      := 0          // Posição do elemento do array
582
583 If nPos > 0 .AND. nPos <= Len(aSelecionado)
584     lRet := aSelecionado[nPos]
585 EndIf
586
587 Return lRet
588
589
590 //-----
591 /*{Protheus.doc} STITitDbClick
592 Seleciono (ou não) o título, permitindo multiseleção
593
594 @param
595 @return
596 */
597 //-----
598 Static Function STITitDbClick()
599
600 Local nGetPos := oLst:GetPos()          // Linha do título
601
602 STISetTitSelecionado(nGetPos,!aSelecionado[nGetPos])
603
604 oLst:Modify(STITitDescricao(nGetPos),nGetPos)
605
606 // Refresh da linha
607 oLst:Select(nGetPos)
608
609 Return .T.
610
611
612 //-----
613 /*{Protheus.doc} STITitDescricao
614 Preenche no Listbox a descrição de cada linha
615

```

```

616 @param
617 @return
618 /*/
619 //-----
620 Function STITitDescricao(nX)
621
622 Local   cRet           := ""    // Retorno
623 Local   cSelecioneado  := ""    // Seleção do Título
624 Local   nValorTot      := 0     // Valor total da parcela
625 Default nX            := 0     // Posição do elemento do array
626
627 If nX <= 0 .OR. nX > Len(aLista)
628     cRet := ""
629 Else
630
631     If nX <= 0 .OR. nX > Len(aSelecioneado)
632         cSelecioneado := Space(3)
633     Else
634         cSelecioneado := iif(aSelecioneado[nX], "[+]", Space(3))
635     EndIf
636
637     //Valor total:
638     nValorTot := aLista[nX,10]
639
640     //Quando estorno deve exibe o valor total +juros +multa - desconto
641     If AllTrim(STIGetTpOp()) == "E"
642         nValorTot := aLista[nX,6]    // quando estorno pegar o valor original do titulo
643         nValorTot += aLista[nX,7] + aLista[nX,8] + aLista[nX,17] - aLista[nX,9]
644     EndIf
645
646     cRet := (   aLista[nX,15] + Space( (Int(nLargura * 0)) + (TamSX3("E1_FILIAL")[1] - Len(AllTrim(aLista[nX,14])))) ) +;
647               aLista[nX,2] + Space( (Int(nLargura * 0.04)) + (TamSX3("E1_PREFIXO")[1] - Len(AllTrim(aLista[nX,2])))) ) +;
648               aLista[nX,3] + Space( (Int(nLargura * 0.04)) + (TamSX3("E1_NUM")[1] - Len(AllTrim(aLista[nX,3])))) ) +;
649               aLista[nX,4] + Space( (Int(nLargura * 0.05)) + (TamSX3("E1_PARCELA")[1] - Len(AllTrim(aLista[nX,4])))) ) +;
650               Transform(aLista[nX][5], '') + Space( (Int(nLargura * 0.01)) + (TamSX3("E1_VENCTO")[1] - Len(AllTrim(aLista[nX,5])))) ) +;
651               Str(nValorTot, 12,2) +;
652               Space(4) + Str(aLista[nX,17], 12,2)) + Space( (Int(nLargura * 0.02)) + (TamSX3("E1_ACRESC")[1] - Len(AllTrim(aLista[nX,17])))) ) +;
653               aLista[nX,11] + Space(2) + cSelecioneado
654
655 EndIf
656
657 Return cRet
658
659
660 //-----
661 /*/{Protheus.doc} STITitAtualiza()
662 Alimenta os gets com os valores de multa, juros e desconto
663
664 @param
665 @return
666 /*/
667 //-----

```

```

668 Static Function STITitAtualiza(nLine)
669
670 Local cTpOp      := STIGetTpOp()          // Tipo de operacao
671 Local nX         := 0                    // Contador
672 Local nPosAnt    := oLst:GetPos()         // Linha anterior
673
674 DEFAULT nLine    := 0
675 // Cenario aonde o cliente alterava um campo e ao inves de dar tab ja clicava em outro titulo no list
676 If nLine > 0 // tem q manter a mesma linha do titulo
677     nPosAnt := nLine
678 EndIf
679
680 nGetMulta      := 0
681 nGetJuros      := 0
682 nGetDesconto   := 0
683 nGetQtdTitSel  := 0
684 nGetVlrTot     := 0
685 nGetVlrAPagar  := 0
686
687 If nPosAnt > 0
688     nGetTotOrig := oObjRec:GetParcOrig(nPosAnt)
689
690     nGetMulta      := IIf(ValType(oObjRec:GetParcMulct(nPosAnt)) == "N",oObjRec:GetParcMulct(nPosAnt),0)
691     nGetJuros      := IIf(ValType(oObjRec:GetParcInterest(nPosAnt)) == "N",oObjRec:GetParcInterest(nPosAnt),0)
692     nGetDesconto   := IIf(ValType(oObjRec:GetParcDiscount(nPosAnt)) == "N",oObjRec:GetParcDiscount(nPosAnt),0)
693     nGetVlrTot     := IIf(ValType(oObjRec:GetParcTotal(nPosAnt)) == "N",oObjRec:GetParcTotal(nPosAnt),0)
694     nGetVlrAcres   := IIf(ValType(oObjRec:GetParcIncrease(nPosAnt)) == "N",oObjRec:GetParcIncrease(nPosAnt),0)
695 EndIf
696
697 For nX := 1 to Len(aLista)
698     If STIGetTitSelecioneado(nX)
699         nGetQtdTitSel := nGetQtdTitSel + 1
700
701         //Quando estorno deve mostrar o valor total + multa + juros - desconto
702         If cTpOp == "E"
703             nGetVlrAPagar := nGetVlrAPagar + IIf(ValType(oObjRec:GetParcValor(nX)) == "N",oObjRec:GetParcValor(nX),0) //Quando estorno busca o valor origem do titulo
704             nGetVlrAPagar += IIf(ValType(oObjRec:GetParcInterest(nX)) == "N",oObjRec:GetParcInterest(nX),nGetJuros) //Juros da parcela
705             nGetVlrAPagar += IIf(ValType(oObjRec:GetParcMulct(nX)) == "N",oObjRec:GetParcMulct(nX),nGetMulta) //Multa da parcela
706             nGetVlrAPagar -= IIf(ValType(oObjRec:GetParcDiscount(nX)) == "N",oObjRec:GetParcDiscount(nX),nGetDesconto) //Desconto da parcela
707             nGetVlrAPagar += IIf(ValType(oObjRec:GetParcIncrease(nPosAnt)) == "N",oObjRec:GetParcIncrease(nPosAnt),0) //Acrescimo da parcela
708         Else
709             nGetVlrAPagar := nGetVlrAPagar + IIf(ValType(oObjRec:GetParcTotal(nX)) == "N",oObjRec:GetParcTotal(nX),0)
710         EndIf
711     EndIf
712 Next
713
714 If cTpOp == "R" // Recebimento
715     oMulta:CtrlRefresh()
716     oJuros:CtrlRefresh()
717     oDesconto:CtrlRefresh()
718     oVlrTot:CtrlRefresh()
719     oTotOrig:CtrlRefresh()

```

```

720 EndIf
721 oQtdTitSel:CtrlRefresh()
722 oVlrTotAPagar:CtrlRefresh()
723
724 oLst:Select(nPosAnt)
725
726 Return .T.
727
728
729 //-----
730 /*{Protheus.doc} STITitIni()
731 Reinicializa variáveis de multa, juros, valores totais.
732
733 @param
734 @return
735 */
736 //-----
737 Static Function STITitIni()
738
739 Local cTpOp := STIGetTpOp()          // Tipo de operação
740
741 nGetMulta      := 0
742 nGetJuros      := 0
743 nGetDesconto   := 0
744 nGetQtdTitSel  := 0
745 nGetVlrTot     := 0
746 nGetVlrAPagar  := 0
747 lAtuValores    := .F.      //Habilita a edição dos campos de multa/juros e desconto (Editavel = .T. / Bloqueado = .F.)
748 lAtuValMulta   := .F.      //Habilita a edição dos campos de multa (Editavel = .T. / Bloqueado = .F.)
749 lAtuValJuros   := .F.      //Habilita a edição dos campos de juros (Editavel = .T. / Bloqueado = .F.)
750 lAtuValDesconto := .F.      //Habilita a edição dos campos de desconto (Editavel = .T. / Bloqueado = .F.)
751 lAtuValTotal   := .F.      //Habilita a edição dos campos de Total (Editavel = .T. / Bloqueado = .F.)
752
753 If cTpOp == "R" // Recebimento
754     oMulta:CtrlRefresh()
755     oJuros:CtrlRefresh()
756     oDesconto:CtrlRefresh()
757     oVlrTot:CtrlRefresh()
758     oTotOrig:CtrlRefresh()
759 EndIf
760 oQtdTitSel:CtrlRefresh()
761 oVlrTotAPagar:CtrlRefresh()
762
763 Return .T.
764
765 //-----
766 /*{Protheus.doc} STIVldFldRec()
767 Chamada da função responsavel por validar se habilita ou não
768 os campos Multa,Juros e Descontos para edição para o titulo selecionado.
769
770 @param  objt - Objeto responsavel pela chamada
771 @return

```

```

772 /*/
773 //-----
774 Static Function STIVldFldRec()
775 Local lRet := .T.
776
777 //habilita todos os botoes de edicao
778 lAtuValMulta := lAtuValJuros := lAtuValDesconto := lAtuValtotal := lRet
779
780 If FindFunction("STWVldFldRec")
781     lRet := STWVldFldRec(aLista[oLst:GetPos()],@lAtuValMulta, @lAtuValJuros, @lAtuValDesconto,;
782                         @lAtuValtotal      , "R" , aLista      , aSelecioneado)
783 EndIf
784
785 Return lRet
786
787 //-----
788 /*/{Protheus.doc} STIVldVlFinal()
789 Chamada da função responsavel por validar se o valor informado no
790 no campo Valor final esta correto
791
792 @return
793 /*/
794 //-----
795 Static Function STIVldVlFinal()
796 Local lRet := .T.
797 Local nVlrAdicio := ( nGetJuros + nGetMulta + nGetVlrAcres ) - nGetDesconto
798
799 If FindFunction("STWVldVlFinal")
800     lRet := STWVldVlFinal(nGetVlrTot, nGetTotOrig , nVlrAdicio ) //Valor recebido do titulo maior que o valor do titulo.
801 EndIf
802
803 If lRet // Chamamos no final para atualizar os valores pois os caixas estavam esquecendo de clicar em atualiza valores
804     STIUpdTit()
805 EndIf
806
807 Return lRet
808
809 //-----
810 /*/{Protheus.doc} STIExcMHX()
811 Função responsavel por enviar os dados para exclusão nas tabelas MHJ e MHK
812 @Paran oObjRec - Objeto com os titulos a serem cancelados
813 @return
814 /*/
815 //-----
816 Static Function STIExcMHX(oObjRec)
817 Local nTamMHJ_PRXTIT := TAMSX3("MHJ_PRXTIT")[1] //Variavel que armazena o tamanho do campo MHJ_PRXTIT
818 Local nTamMHJ_NUMTIT := TAMSX3("MHJ_NUMTIT")[1] //Variavel que armazena o tamanho do campo MHJ_NUMTIT
819 Local nTamMHJ_PARTIT := TAMSX3("MHJ_PARTIT")[1] //Variavel que armazena o tamanho do campo MHJ_PARTIT
820 Local nX              := 0                      // Variavel para For
821 Local cChvDel          := ""                     //Chave para exclusão
822
823 If ExistFunc("STDGrvMHX")

```

```

824
825     For nX := 1 To len(oObjRec:aListTitles[4]) //Pego a lista de titulos baixados
826         If oObjRec:aListTitles[4][nX][1]
827             //MHJ_FILIAL+MHJ_PRXTIT+MHJ_NUMTIT+MHJ_PARTIT
828             cChvDel := xFilial("MHJ") + PadR(oObjRec:aListTitles[4][nX][2],nTamMHJ_PRXTIT) + PadR(oObjRec:aListTitles[4][nX][3],nTamMHJ_NUMTIT) +
            PadR(oObjRec:aListTitles[4][nX][4],nTamMHJ_PARTIT)
829             STDGrvMHX(,cChvDel)
830         EndIf
831     Next nX
832
833 Else
834     LjGrvLog("STBIncMHX","Função STDGrvMHX não existe no RPO, a ausencia desta função implicara na conferência de caixa.")
835 EndIf
836
837 Return
838
839
840 //-----
841 /*{Protheus.doc} STIAtuVlFinal()
842 Atualiza valor do campo "Valor Final" com base nos campos Valor Original, Multa, Juros, Desconto
843
844 @return
845 */
846 //-----
847 Static Function STIAtuVlFinal()
848     Local lRet      := .T.
849     Local cCash      := xNumCaixa()                // Caixa atual
850     Local nVlrAdicio := ( nGetJuros + nGetMulta + nGetVlrAcres ) - nGetDesconto // Valores adicionais Multa, Jutos, Desconto
851     Local nPercDesc   := ( nGetDesconto * 100 ) / nGetTotOrig // Percentual do desconto
852     Local lValidDesc  := nPercDesc <= 99.99
853
854     If lValidDesc
855
856         lValidDesc := STFPROFILE( 11 , cCash , nPercDesc , nGetDesconto , "T" )[1] // valida se caixa tem permissao para conceder o desconto informado
857
858         If lValidDesc
859
860             If FindFunction("STWVldVlFinal")
861                 lRet := STWVldVlFinal(nGetVlrTot, nGetTotOrig , nVlrAdicio ) //Valor recebido do titulo maior que o valor do titulo.
862             EndIf
863
864             If lRet // Chamamos no final para atualizar os valores pois os caixas estavam esquecendo de clicar em atualiza valores
865                 STIUpdTit(,nGetDesconto,nGetMulta,nGetJuros)
866                 STIAtlValView()
867             EndIf
868
869         EndIf
870
871     EndIf
872
873     If !lValidDesc
874         STFMessage(ProcName(0), "STOP", STR0027) //"Valor de desconto informado MAIOR que o permitido para o caixa atual."

```

```
875     STFShowMessage(ProcName(0))
876     lRet := .F.
877 EndIf
878
879 Return lRet
```

Componentes/Interface/STISalesmanSelection.prw

236 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE 'FWBROWSE.CH'
3 #INCLUDE "PARMTYPE.CH"
4 #INCLUDE "FWEVENTVIEWCONSTS.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE "FWMVCDEF.CH"
7 #INCLUDE "POSCSS.CH"
8 #INCLUDE "STPOS.CH"
9 #INCLUDE "STISALESMANSELECTION.CH"
10
11 Static aRecno      := {}
12 Static lVendSel    := .F. //Indica se a tela de vendedor ja foi solicitada
13
14 //-----
15 /*{Protheus.doc} STISalesmanSelection
16 Funcao responsavel por chamar a troca do painel atual pelo painel de selecao de Vendedores.
17 @param
18 @return
19 */
20 //-----
21
22 Function STISalesmanSelection()
23
24 STIExchangePanel( { || STIPaSalesmanSelection() } )
25
26 Return
27
28 //-----
29 /*{Protheus.doc} STIPaSalesmanSelection
30 Funcao responsavel pela criacao do Painel de selecao de Vendedores.
31 @param
32 @return oMainPanel - Objeto contendo o painel principal de selecao de Vendedores.
33 */
34 //-----
35
36 Function STIPaSalesmanSelection()
37 Local oPanItCancel    := Nil           // Painel Cancelamento de Venda
38 Local oPanBtnSale     := Nil           // Painel de Botões da Venda
39 Local oPanelMVC       := STIGetPanel() // Objeto do Panel onde a interface de selecao de Vendedores sera criada
40 Local oMainPanel      := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
41
42 Local oLblCab         := Nil           // Objeto do label "Seleção de Vendedor"
43
44 /*/
45     Get Combo de Busca
46 /*/
47 Local oLblList        := Nil
48 Local oGetList        := Nil           // TComboBox Escolha do campo a ser utilizado na busca
49 Local cGetList        := ""           // Variável do Get
50
51
52 Local oLblCPFList     := Nil           // Label do ComboBox

```

```

53 Local oGetCPFList      := Nil          // TComboBox Escolha do campo a ser utilizado na busca
54 Local cGetCPFList      := ""          // Variável do Get
55
56 /*/
57   Get de Busca
58 /*/
59 Local oLblSearch        := Nil          // Label do ComboBox
60 Local oGetSearch        := Nil          // TGet busca
61 Local cGetSalesman      := Space(40)   // Variável do Get
62 Local nGetVerPerc       := oMainPanel:nHeight/2 * 0.1 // Percentual da Posição Vertical do Get
63 Local nGetHorPerc       := oMainPanel:nWidth/2 * 0.5 // Percentual da Posição Horizontal do Get
64 Local nGetHeiPerc       := 0.03        // Percentual de Altura do Get
65
66 /*/
67   Botão "Selecionar Vendedor"
68 /*/
69 Local oButton           := Nil          // Botao "Selecionar Vendedor"
70 Local nBtnWidPerc       := 0.25        // Percentual de Largura do Botão "Selecionar Vendedor"
71 Local nBtnHeiPerc       := 0.08        // Percentual de Altura do Botão "Selecionar Vendedor"
72 Local nBtnVerPerc       := 0.8         // Percentual da Posição Vertical do Botão "Selecionar Vendedor"
73 Local nBtnHorPerc       := 0.65        // Percentual da Posição Horizontal do Botão "Selecionar Vendedor"
74
75 /*/
76   Posicoes do oSalesmanPanel
77 /*/
78 Local nCustPanHor       := 15           // Posicao horizontal de oSalesmanPanel
79 Local nCustPanVer       := oMainPanel:nHeight/2 * 0.3 // Posicao vertical de oSalesmanPanel
80
81 Local oListFont         := TFont():New("Courier New") // Fonte utilizada no listbox
82 Local oSalesmanPanel    := Nil          // Objeto do panel onde será criado o Browse com as informacoes do Vendedor.
83
84 Local bConfirm          := { || STIFilSalesmanData(oGetList,oGetSearch)}
85 Local oButCan           := Nil //Botao cancelar
86 Local oButPesq          := Nil //Botao pesquisar
87 Local lMobile           := STFGetCfg("lMobile", .F.) //Smart Client Mobile
88 /*/
89   Objetos
90 /*/
91
92 oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oMainPanel,,,,,.T.,,,) //"Seleção de Vendedor"
93 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
94
95 oLblSearch := TSay():New(POSVERT_LABEL1,POSHOR_1, {||STR0002}, oMainPanel,,,,,.T.) //"Pesquisar Vendedor:"
96
97 oGetSearch:= TGet():New(POSVERT_GET1,POSHOR_1,{|u| If(PCount())>0,cGetSalesman:=u,cGetSalesman)};;
98   oMainPanel,120 ,ALTURAGET,"@!",{|| If( Empty(cGetSalesman), oButton:SetFocus(), Nil ) },,,,,.T.,,,,,,,,,,"cGetSalesman")
99 oGetSearch:SetCSS( POSCSS (GetClassName(oGetSearch), CSS_GET_NORMAL ))
100 oGetSearch:bLostFocus := { || IIF( !STISalManFilter(cGetSalesman,oGetList,aRecno),oGetSearch:SetFocus(),Nil)}
101
102 //Botao pesquisar para versao mobile
103 If ValType(lMobile) == "L" .AND. lMobile
104   oButPesq := TButton():New( POSVERT_GET1,POSHOR_1+130,"",oMainPanel,, ;

```

```

105             20,20,,,,.T.,,,,{||.T.})
106         oButPesq:SetCSS( POSCSS (GetClassName(oButPesq), CSS_BTN_LUPA ))
107 EndIf
108
109 oLblSearch:SetCSS( POSCSS (GetClassName(oLblSearch),CSS_LABEL_FOCAL ))
110
111 oLblList := TSay():New(POSVERT_LABEL2,POSHOR_1, {||STR0003}, oMainPanel,,,,,.T.) //"Código / Nome / CPF/CNPJ"
112 oLblList:SetCSS( POSCSS (GetClassName(oLblList),CSS_LABEL_FOCAL ))
113
114 //ListBox
115 oGetList := TListBox():Create(oMainPanel, POSVERT_GET2, POSHOR_1, {||u| If(PCount(>0,cGetList:=u,cGetList)}, , LARG_LIST_CONSULT ,
    ALT_LIST_CONSULT,,,,.T.,,bConfirm,oListFont)
116 oGetList:SetCSS( POSCSS (GetClassName(oGetList),CSS_LISTBOX ))
117
118 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0004,oMainPanel,bConfirm, ;
    LARGBTN,ALTURABTN,,,,.T.) //"Selecionar Vendedor"
119
120
121 oButton:SetCSS( POSCSS (GetClassName(oButton),CSS_BTN_FOCAL ))
122
123 oButCan := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0007,oMainPanel,{|| STIRegItemInterface() }, ; //"Cancelar"
    LARGBTN,ALTURABTN,,,,.T.,,,,{||.T.})
124
125 oButCan:SetCSS( POSCSS (GetClassName(oButCan), CSS_BTN_NORMAL ))
126
127 oGetSearch:SetFocus()
128
129 Return oMainPanel
130
131 //-----
132 /* {Protheus.doc} STIFilSalesmanData
133 Funcao responsavel por setar os dados do Vendedor selecionado na tabela SL1, além de carregar um model com as informações do Vendedor
134 @param oGetList - Objeto de escolha do campo a ser utilizado na busca
135 @return
136 */
137 //-----
138
139 Function STIFilSalesmanData(oGetList,oGetSearch)
140 Local oModelVen := Nil // Model do Vendedor selecionado
141 Local nPos := oGetList:GetPos() // Posicao da selecao
142 Local nRecno := 0 // Recno do Registro
143 Local cVenPad := SuperGetMv( "MV_VENDPAD" ) // Vendedor Padrao
144 Local nAtuVen := SuperGetMv("MV_LJATVEN",,0) // Define se atualiza L2_VEND apos trocar vendedor
145 Local lSTValidVen := ExistBlock("STValidVen") //Ponto de entrada para validar o vendedor
146 Local lNext := .T. //Logico dizendo se continua o processamento ou nao
147 Local cCodVen := '' //Codigo do vendedor para o ponto de entrada
148 Local aRetVal := {} //Retorno do ponto de entrada
149 Local cFrase := '' //Frase a ser exibida do ponto de entrada
150 Local nTotItSL2 := 0
151 Local nItem := 0
152
153 If lSTValidVen
154     If nPos > 0 .AND. !Empty(aRecno)
155         nRecno := aRecno[nPos]

```

```

156      SA3->(DbGoTo(nRecno))
157      cCodVen := SA3->A3_COD
158  Else
159      cCodVen := cVenPad
160  EndIf
161  aRetValid := ExecBlock("STValidVen",.F.,.F.,{cCodVen})
162  lNext := aRetValid[1]
163  cFrase := aRetValid[2]
164 EndIf
165
166 If lNext
167   If nPos > 0 .AND. !EMPTY(aRecno)
168     nRecno := aRecno[nPos]
169     SA3->(DbGoTo(nRecno))
170
171     oModelVen := STWSalesmanSelection(SA3->A3_COD)
172     STDSPBasket("SL1","L1_VEND" ,oModelVen:GetValue("SA3MASTER","A3_COD"))
173     STDSPBasket("SL1","L1_VEND2" ,oModelVen:GetValue("SA3MASTER","A3_SUPER"))
174     STDSPBasket("SL1","L1_VEND3" ,oModelVen:GetValue("SA3MASTER","A3_GEREN"))
175     STDSPBasket("SL1","L1_COMIS" ,STDGComission( oModelVen:GetValue("SA3MASTER","A3_COD") ))
176
177     If nAtuVen == 1
178       nTotItSL2 := STDPBLength("SL2")
179       For nItem := 1 to nTotItSL2
180         If Empty(STDGPBasket("SL2","L2_NUMORIG", nItem)) //Valida se o item e de um orcamento importado, nao permitindo alterar o vendedor
181           STDSPBasket( "SL2", "L2_VEND", oModelVen:GetValue("SA3MASTER","A3_COD"), nItem )
182           STDSaveSale(nItem)
183         EndIf
184       Next
185     EndIf
186
187     STISSelVend(.T.) //Indica que a tela de vendedor ja foi solicitada, para nao solicitar mais nesta venda.
188     STFMessage(ProcName(),"STOP",STR0005) //"Vendedor Selecionado"
189     STFShowMessage(ProcName())
190   Else
191     STDSPBasket("SL1","L1_VEND" ,cVenPad)
192     STFMessage(ProcName(),"STOP",STR0006) //"Nenhum Vendedor foi selecionado. Será utilizado o Vendedor padrão."
193     STFShowMessage(ProcName())
194     STFCleanMessage(ProcName())
195   EndIf
196
197   aRecno := {}
198
199   STIBtnActivate() //Ativa os botoes da tela
200   STIExchangePanel( { || STIPanItemRegister() } )
201 Else
202   STFMessage(ProcName(),"STOP",cFrase)
203   STFShowMessage(ProcName())
204   oGetSearch:SetFocus()
205 EndIf
206
207

```

```
208 Return
209
210
211 //-----
212 /*{Protheus.doc} STIGSelVend
213 Retorna o valor da variavel static lVendSel, que indica se a tela de vendedor ja foi solicitada.
214
215 @return      lVendSel - Indica se a tela de vendedor ja foi solicitada
216 @obs
217 @sample
218 */
219 //-----
220 Function STIGSelVend()
221 Return lVendSel
222
223 //-----
224 /*{Protheus.doc} STISSelVend
225 Controla o valor da variavel static lVendSel, que indica se a tela de vendedor ja foi solicitada.
226
227 @param      lSet .T. -> Vendedor ja selecionado (nao apresenta mais a tela de vendedor)
228              .F. -> Vendedor nao selecionado ainda (apresentar tela para seleção)
229 @return      Nil
230 @obs
231 @sample
232 */
233 //-----
234 Function STISSelVend(lSet)
235 lVendSel := lSet
236 Return Nil
```

Componentes/Interface/STISupplyBleeding.prw

508 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STISUPPLYBLEEDING.CH"
5
6 #DEFINE TYPEOPERATION      1           // 01 - Tipo da rotina: (1) Sangria / Entrada de troco (2)
7 #DEFINE CASHIER_ORIGIN     2           // 02 - Codigo do caixa de origem
8 #DEFINE CASHIER_DESTINY    3           // 03 - Codigo do caixa de destino
9 #DEFINE VALUE_MONEY        4           // 04 - Valor em dinheiro
10 #DEFINE VALUE_CHECK        5           // 05 - Valor em cheque
11 #DEFINE VALUE_CREDITCARD   6           // 06 - Valor em cartao de credito
12 #DEFINE VALUE_DEBITCARD    7           // 07 - Valor em cartao de debito
13 #DEFINE VALUE_FINANCED     8           // 08 - Valor em financiado
14 #DEFINE VALUE_COVENANT     9           // 09 - Valor em convenio
15 #DEFINE VALUE_COUPONS      10          // 10 - Valor em vales
16 #DEFINE VALUE_OTHERS       11          // 11 - Valor em outros
17 #DEFINE AGENCY             12          // 12 - Numero da agencia
18 #DEFINE DV_AGENC           13          // 13 - Digito da agencia
19 #DEFINE ACCOUNT            14          // 14 - Numero da conta
20 #DEFINE DV_ACCOUNT         15          // 15 - Digito da conta
21 #DEFINE VALUE_PD           16          // 16 - Pagamento Digital
22 #DEFINE VALUE_PIX           17          // 17 - PIX
23
24 //-----
25 /*{Protheus.doc} STISupplyBleeding
26 Verifica se caixa existe(tratamento multi-moeda).
27 0 portador do titulo(caixa) sera definido pela moeda
28
29 @param lOnlyMoney - Operacao apenas em dinheiro
30 @return lRet      Retorno se executou corretamente a funcao
31 @obs
32 @sample
33 */
34 //-----
35
36 Function STISupplyBleeding( cOption, lOnlyMoney )
37
38 Local nMV_LJ350MV := SuperGetMv("MV_LJ350MV",,.T.) // Na rotina de sangria de caixa, para habilitar os campos que permitirão edição. 1 = Apenas dinheiro; 2 = Todos
39
40 Default lOnlyMoney := .F. //Somente Dinheiro
41
42 IIF(nMV_LJ350MV == 1, lOnlyMoney := .T., )
43
44 If (STFProFile(5)[1])
45     If cOption == "1" //Sangria
46         STIExchangePanel( { || STIPanSupplyBleeding(lOnlyMoney) } )
47     ElseIf cOption == "2"//Suprimento/Troco
48         STIExchangePanel( { || STICashierSupply() } )
49     EndIf
50 Else
51     STFMessage(ProcName(),"STOP",STR0021) //"Usuario sem permissao para realizar Sangria / Entrada de Troco."
52     STFShowMessage(ProcName())

```

```

53 EndIf
54
55 Return
56
57 //-----
58 /*{Protheus.doc} STIPanSupplyBleeding
59 Interface de sangria de suprimentos
60 @param lOnlyMoney - Sangria apenas em dinheiro
61 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
62 */
63 //-----
64 Static Function STIPanSupplyBleeding(lOnlyMoney)
65 Local oPanelMVC      := STIGetDlg()      //Painel principal do dialog
66 Local oMainPanel    := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Painel Abertura de Caixa
67
68 Local nLargGets      := 160
69
70 Local oCab           := Nil
71
72 Local oLblDestino    := Nil
73 Local oLblBanco      := Nil
74 Local oDestino       := Nil
75
76 Local oLblDinheiro   := Nil
77 Local oDinheiro      := Nil
78 Local nDinheiro      := 0              //Alteracao do conteudo do get
79
80 Local oLblCheque     := Nil
81 Local oCheque        := Nil
82 Local nCheque        := 0              //Alteracao do conteudo do get
83
84 Local oLblCDebito    := Nil
85 Local oCDebito      := Nil
86 Local nCDebito      := 0              //Alteracao do conteudo do get
87
88 Local oLblCCredito   := Nil
89 Local oCCredito      := Nil
90 Local nCCredito      := 0              //Alteracao do conteudo do get
91
92 Local oLblFinanciado := Nil
93 Local oFinanciado    := Nil
94 Local nFinanciado    := 0              //Alteracao do conteudo do get
95
96 Local oLblConven     := Nil
97 Local oConven        := Nil
98 Local nConven        := 0              //Alteracao do conteudo do get
99
100 Local oLblVales      := Nil
101 Local oVales         := Nil
102 Local nVales         := 0              //Alteracao do conteudo do get
103
104 Local oLblOutros     := Nil

```

```

105 Local oOutros      := Nil
106 Local nOutros      := 0                //Alteracao do conteudo do get
107
108 Local oLblPD        := Nil
109 Local oPD           := Nil
110 Local nPD           := 0
111 Local nPIX          := 0
112
113 Local oBtnConfirm   := Nil
114 Local oBtnCancel    := Nil
115
116 Local aStation      := STBInfoEst( 1, .T. )
117 Local cSeparator    := ") "
118 Local aCaixas       := STIGetSA6(aStation[1])
119 Local aAuxCaixas    := {}
120 Local nX            := 0
121 Local nTamSA6       := 0                //Tamanho do codigo do caixa
122 Local cComboCx      := ""              //Variavel de combo de caixas
123
124 Local nVertCab       := oPanelMVC:nHeight/81.3    // Posicao vertical do cabecalho
125 Local nVertLbl1      := oPanelMVC:nHeight/23.228  // Posicao vertical dos label da linha 1
126 Local nVertLbl2      := oPanelMVC:nHeight/18.228  // Posicao vertical dos label da linha 2
127 Local nVertLbl3      := oPanelMVC:nHeight/9.0333   // Posicao vertical dos label da linha 3
128 Local nVertLbl4      := oPanelMVC:nHeight/5.6068   // Posicao vertical dos label da linha 4
129 Local nVertLbl5      := oPanelMVC:nHeight/4.065    // Posicao vertical dos label da linha 5
130 Local nVertLbl6      := oPanelMVC:nHeight/3.18823   // Posicao vertical dos label da linha 6
131 Local nVertLbl7      := oPanelMVC:nHeight/2.700    // Posicao vertical dos label da linha 6
132 Local nVertGet1      := oPanelMVC:nHeight/14.781   // Posicao vertical dos get da linha 1
133 Local nVertGet2      := oPanelMVC:nHeight/7.3909   // Posicao vertical dos get da linha 2
134 Local nVertGet3      := oPanelMVC:nHeight/4.92727  // Posicao vertical dos get da linha 3
135 Local nVertGet4      := oPanelMVC:nHeight/3.69545  // Posicao vertical dos get da linha 4
136 Local nVertGet5      := oPanelMVC:nHeight/2.95636  // Posicao vertical dos get da linha 5
137 Local nVertGet6      := oPanelMVC:nHeight/2.51782  // Posicao vertical dos get da linha 5
138 Local lAlterCx       := !(SuperGetMv( "MV_LJALTCX",,.T.))
139 Local lMobile        := STFCfg( "lMobile" , .F. ) // Versao mobile
140 Local nCaixas        := 0
141 Local nRes            := 0
142
143 Default lOnlyMoney := .F. //Somente Dinheiro
144
145 nRes                  := GetScreenRes()[1] // Recupera Largura atual da tela
146 Do Case
147     Case nRes <= 1023
148         nLargGets := 80
149     Case nRes <= 1679
150         nLargGets := 100
151 EndCase
152
153 lMobile := ValType(lMobile) == "L" .AND. lMobile
154
155 If Findfunction( 'STBCashList' )
156     aAuxCaixas := STBCashList()

```

```

157 Else
158     nTamSA6 := At("/", "MV_CXLOJA") -1
159     If nTamSA6 <= 0
160         nTamSA6 := SA6->(TamSx3("A6_COD"))[1]
161     EndIf
162
163     cComboCx := Left(GetMv("MV_CXLOJA"),nTamSA6 )
164
165     For nCaixas := 1 to len(aCaixas)
166         AADD(aAuxCaixas,aCaixas[nCaixas][1][1])
167     Next
168 EndIf
169
170 oMainPanel:SetCSS( POSCSS (GetClassName(oMainPanel), CSS_PANEL_CONTEXT ))
171
172 oCab := TSay():New(nVertCab,POSHOR_1,{||STR0001+aStation[1]},oMainPanel,,,,,.T,,,,) //"Sangria de Caixa | Caixa Origem: "
173 oCab:SetCSS( POSCSS (GetClassName(oCab), CSS_BREADCUMB ))
174
175 oLbldestino:= TSay():New(nVertLb1,POSHOR_1,{||STR0002},oMainPanel,,,,,.T,,,,) //"Caixa Destino"
176 oLbldestino:SetCSS( POSCSS (GetClassName(oLbldestino), CSS_LABEL_FOCAL ))
177
178 oLb1Banco:= TSay():New(nVertLb1,POSHOR_1,{||STR0022},oMainPanel,,,,,.T,,,,) //"Codigo | Agencia | Dv. Agencia | Conta | DV. Conta"
179 oLb1Banco:SetCSS( POSCSS (GetClassName(oLbldestino), CSS_LABEL_FOCAL ))
180
181 oDestino := TComboBox():Create(oMainPanel, {||u| if( Pcount( )>0, cComboCx := u, cComboCx) }, nVertGet1, POSHOR_1, ;
182     aAuxCaixas, nLargGets, ALTURAGET,,;
183     ,,,,;
184     .T.,,,,;
185     ,,,,;
186     ,cComboCx)
187
188 oDestino:SetCSS( POSCSS (GetClassName(oDestino), IIF(!Mobile ,CSS_COMBOBOX ,CSS_LISTBOX) ))
189 oDestino:lReadOnly := !AlterCx
190
191 oLb1Dinheiro:= TSay():New(nVertLb1,POSHOR_1,{||STR0003},oMainPanel,,,,,.T,,,,) ////"Dinheiro"
192 oLb1Dinheiro:SetCSS( POSCSS (GetClassName(oLb1Dinheiro), CSS_LABEL_FOCAL ))
193
194 @ nVertGet2,POSHOR_1 MSGET oDinheiro VAR nDinheiro SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
195 oDinheiro:SetCSS( POSCSS (GetClassName(oDinheiro), CSS_GET_FOCAL ))
196 oDinheiro:lNoButton := .T. //nao mostra calculadora
197
198 If !lOnlyMoney
199
200     oLb1Cheque:= TSay():New(nVertLb1,POSHOR_2,{||STR0004},oMainPanel,,,,,.T,,,,) //"Cheques"
201     oLb1Cheque:SetCSS( POSCSS (GetClassName(oLb1Cheque), CSS_LABEL_FOCAL ))
202
203     @ nVertGet2,POSHOR_2 MSGET oCheque VAR nCheque SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
204     oCheque:SetCSS( POSCSS (GetClassName(oCheque), CSS_GET_FOCAL ))
205
206     oLb1CDebito:= TSay():New(nVertLb1,POSHOR_1,{||STR0005},oMainPanel,,,,,.T,,,,) //"Cartão Débito"
207     oLb1CDebito:SetCSS( POSCSS (GetClassName(oLb1CDebito),CSS_LABEL_FOCAL))
208

```

```

209 @ nVertGet3,POSHOR_1 MSGET oCDebito VAR nCDebito SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
210 oCDebito:SetCSS( POSCSS (GetClassName(oCDebito),CSS_GET_FOCAL))
211
212 oLblCCredito:= TSay():New(nVertLbl4,POSHOR_2,{||STR0006},oMainPanel,,,,,.T,,,,) ////"Cartão Crédito"
213 oLblCCredito:SetCSS( POSCSS (GetClassName(oLblCCredito),CSS_LABEL_FOCAL))
214
215 @ nVertGet3,POSHOR_2 MSGET oCCredito VAR nCCredito SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
216 oCCredito:SetCSS( POSCSS (GetClassName(oCCredito), CSS_GET_FOCAL))
217
218 oLblFinanciado:= TSay():New(nVertLbl5,POSHOR_1,{||STR0007},oMainPanel,,,,,.T,,,,) //"Financiado"
219 oLblFinanciado:SetCSS( POSCSS (GetClassName(oLblFinanciado), CSS_LABEL_FOCAL))
220
221 @ nVertGet4,POSHOR_1 MSGET oFinanciado VAR nFinanciado SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
222 oFinanciado:SetCSS( POSCSS (GetClassName(oFinanciado), CSS_GET_FOCAL))
223
224 oLblConven:= TSay():New(nVertLbl5,POSHOR_2,{||STR0008},oMainPanel,,,,,.T,,,,) //"Convênio"
225 oLblConven:SetCSS( POSCSS (GetClassName(oLblConven), CSS_LABEL_FOCAL))
226
227 @ nVertGet4,POSHOR_2 MSGET oConven VAR nConven SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
228 oConven:SetCSS( POSCSS (GetClassName(oConven), CSS_GET_FOCAL))
229
230 oLblVales:= TSay():New(nVertLbl6,POSHOR_1,{||STR0009},oMainPanel,,,,,.T,,,,) //"Vales"
231 oLblVales:SetCSS( POSCSS (GetClassName(oLblVales), CSS_LABEL_FOCAL))
232
233 @ nVertGet5,POSHOR_1 MSGET oVales VAR nVales SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
234 oVales:SetCSS( POSCSS (GetClassName(oVales), CSS_GET_FOCAL))
235
236 oLblPD:= TSay():New(nVertLbl6,POSHOR_2,{||STR0023 },oMainPanel,,,,,.T,,,,) //"Pagamento Digital"
237 oLblPD:SetCSS( POSCSS (GetClassName(oLblPD), CSS_LABEL_FOCAL))
238
239 @ nVertGet5,POSHOR_2 MSGET oPD VAR nPD SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
240 oPD:SetCSS( POSCSS (GetClassName(oPD), CSS_GET_FOCAL))
241
242 oLblPIX:= TSay():New(nVertLbl7,POSHOR_1,{||STR0024},oMainPanel,,,,,.T,,,,) //"PIX"
243 oLblPIX:SetCSS( POSCSS (GetClassName(oLblPD), CSS_LABEL_FOCAL))
244
245 @ nVertGet6,POSHOR_1 MSGET oPIX VAR nPIX SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
246 oPIX:SetCSS( POSCSS (GetClassName(oPIX), CSS_GET_FOCAL))
247
248 oLblOutros:= TSay():New(nVertLbl7,POSHOR_2,{||STR0010},oMainPanel,,,,,.T,,,,) //"Outros"
249 oLblOutros:SetCSS( POSCSS (GetClassName(oLblOutros), CSS_LABEL_FOCAL))
250
251 @ nVertGet6,POSHOR_2 MSGET oOutros VAR nOutros SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
252 oOutros:SetCSS( POSCSS (GetClassName(oOutros), CSS_GET_FOCAL))
253
254 EndIf
255
256 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,;
257 POSHOR_BTNFOCAL ;;
258 STR0011,; //"Confirmar"
259 oMainPanel ;;
260 {|| STIConfirmBleeding(1,cComboCx,nDinheiro, nCheque, nCDebito, nCCredito,nFinanciado,nConven,nVales,nOutros,aCaixas,nPD,nPIX, oMainPanel)},;

```

```

261          LARGBTN      ;;
262          ALTURABTN    ;;
263          ,,,.T.,;
264          ,,,{|| !Empty(cComboCx)})
265
266 oBtnCancel := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0020,oMainPanel,{ || Iif(ExistFunc("StiMataObj"),StiMataObj(oMainPanel),Nil), IIf(STB0openCash(),
          STIRegItemInterface(), STIExchangePanel( { || STIPanOpenCash() } )) }, ; //"Cancelar"
          LARGBTN,ALTURABTN,,,,.T. )
267
268
269
270 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL))
271
272 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel), CSS_BTN_NORMAL))
273
274 oDestino:SetFocus()
275
276 Return(oMainPanel)
277
278 //-----
279 /*{Protheus.doc} STIPanSupplyBleeding
280 Interface de suprimentacao de caixa
281
282 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
283 */
284 //-----
285
286 Static Function STICashierSupply()
287 Local oPanelMVC      := STIGetDlg()      //Painel principal do dialog
288 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Painel Abertura de Caixa
289
290 Local nLargGets      := 90
291
292 Local oCab           := Nil
293
294 Local oLblOrigem     := Nil
295 Local oOrigem        := Nil
296
297 Local oLblDinheiro   := Nil
298 Local oDinheiro      := Nil
299 Local nDinheiro      := 0                //Alteracao do conteudo do get
300
301 Local aStation       := STBInfoEst( 1, .T. )
302 Local cSeparator     := ") "
303 Local aCaixas        := STIGetSA6(aStation[1])
304 Local aAuxCaixas     := {}
305 Local nX             := 0
306
307 Local oBtnConfirm    := Nil
308 Local oBtnCancel     := Nil
309 Local nTamSA6        := 0
310 Local cComboCx       := ""
311 Local lMobile        := STFGetCfg( "lMobile" , .F. ) //versao mobile

```

```

312 Local lAlterCx      := SuperGetMv("MV_LJALTCX",,.T.) //permite ou não a seleção do caixa de origem do suprimento de caixa.
313
314
315 Local nCaixas        := 0
316 lMobile := ValType(lMobile) == "L" .AND. lMobile
317
318 If Findfunction( 'STBCashList' )
319     aAuxCaixas := STBCashList()
320 Else
321     nTamSA6 := At("/", "MV_CXLOJA") -1
322     If nTamSA6 <= 0
323         nTamSA6 := SA6->(TamSx3("A6_COD"))[1]
324     EndIf
325
326     cComboCx := Left(GetMv("MV_CXLOJA"),nTamSA6 )
327
328     For nCaixas := 1 to len(aCaixas)
329         AADD(aAuxCaixas,aCaixas[nCaixas][1][1])
330     Next
331 EndIf
332
333 oMainPanel:SetCSS( POSCSS (GetClassName(oMainPanel), CSS_PANEL_CONTEXT ))
334
335 oCab      := TSay():New(10,POSHOR_1,{||STR0012+aStation[1]},oMainPanel,,,,,.T.,,,) //"Suprimento de Caixa | Caixa Destino: "
336 oCab:SetCSS( POSCSS (GetClassName(oCab), CSS_BREADCUMB))
337
338 oLblOrigem:= TSay():New(35,POSHOR_1,{||STR0013},oMainPanel,,,,,.T.,,,) //"Caixa Origem"
339 oLblOrigem:SetCSS( POSCSS (GetClassName(oLblOrigem), CSS_LABEL_FOCAL))
340
341 oOrigem := TComboBox():Create(oMainPanel, {||u| if( Pcount( )>0, cComboCx := u, cComboCx )}, 55, POSHOR_1, ;
342     aAuxCaixas, nLargGets, ALTURAGET,,;
343     ,,,;
344     .T.,,,;
345     ,,,;
346     ,cComboCx)
347 oOrigem:SetCSS( POSCSS (GetClassName(oOrigem), IIF(lMobile ,CSS_COMBOBOX ,CSS_LISTBOX) ))
348 oOrigem:lReadOnly := !lAlterCx
349
350
351 oLblDinheiro:= TSay():New(90,POSHOR_1,{||STR0014},oMainPanel,,,,,.T.,,,) //"Dinheiro"
352 oLblDinheiro:SetCSS( POSCSS (GetClassName(oLblDinheiro), CSS_LABEL_FOCAL))
353
354 @ 110,POSHOR_1 MSGET oDinheiro VAR nDinheiro SIZE nLargGets,ALTURAGET PICTURE "@E 99,999,999.99" OF oMainPanel PIXEL
355 oDinheiro:SetCSS( POSCSS (GetClassName(oDinheiro), CSS_GET_FOCAL))
356 oDinheiro:lNoButton := .T. //nao mostra calculadora
357
358 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,;
359     POSHOR_BTNFOCAL ,;
360     STR0011,; //"Confirmar"
361     oMainPanel ,;
362     {|| STIConfirmBleeding(2,cComboCx,nDinheiro,,,,,,aCaixas,,oMainPanel)},;
363     LARGBTN ,;

```

```

364             ALTURABTN      ,;
365             ,,,.T.;;
366             ,,,{|| !Empty(cComboCx)})
367
368 oBtnCancel := TButton():New(    POSVERT_BTNFOCAL,POSHOR_1,STR0020,oMainPanel,{ || IIf(STBOpenCash(),STIRegItemInterface(),STIExchangePanel( { || STIPanOpenCash() } )),
    IIf(ExistFunc("STIMataObj"),STIMataObj(oMainPanel),Nil) }, ; //"Cancelar"
369             LARGBTN,ALTURABTN,,,,.T.    )
370
371 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL))
372 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel),CSS_BTN_NORMAL))
373
374 oOrigem:SetFocus()
375
376 Return oMainPanel
377
378 //-----
379 /*{Protheus.doc} STIConfirmBleeding
380 Funcao responsavel pela criacao do Painei de selecao de clientes.
381
382 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
383 */
384 //-----
385
386 Static Function STIConfirmBleeding(nOpc,cCaixa,nDinheiro, nCheque, nCDebito, nCCredito,nFinanciado,nConven,nVales,nOutros,aCaixas,nPD,nPIX, oMainPanel)
387 Local nCurrency      := STBGetCurrency()
388 Local aValues        := Array(17)
389 Local lRet           := .T.
390 Local aStation       := STBInfoEst( 1, .T. )
391 local nCaixas        := 0           //Variavel de loop
392 Local cAgencia       := ""          //Variavel que armazena a Agencia recebita pelo aCaixas
393 Local cDvage         := ""          //Variavel que armazena o Digito da agencia recebita pelo aCaixas
394 Local cConta         := ""          //Variavel que armazena a Conta recebita pelo aCaixas
395 Local cDvConta       := ""          //Variavel que armazena o Digito da conta recebita pelo aCaixas
396
397 DEFAULT nCheque      := 0
398 DEFAULT nCDebito     := 0
399 DEFAULT nCCredito    := 0
400 DEFAULT nFinanciado  := 0
401 DEFAULT nConven      := 0
402 DEFAULT nVales       := 0
403 DEFAULT nOutros      := 0
404 Default aCaixas      := {}
405 Default nPD          := 0
406 Default nPIX         := 0
407 Default oMainPanel   := Nil
408
409 If !Empty(cCaixa)
410
411     For nCaixas := 1 to len(aCaixas)
412         If AllTrim(cCaixa) == AllTrim(aCaixas[nCaixas][1][1])
413             cAgencia := aCaixas[nCaixas][2][1]
414             cDvage    := aCaixas[nCaixas][2][2]

```

```

415         cConta      := aCaixas[nCaixas][2][3]
416         cDvConta    := aCaixas[nCaixas][2][4]
417         Exit
418     EndIf
419 Next
420
421 cCaixa := SubSTR(cCaixa,1,TamSX3("A6_COD")[1])//Extraio apenas o A6_COD
422
423 If AllTrim(Upper(cCaixa)) <> AllTrim(Upper(aStation[1]))
424     aValues[TYPEOPERATION] := n0pc
425     aValues[VALUE_MONEY]   := nDinheiro
426     If n0pc == 1
427         aValues[CASHIER_ORIGIN] := aStation[1]
428         aValues[CASHIER_DESTINY] := cCaixa
429         aValues[VALUE_CHECK] := nCheque
430         aValues[VALUE_CREDITCARD] := nCCredito
431         aValues[VALUE_DEBITCARD] := nCDebito
432         aValues[VALUE_FINANCED] := nFinanciado
433         aValues[VALUE_COVENANT] := nConven
434         aValues[VALUE_COUPONS] := nVales
435         aValues[VALUE_OTHERS] := nOutros
436         aValues[VALUE_PD] := nPd
437         aValues[VALUE_PIX] := nPIX
438     ElseIf n0pc == 2
439         aValues[CASHIER_ORIGIN] := cCaixa
440         aValues[CASHIER_DESTINY] := aStation[1]
441     EndIf
442
443     aValues[AGENCY] := cAgencia
444     aValues[DV_AGENCY] := cDvage
445     aValues[ACCOUNT] := cConta
446     aValues[DV_ACCOUNT] := cDvConta
447
448     lRet := STWSupplyBleeding( n0pc , aValues , nCurrency )
449
450     If lRet
451         If n0pc == 1
452             STFMessage(ProcName(),"STOP",STR0015) //"Sangria realizada com sucesso"
453         ElseIf n0pc == 2
454             STFMessage(ProcName(),"STOP",STR0016) //"Suprimentação de caixa realizada com sucesso"
455         EndIf
456     Else
457         STFMessage(ProcName(),"STOP",STR0017) //"Não foi possivel realizar a operação"
458     EndIf
459 Else
460     lRet := .F.
461     STFMessage(ProcName(),"STOP",STR0018) //"0 caixa destino e origem são iguais!"
462 EndIf
463 Else
464     lRet := .F.
465     STFMessage(ProcName(),"STOP",STR0019) //"Informe o caixa destino!"
466 EndIf

```

```

467
468 STFSHMessage(ProcName())
469
470 If lRet
471     Iif(ExistFunc("StiMataObj"),StiMataObj(oMainPanel),Nil)
472     STIRegItemInterface()
473 EndIf
474
475 Return lRet
476
477 //-----
478 /*{Protheus.doc} STIGetSA6
479 Retorna todos os caixas cadastrados na SA6, com exceção do caixa que esta sendo utilizado.
480
481 @param cUsedCashier - Caixa atual.
482 @return aRet - Array que contem os codigos dos caixas que podem ser utilizados.
483 */
484 //-----
485
486 Function STIGetSA6(cUsedCashier)
487 Local aArea      := GetArea()
488 Local aRet       := {}
489 Local cCxFil     := xFilial("SA6")
490
491 DEFAULT cUsedCashier := ""
492
493 DbSelectArea("SA6")
494 SA6->(DbSetOrder(1))
495 SA6->(DbGoTop())
496
497 While SA6->(!EOF())
498
499     If AllTrim(SA6->A6_COD) <> AllTrim(cUsedCashier) .AND. SA6->A6_FILIAL == cCxFil
500         Add(aRet,{SA6->A6_COD + " | " + SA6->A6_AGENCIA + " | " + SA6->A6_DVAGE + " | " + SA6->A6_NUMCON + " | " +
501             SA6->A6_DVCTA},{SA6->A6_AGENCIA,SA6->A6_DVAGE,SA6->A6_NUMCON,SA6->A6_DVCTA})
502     EndIf
503     SA6->(DbSkip())
504 End
505
506 RestArea(aArea)
507
508 Return aRet

```

Componentes/Interface/STIValePresente.PRW

658 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "POSCSS.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STIValePresente.CH"
5
6 Static oPanGetValPre := Nil
7 Static cQtdDesc := ''
8 Static cVale := {}
9 Static oLbllimiteVP := Nil
10 Static aVpMinMax := {}
11 Static oVlrVP := Nil
12 Static nVlrVP := 0
13 Static oLbldescricao := Nil
14 Static cCodVPre := ""
15
16 //-----
17 /*{Protheus.doc} STIValPre
18 Abre a tela de vale presente
19
20 @param
21 @return
22 @obs
23 @sample
24 */
25 //-----
26 Function STIValPre()
27
28 Local lVPProd := SuperGetMv("MV_LJVPPRO",,.F.) //Permite vender Vale Presente e produtos na mesma venda
29
30 If (!lVPProd .And. !STBExistItemFiscal(.F.)) .Or. lVPProd
31     STIChangeCssBtn('oBtnRegItem')
32     STIExchangePanel( { || STIValePresente() } )
33 Else
34     STFMessage(ProcName(0),"STOP",STR0014) //"A venda do vale presente deve ser feita separada."
35     STFShowMessage(ProcName(0))
36 EndIf
37
38 Return .T.
39
40
41 //-----
42 /*{Protheus.doc} STIValePresente
43 Monta a tela de vale presente
44
45 @param
46 @return
47 @obs
48 @sample
49 */
50 //-----
51 Function STIValePresente()
52

```

```

53 Local oPanelMVC := STIGetPanel()
54 Local oLblCab      := Nil
55 Local oLblCodProd  := Nil
56 Local nMddCodigo   := TamSX3('MDD_CODIGO')[1] //tamanho do campo MDD_CODIGO
57 Local cGetProd     := Space(nMddCodigo)
58 Local oLblHelpGet  := Nil
59 Local oCodProd     := Nil
60 Local oLblInfProd  := Nil
61 Local cTipoVp      := "P"
62
63 nVlrVP              := 0
64
65 oPanGetValPre       := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2)
66
67 /* Objetos */
68 oLblCab := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanGetValPre,,,,,.T,,,,) //"Vale Presente"
69 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
70
71 oLblCodProd := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0002},oPanGetValPre,,,,,.T,,,,) //"Código do Vale Presente"
72 oLblCodProd:SetCSS( POSCSS (GetClassName(oLblCodProd), CSS_LABEL_FOCAL ))
73
74 oCodProd := TGet():New(POSVERT_GET1,POSHOR_1,{|u| If(PCount())>0,cGetProd:=u,cGetProd)},oPanGetValPre,170,ALTURAGET,"@!",,,,,,.T,,,,,"cGetProd")
75 oCodProd:SetCSS( POSCSS (GetClassName(oCodProd), CSS_GET_FOCAL ))
76
77 //Venda valor variável do vale presente
78 If SuperGetMV("MV_LJVPVAR",,.F.)
79
80     oCodProd:LostFocus := {| nVlrVP := 0, oLblLimiteVP:SetText(""), oLblDescricao:SetText("") , oVlrVP:ReadOnly := .T. } //reseta os campos
81
82     oButtonVP := TButton():New(POSVERT_GET1 ,POSHOR_1 * 14,STR0008,oPanGetValPre,{ || nVlrVP := 0, aVpMinMax := STBGetMinMaxVP(cGetProd), STIUpdMinMaxVP(),;
83                                     (cTipoVp := iIf(len(aVpMinMax)>2,aVpMinMax[3],cTipoVp)) },40,15,,,,.T.)
84
85     //Buscar Vale Presente"
86     oButtonVP:SetCSS( POSCSS (GetClassName(oButtonVP), CSS_BTN_FOCAL ))
87
88     //Descrição do produto:
89     oLblDescricao := TSay():New(oPanGetValPre:nHeight/7.55,POSHOR_1,{||"                                     "},oPanGetValPre,,,,,.T,,,,) //Campo de descricao do produto.
90     oLblDescricao:SetCSS( POSCSS (GetClassName(oLblDescricao), CSS_LABEL_FOCAL ))
91
92     //Valores do Vale Presente (Min e Max:
93     oLblLimiteVP := TSay():New(oPanGetValPre:nHeight/6.30,POSHOR_1,{||"                                     "},oPanGetValPre,,,,,.T,,,,) //Valores: Min - Max
94     oLblLimiteVP:SetCSS( POSCSS (GetClassName(oLblLimiteVP), CSS_LABEL_FOCAL ))
95
96     //Valor do vale presente:
97     oLblVlrVP := TSay():New(oPanGetValPre:nHeight/5.30,POSHOR_1,{||STR0009},oPanGetValPre,,,,,.T,,,,) //"Valor do vale presente"
98     oLblVlrVP:SetCSS( POSCSS (GetClassName(oLblVlrVP), CSS_LABEL_FOCAL ))
99
100    //Campo para informar o valor
101    oVlrVP := TGet():New(oPanGetValPre:nHeight/4.90,POSHOR_1,{|u| If(PCount())>0,nVlrVP:=u,nVlrVP)},oPanGetValPre,100,ALTURAGET,"@E 99,999,999.99",,,,,.T,,,,)
102    //Informar valor do vale presente
103    oVlrVP:SetCSS( POSCSS (GetClassName(oVlrVP), CSS_GET_FOCAL ))
104
105    oVlrVP:ReadOnly := .T.

```

```

103 EndIf
104
105 oButton := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0001,oPanGetValPre,{ || STISelValePresente(cGetProd,cTipoVp) ,oCodProd:SetFocus() }, ; //"Vale Presente"
106             LARGBTN,ALTURABTN,,,,.T.)
107 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
108
109 oCodProd:SetFocus()
110
111 Return(oPanGetValPre)
112
113 //-----
114 /*{Protheus.doc} STISelValePresente
115 Selecciona o vale presente na retaguarda
116
117 @param
118 @return
119 @obs
120 @sample
121 */
122 //-----
123 Function STISelValePresente(cGetCodPres,cTipoVp)
124
125 Local uResult          := Nil                //Resultado
126 Local lRet             := .F.                //Retorno
127 Local cStatus          := '0'                //Status
128 Local lMVLJVPAR        := SuperGetMV("MV_LJVPAR",,.F.) //Venda de vale presente valor variável
129 Local lContinua        := .T.                //Controla o fluxo do processo
130 Local lVPImpFiscal     := .F.                //Define se o item de vale presente é impresso em cupom fiscal
131 Local lExistVP         := IIF(FindFunction("STDVldVP"),!STDVldVP(cGetCodPres,cTipoVp),.F.) //Valida se o vale presente já foi registrado
132 Local cTipo            := ""                 //Tipo do Vale - P=Presente ou C=Credito
133 Local aRet             := {}                 //Retorno da Busca do item encontrado
134
135 Default cGetCodPres := ""
136
137 /*Validacao do vale presente*/
138 If lExistVP
139
140     If SuperGetMV("MV_LJPCNF",,.F.) //verifica se imprime vale presente no cupom não fiscal
141         lVPImpFiscal := .T.         //como não imprime em cupom fiscal desabilita esta variavel
142     EndIf
143
144     lRet := STBRemoteExecute("STBGDatPre" ,{cGetCodPres}, NIL,.F.,@uResult)
145
146     If ValType(uResult) == 'A'
147
148         If Len(uResult) > 0
149             cStatus := uResult[1][1]
150         EndIf
151
152         If cStatus == '0'
153
154             STFMessage(ProcName(0),"STOP",STR0003) //"Vale Presente não encontrado!"

```

```

155         STFSShowMessage(ProcName(0))
156
157     ElseIf cStatus == '1'
158
159         nPrice          :=      uResult[1][2]
160         cGetProd        :=      uResult[1][3]
161         If Len(uResult[1]) > 3
162             cTipo       :=      uResult[1][4]
163         EndIf
164
165         If Len(uResult[1]) >= 5 .AND. uResult[1][5] < DATE()
166             STFSMessage(ProcName(0), "STOP", STR0028) //Vale Presente/Crédito Vencido!
167             STFSShowMessage(ProcName(0))
168         Else
169             aRet := STWFindItem(cGetProd)
170             If len(aRet) > 0 .AND. aRet[1]
171
172                 cVale := cGetCodPres
173
174                 //Seta o valor do produto
175                 If lMVLJVPAR .AND. Len(aVpMinMax) > 0 .AND. !oVlrVP:lReadOnly
176                     //Verifica se o valor esta no Range limite para venda do vale presente
177                     If nVlrVP > 0 .AND. (aVpMinMax[1] > 0 .AND. aVpMinMax[2] > 0) .AND. (nVlrVP >= aVpMinMax[1] .AND. nVlrVP <= aVpMinMax[2])
178                         nPrice := nVlrVP
179                         STBSetVlrVP(cVale, nPrice)
180                     Else
181                         STFSMessage(ProcName(0), "STOP", STR0010 + AllTrim(Str(aVpMinMax[1])) + STR0011 + AllTrim(Str(aVpMinMax[2])) ) //O valor do vale presente informado
182                         STFSShowMessage(ProcName(0))
183                         lContinua := .F.
184                     EndIf
185                 EndIf
186             EndIf
187
188             //Valida se o cliente foi informado quando o tipo do vale for credito.
189             If lContinua
190                 lContinua := STIValidVP(cTipo, cGetProd)
191             EndIf
192
193             If lContinua
194
195                 STBSetCodVP(cVale) //Guarda Codigo do Vale Presente
196
197                 STBSetPric(nPrice)
198
199                 //Registra o Item
200                 STIItemRegister( cGetProd, lVPImpFiscal )
201
202                 //Abre Registro do Item
203                 STIExchangePanel( { || STIPanItemRegister() } )
204             EndIf
205         Else

```

```

206         STFMessagem(ProcName(0),"STOP",STR0019) //"Produto Vale Presente/Crédito não encontrado no PDV"
207         STFSHOWMESSAGE(ProcName(0))
208     EndIf
209 EndIf
210 ElseIf cStatus == '2'
211     STFMessagem(ProcName(0),"STOP",STR0004) //"Vale Presente/Crédito já foi vendido!"
212     STFSHOWMESSAGE(ProcName(0))
213 ElseIf cStatus == '3'
214     STFMessagem(ProcName(0),"STOP",STR0005) //"Vale Presente/Crédito já foi utilizado!"
215     STFSHOWMESSAGE(ProcName(0))
216 ElseIf cStatus == '4'
217     STFMessagem(ProcName(0),"STOP",STR0006) //"Vale Presente/Crédito cancelado!"
218     STFSHOWMESSAGE(ProcName(0))
219 EndIf
220 Else
221     STFMessagem(ProcName(0),"STOP",STR0007) //"Sem Conexão com a Retaguarda!"
222     STFSHOWMESSAGE(ProcName(0))
223
224 EndIf
225 Else
226     STFMessagem(ProcName(0),"STOP",STR0013) //"Vale presente/Crédito já registrado ou não foi possível validar."
227     STFSHOWMESSAGE(ProcName(0))
228 EndIf
229
230 Return .T.
231
232 //-----
233 /*{Protheus.doc} STBGDatPre
234 Funcao retorna o vale presente
235
236 @param cCodVP Codigo do vale presente
237 @return aRet
238 @obs
239 @sample
240 */
241 //-----
242 Function STBGDatPre(cCodVP)
243
244 Local aArea      := GetArea()           // Guarda area corrente
245 Local aRet       := {}                  // Retorno
246 Local lRet       := .F.                 // Retorno
247 Local lMDDTIPO  := MDD->(ColumnPos("MDD_TIPO"))>0 // Tipo do Vale
248 Local cTipo      := ""
249
250 Default cCodVP := ""
251
252 cCodVP := AllTrim(cCodVP)
253
254 DbSelectArea("MDD")
255 DbSetOrder(1)//MDD_FILIAL+MDD_CODIGO
256
257 If DbSeek(xFilial("MDD")+cCodVP)

```

```

258
259     If !EOF() .AND. MDD->MDD_FILIAL == xFilial("MDD") .AND. AllTrim(MDD->MDD_CODIGO) == cCodVP
260
261         If !MDDTIPO
262             cTipo := MDD->MDD_TIPO
263         Else
264             cTipo := "P"
265         EndIf
266
267         Add(aRet, {MDD_STATUS, MDD_VALOR, MDD_PROD, cTipo, MDD_DTFIM})
268         lRet := .T.
269
270     EndIf
271
272 EndIf
273
274 If !lRet
275     //Nao Encontrou. retorna status "0"
276     Add( aRet, {"0"} )
277 EndIf
278
279 RestArea(aArea)
280
281 Return aRet
282
283
284 //-----
285 /*{Protheus.doc} STBBaixaPr
286 Funcao que baixa o vale presente depois de vendido
287
288 @param cVale Vale presente
289 @return lRet
290 @obs
291 @sample
292 */
293 //-----
294 Function STBBaixaPr(cVale      , aVlrVP      , cDoc  , cSerie,;
295                   cEstacao   , cPDV        , dData , cHora ,;
296                   cVend      , cCliente    , cLoja )
297
298 Local aArea      := GetArea()           // Guarda area corrente
299 Local cGetProd   := ""                  // Vale presente
300 Local lRet       := .F.                  // Retorno
301 Local nX         := 0                    //contador
302 Local aInfoNcc   := {}
303 Local lMDDTIPO   := MDD->(ColumnPos("MDD_TIPO"))>0// Tipo do Vale
304 Local nValVP     := 0 // Valor do Vale Presente
305
306 Default aVlrVP := {}                    //Valores Vale presente
307 Default cVale  := ""
308 Default cDoc    := ""
309 Default cSerie  := ""

```

```

310 Default cEstacao      := ""
311 Default cPDV          := ""
312 Default dData         := CToD(" / / ")
313 Default cHora         := ""
314 Default cVend         := ""
315 Default cCliente      := ""
316 Default cLoja         := ""
317
318 LjGrvLog(,"STBBaixaPr | Atualizacao na MDD na Retaguarda | Inicio | ", )
319
320 cGetProd := AllTrim(cVale)
321
322 DbSelectArea("MDD")
323 DbSetOrder(1)//MDD_FILIAL+MDD_CODIGO
324
325 //Atualiza os valores do MDD_SALDO e MDD_VALOR caso a venda do vale presente seja valor variável
326 If Len(aVlrVP) > 0 .AND. SuperGetMV('MV_LJVPVAR',,.F.)
327
328   For nX := 1 TO Len(aVlrVP)
329     If DbSeek(xFilial("MDD") + AllTrim(aVlrVP[nX][1]))
330       If lMDDTIPO .And. AllTrim(MDD->(MDD_TIPO)) == "C" //Vale credito - Gera NCC
331         aInfoNcc := { AllTrim(aVlrVP[nX][1]) ,cDoc ,cSerie ,aVlrVP[nX][2],;
332                     cCliente ,cLoja }
333         STIVPGerCred(aInfoNcc)
334
335       Else
336         RecLock( "MDD", .F. )
337         MDD->MDD_VALOR := aVlrVP[nX][2]
338         MDD->MDD_SALDO := aVlrVP[nX][2]
339         MDD->MDD_STATUS := '2'
340         MDD->MDD_DOCV := IIf(!Empty(cDoc) , cDoc , MDD->MDD_DOCV)
341         MDD->MDD_SERIV := IIf(!Empty(cSerie) , cSerie , MDD->MDD_SERIV)
342         MDD->MDD_ESTV := IIf(!Empty(cEstacao) , cEstacao , MDD->MDD_ESTV)
343         MDD->MDD_PDVV := IIf(!Empty(cPDV) , cPDV , MDD->MDD_PDVV)
344         MDD->MDD_DATAV := IIf(!Empty(dData) , dData , MDD->MDD_DATAV)
345         MDD->MDD_HORAV := IIf(!Empty(cHora) , cHora , MDD->MDD_HORAV)
346         MDD->MDD_VEND := IIf(!Empty(cVend) , cVend , MDD->MDD_VEND)
347         MDD->MDD_CLIV := IIf(!Empty(cCliente) , cCliente , MDD->MDD_CLIV)
348         MDD->MDD_LJCLIV := IIf(!Empty(cLoja) , cLoja , MDD->MDD_LJCLIV)
349         MsUnLock()
350       EndIf
351     EndIf
352   Next nX
353 ElseIf DbSeek(xFilial("MDD")+cGetProd)
354
355   nValVP := MDD->MDD_VALOR
356
357   If lMDDTIPO .And. AllTrim(MDD->(MDD_TIPO)) == "C" //Vale credito - Gera NCC
358     aInfoNcc := { AllTrim(cVale) ,cDoc ,cSerie ,MDD->(MDD_VALOR) ,;
359                 cCliente ,cLoja }
360     STIVPGerCred(aInfoNcc)
361

```

```

362 Else
363     RecLock( "MDD", .F. )
364     MDD->MDD_STATUS := '2'
365     MDD->MDD_DOCV := IIf(!Empty(cDoc) , cDoc , MDD->MDD_DOCV)
366     MDD->MDD_SERIV := IIf(!Empty(cSerie) , cSerie , MDD->MDD_SERIV)
367     MDD->MDD_ESTV := IIf(!Empty(cEstacao) , cEstacao , MDD->MDD_ESTV)
368     MDD->MDD_PDVV := IIf(!Empty(cPDV) , cPDV , MDD->MDD_PDVV)
369     MDD->MDD_DATAV := IIf(!Empty(dData) , dData , MDD->MDD_DATAV)
370     MDD->MDD_HORAV := IIf(!Empty(cHora) , cHora , MDD->MDD_HORAV)
371     MDD->MDD_VEND := IIf(!Empty(cVend) , cVend , MDD->MDD_VEND)
372     MDD->MDD_CLIV := IIf(!Empty(cCliente) , cCliente , MDD->MDD_CLIV)
373     MDD->MDD_LJCLIV := IIf(!Empty(cLoja) , cLoja , MDD->MDD_LJCLIV)
374     MDD->MDD_SALDO := nValVP
375     MsUnLock()
376 EndIf
377 EndIf
378
379 RestArea(aArea)
380
381 Return lRet
382
383
384 //-----
385 /*{Protheus.doc} STBGetVales
386 Retorna o codigo do vale presente no fechamento da venda
387
388 @param
389 @return cVale
390 @obs
391 @sample
392 */
393 //-----
394 Function STBGetVales()
395 Return cVale
396
397
398 //-----
399 /*{Protheus.doc} STBSetVales
400 Funcao que limpa o codigo da static cVale
401
402 @param
403 @return
404 @obs
405 @sample
406 */
407 //-----
408 Function STBSetVales(cValPre)
409 cVale := cValPre
410 Return .T.
411
412 //-----
413 /*{Protheus.doc} STIUpdMinMaxVP

```

```

414 Exibe o limite do range do vale presente a ser inserido
415
416 @param
417 @return Nil
418 @obs
419 @sample
420 /*/
421 //-----
422 Function STIUpdMinMaxVP()
423
424 If Len(aVpMinMax) > 0
425     If aVpMinMax[1] > 0 .AND. aVpMinMax[2] > 0 //valores minimo e maximo do vale presente
426         oLblLimiteVP:SetText(STR0018 +"Min. R$ " + AllTrim(Str(aVpMinMax[1])) + " - Max. R$ " + AllTrim(Str(aVpMinMax[2])) ) //"Valores: "
427         If Len(aVpMinMax) > 2 .AND. Len(aVpMinMax) > 3
428             oLblDescricao:SetText(STR0017 + Posicione("SB1",1,xFilial("SB1")+aVpMinMax[4],"B1_DESC")) //"Descrição: "
429         EndIf
430         oVlrVP:lReadOnly := .F. //seta o campo de valor com somente leitura
431         STFMessage(ProcName(0),"STOP","") //Limpa mensagem do rodapé
432         STFShowMessage(ProcName(0))
433     Else
434         nVlrVP := 0 //inicializa o campo valor do vale presente
435         oLblLimiteVP:SetText("") //limpa o texto do limite do vale presente
436         oLblDescricao:SetText("") //exibe texto de descricao do vale presente/credito
437         oVlrVP:lReadOnly := .T. //define o campo do valor como somente leitura
438         STFMessage(ProcName(0),"STOP",STR0012) //Este vale presente não possui venda de valor variável
439         STFShowMessage(ProcName(0))
440     EndIf
441 EndIf
442
443 Return Nil
444
445 //-----
446 /*{Protheus.doc} STIValidVP
447 Validação executada antes da ação do botão Vale Presente
448
449 @param
450 @return lRet - .T. -> operação OK / .F. -> operação invalida
451 @obs
452 @sample
453 /*/
454 //-----
455 Static Function STIValidVP(cTipo,cGetProd)
456 Local lRet := .T.
457 Local cCliVend := ""
458
459 If cTipo == "C"
460     cCliVend := AllTrim(STDGPBasket("SL1", "L1_CLIENTE"))
461
462     If Empty(cCliVend) .Or. cCliVend == AllTrim(SuperGetMV("MV_CLIPAD",,""))
463         STFMessage(ProcName(0),"STOP",STR0015) //"Selecionar cliente para venda de vale do tipo crédito."
464         STFShowMessage(ProcName(0))
465         lRet := .F.

```

```

466     EndIf
467
468     If lRet .And. "VALE PRESENTE" $ Upper(AllTrim(cGetProd)) .OR. "VALE CREDITO" $ Upper(AllTrim(cGetProd))
469         STFMessage(ProcName(0),"STOP",STR0016) //"Descrição do produto Vale Presente/Crédito incorreta."
470         STFShowMessage(ProcName(0))
471         lRet := .F.
472     EndIf
473 EndIf
474
475 Return lRet
476
477 //-----
478 /*{Protheus.doc} STIVPGerCred
479 Gerar a NCC do vale credito na retagaurda
480
481 @param
482 @return lRet - .T. -> operação OK / .F. -> operação invalida
483 @obs
484 @sample
485 */
486 //-----
487 Static Function STIVPGerCred(aInfo)
488 Local lRet      := .F.
489 Local cVale     := ""
490 Local cDoc      := ""
491 Local cSerie    := ""
492 Local nValor    := 0
493 Local cCliente  := ""
494 Local cLoja     := ""
495 Local cTipo     := "NCC"
496 Local cNatu     := &(SuperGetMV("MV_NATNCC"))
497 Local cParcela  := ""
498 Local cIDUP     := SuperGetMv("MV_IDUP")
499 Local nI        := 0
500 Local nQtdParc  := 1
501 Local aRetPE    := {}
502 Local aNcc      := {}
503
504 Default aInfo   := {}
505
506 LjGrvLog( STR0001,"Inicio Vale Credito", FWTimestamp(2))
507
508 If Len( aInfo ) > 0
509
510     cVale      := aInfo[1] //Vale
511     cDoc       := aInfo[2] //Documento
512     cSerie     := aInfo[3] //Serie
513     nValor     := aInfo[4] //Valor
514     cCliente   := aInfo[5] //Cliente
515     cLoja      := aInfo[6] //loja cliente
516
517     If ExistBlock("STGeraCr")

```

```

518     aRetPE := ExecBlock("STGeraCr",.F.,.F.,{cVale,nValor,cCliente,cLoja})
519     If ValType(aRetPE) == "A" .And. Len(aRetPE) > 1
520         nValor      := aRetPE[1] //Valor a ser gerado
521         nQtdParc    := aRetPE[2] //Quantidade de parcelas
522     Endif
523 EndIf
524
525 For nI := 1 To nQtdParc
526     aNcc := {}
527     cParcela := LJParcela( nI, cLDUP )
528     LjGrvLog( STR0001,"Vale Credito:"+cVale+"/DOC: "+cDoc+"/Serie: "+cSerie+"/Parcela: "+cParcela+"/R$: "+AllTrim(STR(nValor)), "Gerando crédito") //Vale presente
529
530     aAdd(aNcc,{"E1_PREFIXO" ,cSerie ,Nil})
531     aAdd(aNcc,{"E1_NUM"      ,cDoc ,Nil})
532     aAdd(aNcc,{"E1_PARCELA" ,cParcela ,Nil})
533     aAdd(aNcc,{"E1_NATUREZ" ,cNatu ,Nil})
534     aAdd(aNcc,{"E1_TIPO"    ,cTipo ,Nil})
535     aAdd(aNcc,{"E1_EMISSAO" ,dDatabase ,Nil})
536     aAdd(aNcc,{"E1_VALOR"   ,nValor ,Nil})
537
538     If SuperGetMV("MV_LJVLNCC" ,,.F.)
539         aAdd(aNcc,{"E1_VENCTO" , (dDataBase + SuperGetMV("MV_LJDTNCC",,0)) ,Nil})
540         aAdd(aNcc,{"E1_VENCREA" ,DataValida((dDataBase + SuperGetMV("MV_LJDTNCC",,0)),.T.) ,Nil})
541     Else
542         aAdd(aNcc,{"E1_VENCTO" ,dDataBase ,Nil})
543         aAdd(aNcc,{"E1_VENCREA" ,DataValida(dDataBase,.T.) ,Nil})
544         aAdd(aNcc,{"E1_VENCORI" ,dDataBase ,Nil})
545     EndIf
546
547     aAdd(aNcc,{"E1_SALDO" ,nValor ,Nil})
548     aAdd(aNcc,{"E1_VLCRUZ" ,xMoeda(nValor,1,1,nValor) ,Nil})
549     aAdd(aNcc,{"E1_CLIENTE" ,cCliente ,Nil})
550     aAdd(aNcc,{"E1_LOJA" ,cLoja ,Nil})
551     aAdd(aNcc,{"E1_MOEDA" ,1 ,Nil})
552     aAdd(aNcc,{"E1_STATUS" ,If(nValor > 0.01,"A","B") ,Nil})
553     aAdd(aNcc,{"E1_ORIGEM" , "STIPOSMAIN" ,Nil})
554     aAdd(aNcc,{"E1_HIST" ,AllTrim(cVale)+"-"+cDoc+cSerie ,Nil})
555
556
557     lRet := Empty(STDIncSE1(aNcc))
558
559     If lRet
560         LjGrvLog( STR0001, "Vale Credito:"+cVale+"/DOC: "+cDoc+"/Serie: "+cSerie+"/R$: "+AllTrim(STR(nValor)), "Gera com sucesso!") //Vale Credito"
561     Else
562         LjGrvLog( STR0001,"Vale Credito:"+cVale+"/DOC: "+cDoc+"/Serie: "+cSerie+"/R$: "+AllTrim(STR(nValor)), "Problema ao gerar o crédito") //Vale Credito"
563     EndIf
564 Next nI
565 EndIf
566
567 LjGrvLog( STR0001,"Fim Vale Credito", FwTimeStamp(2))
568
569 Return lRet

```

```

570 //-----
571 /*{Protheus.doc} STIConsValPre
572 Funcao que consulta os códigos de vale presente na tabela MDD da retaguarda
573
574 @param cCodVP, caracter,Codigo do vale presente
575 @Type Funcion
576
577
578 */
579 //-----
580 Function STIConsValPre(cStVP)
581   Local oDlg
582   Local aRet      := {}
583   Local lSelVp    := .F.
584   Local cDescrSay  := STR0020
585   Local oSay      := NIL
586   Local aCpos     := {}
587   Local lRet      := .T.
588
589   Private oLbx
590   Default cStVP   := "1"
591
592   If ISINCALLSTACK('LJ7TELAFIN')
593     cStVP := "2"
594   Endif
595
596   FWMsgRun(, {|oSay| STIMontGridVp(oSay,cStVP, @aCpos) }, cDescrSay, STR0021) //Consulta, carregando dados
597
598   DEFINE MSDIALOG oDlg TITLE STR0022 FROM 0,0 TO 320,700 PIXEL //"Consulta de Vale Presente"
599
600   @ 05,010 LISTBOX oLbx FIELDS HEADER STR0023      ,; //'Codigo Vale'
601                                     STR0024      ,; //'Valor'
602                                     STR0025      ,; //'Produto'
603                                     STR0026      ,; //'Inicio Vig'
604                                     STR0027 SIZE 335,120 OF oDlg PIXEL //'Fim Vig'
605
606   oLbx:SetArray( aCpos )
607   oLbx:bLine     := {|| aCpos[oLbx:nAt]}
608   oLbx:bLDbClick := {|| {oDlg:End(), lSelVp:=.T.,aRet := oLbx:aArray[oLbx:nAt]} }
609
610   oTbtn2 := TButton():New( 148, 305, "Selecionar",oDlg,{|| oDlg:End(), lSelVp:=.T., aRet := oLbx:aArray[oLbx:nAt] }, 040, 010,,.F.,.T.,.F.,.F.,.F.,.F. )
611
612   ACTIVATE MSDIALOG oDlg CENTER
613
614   If Len(aRet) > 0 .And. lSelVp
615     cCodVPre :=aRet[1]
616   else
617     cCodVPre :=""
618   EndIf
619
620 Return(lRet)
621

```

```

622
623 /*{Protheus.doc} STIRetCodVP
624 Retorna o código do vale presente selecionado
625 @Type Função
626 @return cCodVPre, Caracter, Código do vale presente selecionado
627
628 */
629 Function STIRetCodVP()
630 Return cCodVPre
631
632
633 /*{Protheus.doc} STIMontGridVp
634 Seleciona os vales presentes na tabela MDD e atualiza o objeto oSay com os dados
635
636 @param oSay, objeto, Grid dos vales presentes
637 @param cStVP, caracter, Status do vale presente
638 @param aCpos, array, campos da consulta
639
640 @Type Static função
641
642 */
643 Static Function STIMontGridVp(oSay,cStVP,aCpos)
644
645 Default cStVP := "1" //Status de vale a ser vendido
646
647 aCpos:={}
648
649 If FINDFUNCTION("STDGDPVpre" )
650 STBRemoteExecute("STDGDPVpre" ,{cStVP}, NIL,.F.,@aCpos)
651
652 If Len(aCpos) < 1
653 aAdd(aCpos,{" ", " ", " ", " ", " ", " "})
654 EndIf
655
656 Endif
657
658 Return

```

Componentes/Interface/STIValeTroca.prw

188 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIVALETROCA.CH"
5
6 Static aCVTs := {} // Array dos itens que terão os cupons de vale troca
7 Static nTamItem := TamSX3("L2_ITEM")[1]
8
9 //-----
10 /*{Protheus.doc} STIValeTroca
11 Chama a tela de selecao de produtos para emissao do vale troca
12
13 @param
14 @return lRet          Retorno se executou corretamente a funcao
15 @obs
16 @sample
17 */
18 //-----
19 Function STIValeTroca()
20
21 STFCleanInterfaceMessage()
22 STIExchangePanel({|| STIPanValeTroca() })
23
24 Return
25
26 //-----
27 /*{Protheus.doc} STIPanValeTroca
28 Cria o painel de Vale troca
29
30 @param
31 @return lRet          Retorno se executou corretamente a funcao
32 @obs
33 @sample
34 */
35 //-----
36 Static Function STIPanValeTroca()
37 Local oPanelMVC      := STIGetDlg()          //Painel principal do dialog
38 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Painel vale troca
39
40 Local nLargGets      := 90
41
42 Local oCab           := Nil
43 Local nVertCab       := oPanelMVC:nHeight/81.3      // Posicao vertical do cabecalho
44
45 Local oLblGetItem    := Nil
46
47 Local oGetItem       := Nil
48 Local cItem         := Space(nTamItem)             //Alteracao do conteudo do get
49
50 Local oLblProdutos   := Nil
51
52 Local oGetList       := Nil                        // TComboBox Escolha do campo a ser utilizado na busca

```

```

53 Local cGetList      := ""                // Variável do Get
54
55 Local oBtnConfirm   := Nil
56 Local oBtnCancel    := Nil
57
58 Local nVertLbl1      := oPanelMVC:nHeight/23.228 // Posicao vertical dos label da linha 1
59 Local nVertLbl2      := oPanelMVC:nHeight/9.0333 // Posicao vertical dos label da linha 2
60
61 Local nVertGet1      := oPanelMVC:nHeight/14.781 // Posicao vertical dos get da linha 1
62 Local nVertGet2      := oPanelMVC:nHeight/7.3909 // Posicao vertical dos get da linha 2
63
64 Local oFontCod       := TFont():New('Arial',,-16,.T.)
65 Local oListFont      := TFont():New("Courier New")
66
67 oMainPanel:SetCSS(POSCSS(GetClassName(oMainPanel),CSS_PANEL_CONTEXT))
68
69 oCab      := TSay():New(nVertCab,POSHOR_1,{||STR0001},oMainPanel,,,,,T,,,,) //"Vale Troca - Seleção de Produtos"
70 oCab:SetCSS( POSCSS (GetClassName(oCab), CSS_BREADCUMB ))
71
72 oLblGetItem:= TSay():New(nVertLbl1,POSHOR_1,{||STR0002},oMainPanel,,,,,T,,,,) //"Digite o número do item"
73 oLblGetItem:SetCSS( POSCSS (GetClassName(oLblGetItem), CSS_LABEL_FOCAL ))
74
75 @ nVertGet1,POSHOR_1 MSGET oGetItem VAR cItem SIZE nLargGets,ALTURAGET PICTURE "@!" FONT oFontCod OF oMainPanel PIXEL
76 oGetItem:SetCSS( POSCSS (GetClassName(oGetItem), CSS_GET_FOCAL ))
77 oGetItem:LostFocus := { || IIF( !Empty(AllTrim(cItem)),;
78                          Eval({|| STISelectItem( Val(cItem), oGetList ), cItem := Space(nTamItem),oGetItem:SetFocus()),oBtnConfirm:SetFocus())}
79
80 oLblProdutos:= TSay():New(nVertLbl2,POSHOR_1,{||STR0003},oMainPanel,,,,,T,,,,) // "Relação de produtos com vale troca:"
81 oLblProdutos:SetCSS( POSCSS (GetClassName(oLblProdutos), CSS_LABEL_FOCAL ))
82
83 oGetList := TListBox():Create(oMainPanel, nVertGet2, POSHOR_1, {||u| If(PCount(>0,cGetList:=u,cGetList)}, , LARG_LIST_CONSULT , ALT_LIST_CONSULT ,,,,,T.,{||.T.},oListFont)
84 oGetList:SetCSS( POSCSS (GetClassName(oGetList), CSS_LISTBOX ))
85
86 STILoadItensList( oGetList )
87
88 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0004+CRLF+STR0005,oMainPanel; //"Voltar para" ### "Registro de Itens"
89                               { || Iif(ExistFunc("StiMataObj"),StiMataObj(oMainPanel),Nil), STIRegItemInterface(),STFCleanInterfaceMessage() }, LARGBTN,ALTURABTN,,,,T. )
90
91 oBtnCancel := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0006,oMainPanel; //"Limpar Seleção"
92                               { || aCVTs := {},oGetList:SetArray( aCVTs ) }, LARGBTN,ALTURABTN,,,,T. )
93
94
95 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL ))
96 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel), CSS_BTN_ATIVO ))
97 oGetItem:SetFocus()
98
99 Return oMainPanel
100
101 //-----
102 /*{Protheus.doc} STISelectItem
103 Seleciona o item para o vale troca
104

```

```

105 @param
106 @return lRet          Retorno se executou corretamente a funcao
107 @obs
108 @sample
109 /*/
110 //-----
111 Static Function STISelectItem( nItem, oGetList )
112 Local cItem := ""
113
114 DEFAULT nItem := 0
115
116 cItem := StrZero( nItem, nTamItem, 0 )
117
118 oGetList:SetArray( {} )
119
120 If aScan( aCVTs, cItem ) == 0
121
122     If !STDPBIsDeleted( "SL2", nItem )
123         Add( aCVTs, cItem )
124         STFCleanInterfaceMessage()
125     Else
126         STFMessage( ProcName(), "STOP", STR0007 ) // "Este item foi cancelado!"
127         STFShowMessage(ProcName())
128     EndIf
129
130 Else
131
132     STFMessage( ProcName(), "STOP", STR0008 ) // "Item já selecionado!"
133     STFShowMessage(ProcName())
134
135 EndIf
136
137 STILoadItensList( oGetList )
138
139 Return
140
141 //-----
142 /*{Protheus.doc} STISelectItem
143 Seleciona o item para o vale troca
144
145 @param
146 @return lRet          Retorno se executou corretamente a funcao
147 @obs
148 @sample
149 /*/
150 //-----
151 Static Function STILoadItensList( oGetList )
152 Local aItensList := {}
153 Local cItemList  := ""
154 Local nX         := 0
155
156 For nX := 1 To Len(aCVTs)

```

```
157
158     cItemList := aCVTs[nX]
159     Aadd( aItensList,STR0009+cItemList+" / "+STR0010+STDGPBasket( "SL2" , "L2_DESCRI" , Val(cItemList) ) ) // "Item nº " ### "Produto: "
160
161 Next nX
162
163 oGetList:SetArray( aItensList )
164
165 Return Nil
166
167 //-----
168 /*{Protheus.doc} STIGetCVTs
169 Retorna array com os itens que terao o vale troca impresso.
170
171 @return aCVTs - array com os itens que terao o vale troca impresso
172 */
173 //-----
174 Function STIGetCVTs( )
175
176 Return aCVTs
177
178 //-----
179 /*{Protheus.doc} STICVTClear
180 Limpa o array com os itens que terao o vale troca impresso.
181
182 */
183 //-----
184 Function STICVTClear( )
185
186 aCVTs := {}
187
188 Return
```

Componentes/Interface/STPOS.ch

254 linhas após remoção de metadados de autoria/data

```

1  /*/
2      ATENCAO!!!
3      TODOS OS DEFINES DO "POS" DEVEM SER CRAIDOS NESTE FONTE
4  /*/
5
6
7  /*/
8      Registro de Itens
9  /*/
10 #DEFINE  ITEM_ENCONTRADO      1 // 01 - Encontrou Item?
11 #DEFINE  ITEM_CODIGO         2 // 02 - Codigo do Item encontrado
12 #DEFINE  ITEM_CODBAR         3 // 03 - Codigo de Barras do Item encontrado
13 #DEFINE  ITEM_BLOQUEADO      4 // 04 - Item esta bloqueado para venda?
14 #DEFINE  ITEM_QTDE           5 // 05 - Quantidade do item
15 #DEFINE  ITEM_TES            6 // 06 - TES do item
16 #DEFINE  ITEM_BALANCA        7 // 07 - Tipo produto de balanca
17 #DEFINE  ITEM_DESCRICAO      8 // 08 - Descrição do item
18 #DEFINE  ITEM_UNID_MEDIDA     9 // 09 - Unidade de Medida do item
19 #DEFINE  ITEM_ARMAZEMPAD     10 // 10 - Armazem Padrão do item
20 #DEFINE  ITEM_TIPO           11 // 11 - Tipo do Produto
21 #DEFINE  ITEM_SITTRIB        12 // 12 - ST do item
22 #DEFINE  ITEM_IAT            13 // 13 - IAT do item
23 #DEFINE  ITEM_IPPT           14 // 14 - PPT do item
24 #DEFINE  ITEM_TOTIMP         15 // 15 - Impostos - Lei dos impostos
25 #DEFINE  ITEM_ORIGEM         16 // 16 - Origem produto
26 #DEFINE  ITEM_CODISS        17 // 17 - Codigo ISS
27 #DEFINE  ITEM_POSIPI         18 // 18 - POIPI
28 #DEFINE  ITEM_INT_ICM        19 // 19 - PPT do item
29 #DEFINE  ITEM_EX_NCM         20 // 20 - Extensao do codigo NCM
30 #DEFINE  ITEM_TOTFED         21 // 21 - Impostos - Lei dos impostos Federal
31 #DEFINE  ITEM_TOTEST         22 // 22 - Impostos - Lei dos impostos Estadual
32 #DEFINE  ITEM_TOTMUN         23 // 23 - Impostos - Lei dos impostos Municipal
33 #DEFINE  ITEM_RECNO          24 // 24 - Recno do Produto na tabela SB1
34
35 #DEFINE  ITEM_TOTAL_ARRAY    24 // TOTAL de defines no array
36
37 /*/
38      Regra de Desconto
39 /*/
40 #DEFINE  LRULEAPPLY           1 // Alguma regra foi aplicada?
41 #DEFINE  NNEWTOTAL            2 // Valor total depois da aplicada(s) a(s) regra(s)
42 #DEFINE  NDESCTOTAL           3 // Valor total do desconto aplicada(s) a(s) regra(s)
43 #DEFINE  NPERDESCTO           4 // Percentual de desconto total aplicado em relacao ao valor total antigo
44 #DEFINE  NOLDTOTAL            5 // Valor total antes da aplicada(s) a(s) regra(s)
45
46 #DEFINE  MEI_CODREG           1 // Codigo da Regra
47 #DEFINE  MEJ_PRINUM           2 // Prioridade da Regra
48 #DEFINE  MEI_ACUMUL           3 // Indica se a regra é acumulativa
49 #DEFINE  MEI_DESCRI           4 // Descrição da Regra
50 #DEFINE  MB8_DESCPR           5 // Percentual de desconto
51 #DEFINE  MB8_DESCVL           6 // Valor de desconto
52

```

```

53
54 /*/
55     Abertura Fechamento de caixa
56 /*/
57
58 #DEFINE CAIXA_FECHADO          1 // Caixa foi fechado pelo operador
59 #DEFINE TROCOU_OPERADOR      2 // Houve troca de operador
60 #DEFINE ULTIMO_OPERADOR      3 // Ultimo operador que usou o sistema e nao fechou caixa
61
62
63 /*/
64     Sangria e Suplimento
65 /*/
66 #DEFINE TYPEOPERATION        1 // 01 - Tipo da rotina: (1) Sangria / Entrada de troco (2)
67 #DEFINE CASHIER_ORIGIN       2 // 02 - Codigo do caixa de origem
68 #DEFINE CASHIER_DESTINY      3 // 03 - Codigo do caixa de destino
69 #DEFINE VALUE_MONEY          4 // 04 - Valor em dinheiro
70 #DEFINE VALUE_CHECK          5 // 05 - Valor em cheque
71 #DEFINE VALUE_CREDITCARD     6 // 06 - Valor em cartao de credito
72 #DEFINE VALUE_DEBITCARD      7 // 07 - Valor em cartao de debito
73 #DEFINE VALUE_FINANCED       8 // 08 - Valor em financiado
74 #DEFINE VALUE_COVENANT       9 // 09 - Valor em convenio
75 #DEFINE VALUE_COUPONS       10 // 10 - Valor em vales
76 #DEFINE VALUE_OTHERS        11 // 11 - Valor em outros
77 #DEFINE AGENCY              12 // 12 - Agencia
78 #DEFINE DV_AGENCY           13 // 13 - Dv. Agencia
79 #DEFINE ACCOUNT             14 // 14 - Conta
80 #DEFINE DV_ACCOUNT          15 // 15 - Dv. Conta
81 #DEFINE VALUE_PD            16 // 16 - Valor Pagamento Digital
82 #DEFINE VALUE_PIX           17 // 17 - Valor PIX
83 /*/
84     REGRA DE DESCONTO aRules - Retorno da funcao STDtotRldi - ( STDRULEDISCOUNTTOTAL )
85 /*/
86 #DEFINE MEI_CODREG          1 // Caixa foi fechado pelo operador
87 #DEFINE MEJ_PRINUM          2 // Prioridade da Regra
88 #DEFINE MEI_ACUMUL          3 // Indica se a regra é acumulativa
89 #DEFINE MEI_DESCRI          4 // Descrição da Regra
90
91
92 /*/
93     aRange - Retorno da Funcao STDRangRul - ( STDRULEDISCOUNTTOTAL )
94 /*/
95 #DEFINE nMB2_VLRINI         1 // Valor inicial
96 #DEFINE nMB2_VLRFIM         2 // Valor final
97 #DEFINE nMB2_PDESC          3 // Percentual de desconto
98 #DEFINE nMB2_VLRFEX         4 // Valor de desconto
99
100 #DEFINE MB2_VLRINI          1 // Valor inicial
101 #DEFINE MB2_VLRFIM          2 // Valor final
102 #DEFINE MB2_PDESC           3 // Percentual de desconto
103 #DEFINE MB2_VLRFEX          4 // Valor de desconto
104

```

```

105
106 /*/
107     aRet - Retorno da Funcao STBTotRlDi - ( STBRULEDISCOUNTTOTAL ) (desse fonte)
108 /*/
109 #DEFINE LRULETOTAP          1 // Alguma regra foi aplicada?
110 #DEFINE NNEWTOTAL          2 // Valor total depois da aplicada(s) a(s) regra(s)
111 #DEFINE NPERDESCTO         3 // Percentual de desconto total aplicado em relacao ao valor total antigo
112 #DEFINE NOLDTOTAL          4 // Valor total antes da aplicada(s) a(s) regra(s)
113
114
115 /*/
116     aRet da Funcao STDTotRlDi
117 /*/
118 #DEFINE CODREG              1 // Caixa foi fechado pelo operador
119 #DEFINE PRINUM              2 // Prioridade da Regra
120 #DEFINE ACUMUL              3 // Indica se a regra é acumulativa
121 #DEFINE DESCR              4 // Descrição da Regra
122
123
124 /*/
125     Recebimentos de Titulos
126 /*/
127
128 #DEFINE TIT_SELE  1          // Posicao logica.
129 #DEFINE TIT_PREF  2          // Prefixo do titulo.
130 #DEFINE TIT_NUME  3          // Titulo.
131 #DEFINE TIT_PARC  4          // Parcela.
132 #DEFINE TIT_VREA  5          // Vencimento real.
133 #DEFINE TIT_VALO  6          // Valor.
134 #DEFINE TIT_MULT  7          // Multa.
135 #DEFINE TIT_JURO  8          // Juros.
136 #DEFINE TIT_DESC  9          // Desconto.
137 #DEFINE TIT_RECE 10         // Recebimento.
138 #DEFINE TIT_TIPO 11         // Tipo.
139 #DEFINE TIT_CONT 12         // Contrato.
140 #DEFINE TIT_CLIE 13         // Cliente.
141 #DEFINE TIT_LOJA 14         // Loja.
142 #DEFINE TIT_FILI 15         // Filial.
143 #DEFINE TIT_RECN 16         // Recno.
144 #DEFINE TIT_ACRS 17         // Acrescimo financeiro.
145 #DEFINE TIT_CACR 18         // Posicao logica.
146 #DEFINE TIT_ABAT 19         // Abatimentos.
147 #DEFINE TIT_VENC 20         // Vencimento original.
148 #DEFINE TIT_LOCK 21         // Cod de uso do Registro
149 #DEFINE TIT_MOED 22         //Codigo da moeda do titulo
150 #DEFINE TIT_INTE 23         // Interes (Juros) - Posicao reservada
151 #DEFINE TIT_DTBX 24         // Dt. da baixa
152 #DEFINE TIT_SALD 25         // Saldo do titulo
153
154 #DEFINE OP_BAIXA "1"        // Baixa de titulos
155 #DEFINE OP_ESTOR "2"        // Estorno de baixa
156

```

```

157 #DEFINE REMOTE_TITLE_FUNCTION "STRemoteTi" // Funcao remota do recebimento de titulos para executar na retaguarda
158
159
160 /*/
161     Selecao de Clientes
162 /*/
163 #DEFINE LSELECTED      1 // Indica True se foi selecionado algum cliente.
164 #DEFINE MODEL_CLIENTE  1 // Model Cliente
165
166 #DEFINE POS_CLIENTE     1 // Indica posicao do codigo do cliente no array
167 #DEFINE POS_LOJACLI     2 // Indica posicao do codigo da loja do cliente no array
168
169 #DEFINE FIELD_NAME      1 // Nome do campo
170 #DEFINE FIELD_VALUE     2 // Valor do campo
171
172
173 /*/
174     ECF/ TEF / PAF
175 /*/
176 #DEFINE TEF_DISCADO      "4"                // TEF Discado
177 #DEFINE CRLF              Chr(10)           // Control
178 #DEFINE F_NAME            1                  // Nome
179 #DEFINE EQUIP_DLL_NENHUM  0                  // 0 equipamento nao foi configurado
180 #DEFINE EQUIP_DLL_AUTOCOM 1                  // 0 equipamento foi configurado para utilizar a AUTOCOM
181 #DEFINE EQUIP_DLL_SIGALOJA 2                 // 0 equipamento foi configurado para utilizar a SIGALOJA
182
183 // Tipos de equipamentos
184 #DEFINE EQUIP_IMPFISCAL   1
185 #DEFINE EQUIP_PINPAD     2
186 #DEFINE EQUIP_CMC7       3
187 #DEFINE EQUIP_GAVETA     4
188 #DEFINE EQUIP_IMPCUPOM   5
189 #DEFINE EQUIP_LEITOR     6
190 #DEFINE EQUIP_BALANCA    7
191 #DEFINE EQUIP_DISPLAY    8
192 #DEFINE EQUIP_IMPCHEQUE  9
193 #DEFINE EQUIP_IMPNAOFISCAL 10
194
195 //Menu fiscal
196 #DEFINE _PICTURE          16                // Picture
197 #DEFINE _FORMATEF         "CC;CD"           //Formas de pagamento que utilizam operação TEF para validação
198 #DEFINE _FORMATPD        "PD;PX"           //Formas de pagamento que utilizam operação TOTVS Pagamentos Digitais
199 #DEFINE _RAZSOC           "TOTVS S.A."       // Denominacao da empresa desenvolvedora do PAF-ECF - Razao social
200 #DEFINE _CNPJTOT         "53113791000122"   // CNPJ da empresa desenvolvedora do PAF-ECF - Totvs
201 #DEFINE _ENDTOTVS        "Av Braz Leme,1631 - Sao Paulo-SP" // Endereco da empresa desenvolvedora do PAF-ECF - Totvs
202 #DEFINE _CEPTOTVS      "02511-000"       // Endereco da empresa desenvolvedora do PAF-ECF - Totvs
203 #DEFINE _TELTOTVS        "0800 709 8100"    // Telefone da empresa desenvolvedora do PAF-ECF - Totvs
204 #DEFINE _CONTOTVS        "Wilson de Godoy"  // Contato da empresa desenvolvedora do PAF-ECF - Totvs
205 #DEFINE _INSCST          "111010945111"    // Inscricao Estadual da empresa desenvolvedora do PAF-ECF, se houver
206 #DEFINE _INSCMUN         "90814274"        // Inscricao Municipal da empresa desenvolvedora do PAF-ECF, se houver
207 #DEFINE _NUMLAUDO        IIF(nModulo == 12, "POL4232010", "POL4262010") // Numero do laudo da homologacao *** ajustar
208 #DEFINE _NOMEXEC         IIF(nModulo == 23, "SIGAFRT_PAF.exe",; // Nome do executavel homologado no Frontloja

```

```

209
210
211 /* Definições de Interface do POS */
212 #DEFINE ALTURAGET 13 // Altura (tamanho) dos GETS
213 #DEFINE ALTURABTN 25 // Altura (tamanho) dos Botoes
214 #DEFINE ALT_LIST_CONSULT oPanelMVC:nHeight/4.4865 // Altura (Tamanho) do listBox de consulta
215 #DEFINE LARG_LIST_CONSULT oPanelMVC:nWidth/2.1621 // Largura (Tamanho) do listBox de consulta
216 #DEFINE LARGBTN 80 // Largura (tamanho) dos Botoes
217 #DEFINE POSHOR_1 oPanelMVC:nWidth/49.2 // Posicao horizontal 1
218 #DEFINE POSHOR_2 oPanelMVC:nWidth/3.55555 // Posicao horizontal 2
219 #DEFINE POSVERT_CAB oPanelMVC:nHeight/100 // Posicao vertical do cabeçalho
220 #DEFINE POSVERT_LABEL1 oPanelMVC:nHeight/16.8 // Posicao vertical do label 1
221 #DEFINE POSVERT_LABEL2 oPanelMVC:nHeight/7.084 // Posicao vertical do label 2
222 #DEFINE POSVERT_LABEL3 oPanelMVC:nHeight/4.4866 // Posicao vertical do label 3
223 #DEFINE POSVERT_LABEL4 oPanelMVC:nHeight/3.28292 // Posicao vertical do label 4
224 #DEFINE POSVERT_LABEL5 oPanelMVC:nHeight/2.25192 // Posicao vertical do label 5
225 #DEFINE POSVERT_LABEL6 oPanelMVC:nHeight/2.15292 // Posicao vertical do label 6
226 #DEFINE POSVERT_LABEL7 oPanelMVC:nHeight/2.58292 // Posicao vertical do label 7
227 #DEFINE POSVERT_GET1 oPanelMVC:nHeight/11.215 // Posicao vertical do get 1
228 #DEFINE POSVERT_GET2 oPanelMVC:nHeight/5.852 // Posicao vertical do get 2
229 #DEFINE POSVERT_GET3 oPanelMVC:nHeight/3.95882 // Posicao vertical do get 3
230
231 #DEFINE POSVERT_GET4 oPanelMVC:nHeight/2.99111 // Posicao vertical do get 4
232 #DEFINE POSVERT_BTNFOCAL (oPanelMVC:nHeight/2.10312)-ALTURABTN // Posicao horizontal do botao focal
233 #DEFINE POSHOR_BTNFOCAL (oPanelMVC:nWidth/2.07115)-LARGBTN // Posicao horizontal do botao focal
234 #DEFINE POSHOR_BTNCANC (oPanelMVC:nWidth/2.82115)-LARGBTN // Posicao horizontal do botao focal
235
236 // Interface de Pagamento
237 #DEFINE POSVERT_BTNPAY (oPnlAdconal:nHeight/2.1666) - ALTURABTN // Posicao vertical do botao Efetuar pagamentos
238 #DEFINE POSHOR_BTNCONFPAY (oPnlAdconal:nWidth/2.06779) - LARGBTN // Posicao horizontal do botao Efetuar pagamentos
239 #DEFINE POSVERT_PAYLABEL1 000 // Posicao vertical da segunda linha de labels do pagamento
240 #DEFINE POSVERT_PAYLABEL2 oPnlAdconal:nHeight/8.3 // Posicao vertical da segunda linha de labels do pagamento
241 #DEFINE POSVERT_PAYLABEL3 oPnlAdconal:nHeight/4.1 // Posicao vertical da Terceira linha de labels do pagamento
242 #DEFINE POSVERT_PAYGET1 oPnlAdconal:nHeight/19.5 // Posicao vertical da primeira linha de gets do pagamento
243 #DEFINE POSVERT_PAYGET2 oPnlAdconal:nHeight/5.7 // Posicao vertical da segunda linha de gets do pagamento
244 #DEFINE POSVERT_PAYGET3 oPnlAdconal:nHeight/3.5 // Posicao vertical da Terceira linha de gets do pagamento
245 #DEFINE POSHOR_PAYCOL1 000 // Posicao horizontal da segunda coluna de componentes
246 #DEFINE POSHOR_PAYCOL2 oPnlAdconal:nWidth/5 // Posicao horizontal da segunda coluna de componentes
247 #DEFINE LARG_GET_DATE 80 // Largura dos labels do pagamento
248 #DEFINE LARG_GET_VALOR 80
249 #DEFINE LARG_SAY 200 //Largura Padrao Para say na tela
250
251 //Chave para saber se esta com conexao ou nao com a retaguarda
252
253 #DEFINE CSECAO "CONEXAOPDV"
254 #DEFINE CCHAVE "CONECTADO"

```

Componentes/Item/STBCalculateTax.prw

1052 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "MATXDEF.CH"
4
5 Static aGrupTrib := {} // Armazena valores por alíquotas para rateio pelo método de impressora fiscal
6
7 //-----
8 /*{Protheus.doc} STBTaxIni
9 Inicializa o Calculo das operacoes Fiscais
10
11 @param cCliForCod      Codigo Cliente/Fornecedor
12 @param cCliForLoja     Loja do Cliente/Fornecedor
13 @param cCliOrFor       C:Cliente ou F:Fornecedor
14 @param cTypeNF         Tipo da NF
15 @param lFooterTax      Permite Incluir Impostos no Rodape
16 @param cAlias          Alias do Cadastro de Produtos
17 @param cRoutine        Nome da rotina que esta utilizando a funcao
18 @param lCalcIpi        Define se calcula IPI
19 @return Nil
20 @obs
21 @sample
22 */
23 //-----
24 Function STBTaxIni( cCliForCod , cCliForLoja , cCliOrFor , cTypeNF ,;
25                   lFooterTax , cAlias , cRoutine , lCalcIpi ,;
26                   cCliType , cTpFrete , lEmiteNFe)
27
28 Local lCfgTrib := If(FindFunction("LjCfgTrib"), LjCfgTrib(), .F.) //Verifica se Configurador de Tributos esta habilitado
29
30 Default cCliForCod      := ""           // Codigo Cliente/Fornecedor
31 Default cCliForLoja     := ""           // Loja do Cliente/Fornecedor
32 Default cCliOrFor       := ""           // C:Cliente ou F:Fornecedor
33 Default cTypeNF         := ""           // Tipo da NF
34 Default lFooterTax      := .F.          // Permite Incluir Impostos no Rodape
35 Default cAlias          := ""           // Alias do Cadastro de Produtos
36 Default cRoutine        := ""           // Nome da rotina que esta utilizando a funcao
37 Default lCalcIpi        := .F.          // Define se calcula IPI
38 Default cCliType        := ""           // Tipo do Cliente
39 Default cTpFrete        := ""           // Tipo de Frete (Via importação de Orçamento)
40 Default lEmiteNFe       := .F.          // Esse parametro somente vem quando a rotina é chamada na seleção de cliente.
41
42 LjGrvLog( "MaFisIni", "Inicio")
43
44 // Inicializa o Calculo das operacoes Fiscais
45 MaFisIni( cCliForCod ,; // 01-Codigo Cliente/Fornecedor
46          cCliForLoja ,; // 02-Loja do Cliente/Fornecedor
47          cCliOrFor ,; // 03-C:Cliente , F:Fornecedor
48          cTypeNF ,; // 04-Tipo da NF( "N","D","B","C","P","I","S" )
49          cCliType ,; // 05-Tipo do Cliente/Fornecedor
50          Nil ,; // 06-Relacao de Impostos que suportados no arquivo
51          Nil ,; // 07-Tipo de complemento
52          lFooterTax ,; // 08-Permite Incluir Impostos no Rodape .T./.F.

```

```

53      cAlias      ,; // 09-Alias do Cadastro de Produtos - ("SBI" P/ Front Loja)
54      cRoutine    ,; // 10-Nome da rotina que esta utilizando a funcao
55      Nil          ,; // 11-Tipo de documento
56      Nil          ,; // 12-Especie do documento
57      Nil          ,; // 13-Codigo e Loja do Prospect
58      Nil          ,; // 14-Grupo Cliente
59      Nil          ,; // 15-Recolhe ISS
60      Nil          ,; // 16-Codigo do cliente de entrega na nota fiscal de saida
61      Nil          ,; // 17-Loja do cliente de entrega na nota fiscal de saida
62      Nil          ,; // 18-Informacoes do transportador [01]-UF,[02]-TPTRANS
63      lEmitNF      ,; // 19- No momento o PDV nao emite NF , por isso sempre falso
64      lCalcIpi     ,; // 20-Define se calcula IPI (SIGALOJA)
65      Nil          ,; // 21-Pedido de Venda
66      Nil          ,; // 22-Cliente do Faturamento
67      Nil          ,; // 23-Loja do cliente do faturamento
68      Nil          ,; // 24-Total do Pedido
69      Nil          ,; // 25-Data de emissão do documento
70      cTpFrete     ,; // 26-Tipo de Frete
71      Nil          ,; // 27-Indica se Calcula (PIS,COFINS,CSLL), independete da TES estar configurada para Gerar Duplicata (F4_DUPLIC)
72      Nil          ,; // 28-Indica se Calcula (INSS), independete da TES estar configurada para Gerar Duplicata (F4_DUPLIC)
73      Nil          ,; // 29-Indica se Calcula (IRRF), independete da TES estar configurada para Gerar Duplicata (F4_DUPLIC)
74      Nil          ,; // 30-Tipo de Complemento
75      Nil          ,; // 31-Cliente de destino de transporte (Notas de entrada de transporte )
76      Nil          ,; // 32-Loja de destino de transporte (Notas de entrada de transporte )
77      lCfgTrib     ) // 33-Flag para indicar se os tributos genéricos devem ou não ser calculados - deve ser passado como .T. somente após a preparação da rotina
      para gravação,
78          ,; // visualização e exclusão dos tributos genéricos.
79
80 // Atualiza Total
81 STFRefTot()
82
83 LjGrvLog( "MaFisIni", "Final")
84
85 Return Nil
86
87
88 //-----
89 /*{Protheus.doc} STBTaxAdd
90 Inicializa o Calculo das operacoes Fiscais por item
91
92 @param cItemCode      Codigo do Produto
93 @param cTES           Codigo do TES
94 @param nQuantity      Quantidade
95 @param nPrice         Preco Unitario
96 @param nDiscount      Valor do Desconto
97 @param nItemTotal     Valor Total do Item
98 @return Nil
99 @obs
100 @sample
101 */
102 //-----
103 Function STBTaxAdd( cItemCode, cTES          , nQuantity, nPrice, ;

```

```

104         nDiscount, nItemTotal )
105
106 Default cItemCode      := ""           // Código do Produto
107 Default cTES           := ""           // Código do TES
108 Default nQuantity      := 0            // Quantidade
109 Default nPrice          := 0            // Preço Unitário
110 Default nDiscount       := 0            // Valor do Desconto
111 Default nItemTotal      := 0            // Valor Total do Item
112
113 LjGrvLog( "MaFisAdd", "Inicio")
114 // Inicializa o Calculo das operacoes Fiscais por item
115 MaFisAdd( cItemCode      ,; // 01-Código do Produto      ( Obrigatorio      )
116           cTES           ,; // 02-Código do TES          ( Opcional        )
117           nQuantity      ,; // 03-Quantidade      ( Obrigatorio      )
118           nPrice         ,; // 04-Preço Unitário   ( Obrigatorio      )
119           nDiscount       ,; // 05-Valor do Desconto ( Opcional        )
120           ""             ,; // 06-Numero da NF Original ( Devolucao/Benef )
121           ""             ,; // 07-Serie da NF Original ( Devolucao/Benef )
122           0              ,; // 08-RecNo da NF Original ( No arq SD1/SD2  )
123           0              ,; // 09-Valor do Frete do Item ( Opcional        )
124           0              ,; // 10-Valor da Despesa do item ( Opcional        )
125           0              ,; // 11-Valor do Seguro do item ( Opcional        )
126           0              ,; // 12-Valor do Frete Autonomo ( Opcional        )
127           nItemTotal     ,; // 13-Valor da Mercadoria ( Obrigatorio      )
128           0              ) // 14-Valor da Embalagem      ( Opcional        )
129
130 // Atualiza Total
131 STFRefTot()
132 LjGrvLog( "MaFisAdd", "Final")
133 Return Nil
134
135
136 //-----
137 /*{Protheus.doc} STBTaxRet
138 Retorna os impostos calculados
139
140 @param nItem           Item
141 @param cField          Campo
142 @return nRet - Imposto do item calculado
143 @obs
144 @sample
145 */
146 //-----
147 Function STBTaxRet( nItem, cField )
148
149 Local nRet              := 0              // Retorna imposto
150 Local lRecebTitle       := STIGetRecTit() // Indica se eh recebimento de titulos
151 Local oTotal            := STFGetTot()    // Totalizador
152 Local nTotSale          := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
153
154 Default nItem           := 0              // Item
155 Default cField          := ""             // Campo

```

```

156
157 // Se for Recebimento de Titulo retorna o valor total (L1_VLRTOT)
158 If lRecebTitle .AND. cField = "NF_TOTAL"
159     nRet := nTotSale
160 Else // Retorna os impostos calculados pela MATXFIS
161     nRet := MaFisRet( nItem, cField )
162 EndIf
163
164 Return nRet
165
166
167 //-----
168 /*{Protheus.doc} STBTaxAlt
169 Altera os valores de impostos e bases do item
170
171 @param cField      Campo
172 @param nValue      Valor
173 @param nItem       Linha do Item nas funcoes fiscais
174 @return nRet - Imposto do item alterado .T. ou .F.
175 @obs
176 @sample
177 */
178 //-----
179 Function STBTaxAlt ( cField, nValue , nItem )
180
181 Local lRet      := .F.          // Retorno
182
183 Default cField  := ""          // Campo
184 Default nValue  := 0           // Valor
185 Default nItem   := 0           // Item
186
187 LjGrvLog( "MaFisAlt", "Inicio")
188 //Altera os valores de impostos e bases do item
189 lRet := MaFisAlt( cField , nValue, nItem )
190
191 // Atualiza Total
192 STFRefTot()
193
194 LjGrvLog( "MaFisAlt", "Final")
195
196 Return lRet
197
198 //-----
199 /*{Protheus.doc} STBTaxDel
200 Marca/Desmarca o item especificado como deletado
201
202 @param nItem      Item
203 @param lDeleted   Marca se o item esta Deletado
204 @return Nil
205 @obs
206 @sample
207 */

```

```

208 //-----
209 Function STBTaxDel( nItem , lDeleted )
210
211 Local nX                := 0
212 Local aSATtrib          := {}
213 Local lExistGetTriSat    := LjUseSat() .AND. (ExistFunc("LjGetTriSat") .AND. ExistFunc("LjSetTriSat")) //Tributação Produto SAT
214                                     //Armazenamento para XML SAT
215 Default nItem           := 0 // Item
216 Default lDeleted        := .F. // Marca se o item esta Deletado
217
218 /*/
219     Marca/Desmarca o item especificado como deletado
220     lDelete := ( .T. Item Deletado ) , ( .F. Item Ativo )
221 /*/
222 LjGrvLog( "MaFisDel", "Inicio")
223 MaFisDel( nItem , lDeleted)
224 LjGrvLog( "MaFisDel", "Final")
225
226 If lExistGetTriSat
227     //Array de tributações do SAT
228     aSATtrib := LjGetTriSat() //Tributação produto SAT
229     For nX := 1 to Len(aSatTrib)
230
231         // O procedimento abaixo elimina toda a linha que foi marcada como "deletado" e redimensiona todo o array.
232         // Isto serve para envio de alguns impostos para o XML SAT, como PIS, COFINS, CST...
233         If nX <= Len(aSatTrib)
234             //Elemento 1 é o número do item no grid TOTVSPDV. Como não há como recuperar um item deletado, é deletado também neste array.
235             If aSatTrib[nX][1] = nItem
236                 aDel(aSatTrib,nX)
237                 aSize(aSatTrib,Len(aSatTrib)-1)
238                 nX--
239             Loop
240         EndIf
241     EndIf
242     Next
243     LjSetTriSat(aSATtrib)
244 EndIf
245
246 // Atualiza Total
247 STFRefTot()
248
249 Return Nil
250
251 //-----
252 /*/{Protheus.doc} STBTaxFoun
253 Verifica se o item ja existe na relacao de itens ja incluidos
254
255 @param cField      Campo
256 @param nItem       Item
257 @return lRet - encontrou item .T. ou .F.
258 @obs
259 @sample

```

```

260 */
261 //-----
262 Function STBTaxFoun(   cField, nItem   )
263
264 Local   lRet           := .F.           // Retorno
265
266 Default cField         := ""           // Campo
267 Default nItem          := 0            // Item
268
269 // Verifica se o item ja existe na relacao de itens ja incluidos
270 If Empty(cField) .AND. Empty(nItem)
271     lRet := MaFisFound()
272 Else
273     lRet := MaFisFound(cField,nItem)
274 EndIf
275
276 Return lRet
277
278
279 //-----
280 /*/{Protheus.doc} STBTaxIniL
281 Rotina inicializacao do item da funcao Fiscal
282
283 @param nItemLine      Numero do Item
284 @param lReversal      Indica se o item deve ser estornado caso exista
285 @param cItemCode     Codigo do Produto
286 @param cTES           Codigo do TES
287 @param nQuantity      Quantidade
288 @param nPrice         Preco Unitario
289 @param nDiscount      Valor do Desconto
290 @param nItemTotal     Valor Total do Item
291 @return lRet - Inicializou item .T. ou .F.
292 @obs
293 @sample
294 */
295 //-----
296 Function STBTaxIniL(   nItemLine   ,   lReversal   ,   cItemCode   , cTES           , ;
297                       nQuantity    ,   nPrice      ,   nDiscount   , nItemTotal   , )
298
299 Local lRet           := .F.           // Retorno
300 Local aItemLoad      := {}           // Array iniload da matxfis
301
302 Default nItemLine     := 0            // Item do Array que deve ser inicializado
303 Default lReversal     := .F.         // Indica se o item deve ser estornado caso exista
304 Default cItemCode     := ""          // Codigo do Produto
305 Default cTES          := ""          // Codigo do TES
306 Default nQuantity     := 0           // Quantidade
307 Default nPrice        := 0           // Preco Unitario
308 Default nDiscount     := 0           // Valor do Desconto
309 Default nItemTotal    := 0           // Valor Total do Item
310
311 LjGrvLog( "MaFisIniLoad", "Inicio")

```

```

312
313 Aadd( aItemLoad , cItemCode          ) // IT_PRODUTO
314 Aadd( aItemLoad , cTES                ) // IT_TES
315 Aadd( aItemLoad , ""                 ) // IT_CODISS
316 Aadd( aItemLoad , nQuantity           ) // IT_QUANT
317 Aadd( aItemLoad , Space(TamSx3("D1_NFORI"))[1]) ) // IT_SERIORI
318 Aadd( aItemLoad , Space(TamSx3("D1_SERIORI"))[1]) ) // IT_RECNO SB1
319 Aadd( aItemLoad , 0                   ) // IT_RECNO SF4
320 Aadd( aItemLoad , 0                   ) // IT_RECORI
321 Aadd( aItemLoad , Nil                 ) // IT_LOTECTL
322 Aadd( aItemLoad , ""                 ) // IT_NUMLOTE
323 Aadd( aItemLoad , ""                 ) // Sub-Lote Produto
324 Aadd( aItemLoad , Space(Len(cItemCode)) ) // Código do Produto Fiscal
325 Aadd( aItemLoad , 0                   ) // Recno do Produto Fiscal
326
327 // Rotina inicializacao do item da funcao Fiscal
328 If !MaFisFound("IT",nItemLine)
329     lRet := MaFisIniLoad( nItemLine , aItemLoad , lReversal )
330 Else
331     MaFisAlt("IT_PRODUTO",cItemCode,nItemLine)
332 EndIf
333
334 MaFisTes( cTES , 0 , nItemLine ) // Carrega a TES para a MATXFIS
335
336 MaFisLoad( "IT_VALMERC" , nItemTotal , nItemLine )
337
338 MaFisLoad( "IT_PRCUNI" , nPrice , nItemLine )
339
340 MaFisLoad( "IT_DESCONTO" , nDiscount , nItemLine )
341
342 MaFisRecal( "" , nItemLine ) // Dispara o calculo do item
343
344 MaFisEndLoad( nItemLine , 2 ) // Fecha o calculo do item e atualiza os totalizadores do cabeçalho
345
346 // Atualiza Total
347 STFRefTot()
348
349 LjGrvLog( "MaFisIniLoad", "Final", nItemLine)
350
351 Return lRet
352
353
354 //-----
355 /*{Protheus.doc} STBTaxLoad
356 Rotina inicializacao do item da funcao Fiscal
357
358 @param cField      Campo
359 @param nValue      Valor
360 @param nItem       Item
361 @return lRet - Inicializou item funcao fiscal .T. ou .F.
362 @obs
363 @sample

```

```

364 /*/
365 //-----
366 Function STBTaxLoad(   cField ,   nValue ,   nItem   )
367
368 Local lRet           := .F.           // Retorno
369
370 Default cField       := ""           // Campo
371 Default nValue       := 0            // Valor
372 Default nItem       := 0            // Item
373
374 LjGrvLog( "MaFisLoad", "Inicio")
375
376 // Rotina inicializacao do item da funcao Fiscal
377 lRet := MaFisLoad(   cField ,   nValue ,   nItem   )
378
379 LjGrvLog( "MaFisLoad", "Final")
380
381 Return lRet
382
383
384 //-----
385 /*/{Protheus.doc} STBTaxRef
386 Executa o calculo dos valores do item da NF
387
388 @param cReference      Referencia
389 @param cRoutine        Identificador do arquivo
390 @param cValue          Valor da Referencia
391 @return lRet - MetoExecutado com sucesso .T. ou .F.
392 @obs
393 @sample
394 /*/
395 //-----
396 Function STBTaxRef (   cReference , cRoutine ,   cValue)
397
398 Local lRet           := .F.           // Retorno
399
400 Default cReference   := ""           // Campo
401 Default cRoutine     := ""           // Valor
402 Default cValue       := ""           // Item
403
404 lRet := MaFisRef (   cReference , cRoutine ,   cValue )
405
406 Return lRet
407
408
409 //-----
410 /*/{Protheus.doc} STBTaxSave
411 Salva a NF atual em uma area temporaria
412
413 @param
414 @return Nil
415 @obs

```

```

416 @sample
417 /*/
418 //-----
419 Function STBTaxSave()
420
421 // Salva a NF atual em uma area temporaria
422 MaFisSave()
423
424 Return Nil
425
426
427 //-----
428 /*/{Protheus.doc} STBTaxEnd
429 Finaliza a carga dos itens Fiscais
430
431 @param
432 @return Nil
433 @obs
434 @sample
435 /*/
436 //-----
437 Function STBTaxEnd()
438
439 // Finaliza a carga dos itens Fiscais
440 MaFisEnd()
441
442 Return Nil
443
444
445 //-----
446 /*/{Protheus.doc} STBTaxTES
447 Rotina busca TES do Produto considerando Tes Padrao,
448 TES do Cadastro de produtos e TES Inteligente.
449
450 @param nDocImpOut Documento de 1-Entrada / 2-Saida
451 @param cTypeOper Tipo de Operacao Tabela "DF" do SX5
452 @param cCliForCodCodigo do Cliente ou Fornecedor
453 @param cCliForLoja Loja do Cliente ou Fornecedor
454 @param cTipoCF Tipo Cliente ou Fornecedor
455 @param cItemCodigo Item
456 @param cFieldCampo
457 @param cTESItem TES do Item (Produto)
458 @return cRetTES - Retorna TES
459 @obs
460 @sample
461 /*/
462 //-----
463
464 Function STBTaxTES( nDocImpOut , cTypeOper , cCliForCod , cCliForLoja ,;
465 cCliOrFor , cItem , cField , cTESItem ,;
466 lListProd)
467

```

```

468 Local cSmartTES      := ""           // TES Inteligente
469 Local cRetTES        := ""           // Retorno código da TES
470 Local cTESList       := SuperGetMV("MV_LJLPTSV", "") // Tipo de Entrada/Saida padrao para o Pedido de Venda
471 Local cTpOperLst     := SuperGetMV("MV_LJLPTIV", "") // Tipo da Operacao para o Pedido de Venda (TES Inteligente)
472 Local xRet           := ""           // Retorno de MacroExecução da Lista
473
474
475 Default nDocImpOut     := 0           // Documento de 1-Entrada / 2-Saida
476 Default cTypeOper     := ""           // Tipo de Operacao Tabela "DF" do SX5
477 Default cCliForCod     := ""           // Código do Cliente ou Fornecedor
478 Default cCliForLoja    := ""           // Loja do Cliente ou Fornecedor
479 Default cCliOrFor      := ""           // Tipo Cliente ou Fornecedor
480 Default cItem          := ""           // Código Item
481 Default cField         := ""           // Campo
482 Default cTESItem       := ""           // TES do Item
483 Default lListProd      := .F.         // Produto Lista
484
485 LjGrvLog( "MaTesInt", "Inicio", cItem)
486 If !lListProd
487     //Retorna o conteudo do parâmetro MV_LJOPTES
488     If ExisteSx6("MV_LJOPTES") .And. ExistFunc("LjOpTESInt")
489         cTypeOper := LjOpTESInt()
490     EndIf
491
492     // Busca TES inteligente
493     cSmartTES := MaTesInt( nDocImpOut , cTypeOper , cCliForCod , cCliForLoja ;
494                          cCliOrFor , cItem , cField )
495 Else
496     If !Empty(cTpOperLst)
497         If Substr(cTpOperLst,1,1)= "&"
498             xRet := &(SubStr(cTpOperLst,2,Len(cTpOperLst)))
499             If ValType(xRet) == "C"
500                 cTpOperLst := xRet
501             EndIf
502         EndIf
503     EndIf
504
505     If !Empty(cTpOperLst)
506         cSmartTES := MaTesInt( nDocImpOut , cTpOperLst , cCliForCod , cCliForLoja ;
507                              cCliOrFor , cItem , cField )
508     EndIf
509 EndIf
510 EndIf
511
512
513 /*/
514 Prioridades da utilizacao da TES:
515 1 - TES Inteligente
516 2 - TES do Item (se Lista)
517 3 - TES do Item (Produto)
518 4 - TES Padrao
519 /*/

```

```

520
521 If !Empty(cSmartTES)
522     // TES Inteligente
523     cRetTES := cSmartTES
524 ElseIf lListProd .AND. !Empty(cTESList)
525     cRetTES := cTESList
526 ElseIf !Empty(cTESItem)
527     // TES do Item
528     cRetTES := cTESItem
529 Else
530     // TES Padrao
531     cRetTES := SuperGetMV("MV_TESSAI",,"501")
532 EndIf
533
534 LjGrvLog( "MaTesInt", "Final", cItem)
535
536 Return cRetTES
537
538 //-----
539 /*{Protheus.doc} STBTaxSit
540 Verifica a situacao tributaria do item
541
542 @param  nItem          Numero do item na venda
543
544 @return  cSitTrib - Retorna Situacao tributaria do Item
545 @obs     Usar este metodo somente apos ter adicionado o item
546           nas funcoes fiscais
547 @sample
548 */
549 //-----
550 Function STBTaxSit( nItem )
551 Local cTaxSit      := ""           // Situacao tributaria do Item
552 Local cClasfis     := ""           // Classificacao fiscal
553 Local nAliqRed     := 0            // Valor da aliq reduzida
554 Local lEmitNFCe    := STBGetNFCE() // valida se é NFC-e ou não
555 Local lEmitSAT     := STFGetCfg("lUseSAT",.F.)
556 Local nAliqISS     := 0
557 Local aDadoTES     := {}
558 Local lAlqIssIse:= .F.
559
560 Default nItem      := 0            // Numero do item na venda
561
562 //tratamento para verificar se está no modo SAT
563 lEmitSAT := IIf( ValType(lEmitSAT) == "U", .F., lEmitSAT)
564
565 /*
566     Verifica se o item ja existe nas funcoes fiscais
567 */
568 If STBTaxFoun("IT",nItem)
569
570     /*
571         TABELA - CODIGO DE SITUACAO TRIBUTARIA DO ICMS

```

```

572
573     CODIGO      -   DESCRIÇÃO
574     00          -   Tributada integralmente
575     10          -   Tributada e com cobrança do ICMS por substituição tributária
576     20          -   Com redução de base de cálculo
577     30          -   Isenta ou não tributada e com cobrança do ICMS por substituição tributária
578     40          -   Isenta
579     41          -   Não tributada
580     50          -   Suspensão
581     51          -   Diferimento
582     60          -   ICMS cobrado anteriormente por substituição tributária
583     70          -   Com redução de base de cálculo e cobrança do ICMS por substituição tributária
584     90          -   Outros
585 */
586
587 // Pega classificação fiscal
588 cClasfis := SubStr(STBTaxRet(nItem,"IT_CLASFIS"),2,2)
589 aDadoTES := STBTaxRet(nItem,"IT_TS")
590
591 /*Segundo regras do Fiscal para considerar a isenção de ISS
592 é necessário a regra abaixo para que não seja feito nenhum cálculo*/
593 If aDadoTES[03] == "N" .And. aDadoTES[20] == "N"
594     /*
595         Foi avaliado somente permitir a configuração dos tipos 07 e 06
596         no caso do Isento pois os outros tipos se referem a modalidades
597         que não são comumente usadas no PDV
598     */
599     If aDadoTES[21] == "I" .And. ( ( AllTrim(aDadoTES[112]) $ "07|06" ) .Or. (Empty(AllTrim(aDadoTES[112]))))
600         If AllTrim(aDadoTES[112]) == "07" //Não tributado
601             cTaxSit := "NS"
602         ElseIf AllTrim(aDadoTES[112]) == "06" //Isento
603             cTaxSit := "IS1"
604         Else
605             //Caso não esteja configurado, considero isento
606             cTaxSit := "IS1" //Isento
607         EndIf
608         lAlqIssIse := .T.
609     ElseIf aDadoTES[21] == "0"
610         cTaxSit := "NS"
611         lAlqIssIse := .T.
612     EndIf
613 EndIf
614
615 If !lAlqIssIse
616     If AllTrim(aDadoTES[20]) == "S" // Verifica se ISS
617         nAliqISS := STBTaxRet(nItem,"IT_ALIQISS")
618         If nAliqISS > 0
619             cTaxSit := "S" + AllTrim (Str(nAliqISS,5,2 ))
620         Else
621             cTaxSit := "S" + AllTrim (Str(SuperGetMv("MV_ALIQISS"),5,2 ))
622         EndIf
623

```

```

624     ElseIf lEmitNFCe .AND. !lEmitSAT
625         cTaxSit := AllTrim( Str(STBTaxRet(nItem,"IT_ALIICM"),5,2) )
626     Else
627         Do Case
628             Case cClasfis == "40" // Verifica se e Isento("I")
629                 cTaxSit := "I"
630
631             Case cClasfis == "41" // Verifica se e Nao sujeito a ICMS("N")
632                 cTaxSit := "N"
633
634             Case cClasfis == "60" // Verifica se e Substituicao tributaria (Icms Solidario)("F")
635                 cTaxSit := "F"
636
637             Case cClasfis $ "00|10|20|70" // Verifica se e alguma forma tributada ICMS("T")
638                 cTaxSit := "T" + AllTrim(Str(STBTaxRet(nItem,"IT_ALIICM"),5,2))
639
640                 //Caso possua reducao de base vai buscar a aliquota reduzida
641                 If cClasfis $ "20|70"
642                     nAliqRed := STDAlqRed() // Vai verificar se o produto possui aliquota reduzida la na SB0
643                     If nAliqRed > 0
644                         cTaxSit := "T" + AllTrim(Str(nAliqRed, 5,2))
645                     EndIf
646                 EndIf
647
648             Otherwise
649                 // Se nao Caiu em nenhum dos casos
650                 // Pode nao ter preenchido o campo classificacao fiscal da TES.
651                 // Sera considerado tributado caso tenha aliquota
652                 If !Empty( AllTrim(Str(STBTaxRet(nItem,"IT_ALIICM"),5,2)) )
653                     cTaxSit := "T" + AllTrim(Str(STBTaxRet(nItem,"IT_ALIICM"),5,2))
654                 EndIf
655             EndCase
656         EndIf
657     EndIf
658 EndIf
659
660 Return cTaxSit
661
662 //-----
663 /*{Protheus.doc} STBStAdjust
664 Ajusta a string que vai gravar o tipo de tributacao do item
665
666 @param cSitTrib          Situacao tributaria do item
667 @return cSitTrib          Situacao tributaria do item ajustada
668 */
669 //-----
670 Function STBStAdjust( cSitTrib )
671 Local cSitSFT           := ""           // Situacao tributaria
672 Local cAliqIcm          := ""           // Aliquota
673
674 Default cSitTrib       := ""           // Situacao tributaria
675

```

```

676 //No caso de Tributado , tem que ser da mesma aliquota
677 If "T" $ cSitTrib
678
679     cAliqIcm      := StrTran(cSitTrib," ","")
680     cSitSFT       := StrTran(cAliqIcm,".", "")
681     cSitSFT       := SubStr(cSitSFT,2,Len(cSitSFT))
682     cAliqIcm      := SubStr(cAliqIcm,2,Len(cAliqIcm))
683
684     If Len(cSitSFT) == 1 .OR. Len(SubStr(cAliqIcm,1,At(".",cAliqIcm)-1)) == 1
685         cSitSFT := "0"+cSitSFT
686     EndIf
687
688     cSitTrib := "T"+PadR(cSitSFT,4,"0")
689
690 ElseIf "S" $ cSitTrib
691
692     If SubStr(cSitTrib,1,1) $ "F#I#N" // FS1 - NS1 - IS1
693         cSitTrib := cSitTrib
694     Else
695         cAliqIcm      := StrTran(cSitTrib," ","")
696         cSitSFT       := StrTran(cAliqIcm,".", "")
697         cSitSFT       := SubStr(cSitSFT,2,Len(cSitSFT))
698         cAliqIcm      := SubStr(cAliqIcm,2,Len(cAliqIcm))
699
700         If Len(cSitSFT) == 1 .OR. Len(SubStr(cAliqIcm,1,At(".",cAliqIcm)-1)) == 1
701             cSitSFT := "0"+cSitSFT
702         EndIf
703         cSitTrib := "S"+PadR( cSitSFT ,4,"0")
704     EndIf
705
706 ElseIf SubStr(cSitTrib,1,Len(cSitTrib)) $ "F#I#N"
707
708     cSitTrib += "1"
709
710 EndIf
711
712 Return cSitTrib
713
714 //-----
715 /*{Protheus.doc} STBStartTax
716 Inicia funções fiscais ao inicializr o sistema
717
718 @return Nil
719 @obs
720 @sample
721 */
722 //-----
723 Function STBStartTax()
724
725 Local cCliCode      := STDGPBasket("SL1","L1_CLIENTE") // Código do Cliente
726 Local cCliLoja      := STDGPBasket("SL1","L1_LOJA") // Loja do Cliente
727 Local cCliType      := STDGPBasket("SL1","L1_TIPOCLI") // Tipo do Cliente

```

```

728
729 If !(STBTaxFoun())
730
731     STBTaxIni( cCliCode    ,    cCliLoja    ,    "C"        ,    "S"        ,;
732               .F.         ,    "SB1"       ,    "LOJA701"   ,    .T.       ,;
733               cCliType    )
734
735 EndIf
736
737 Return Nil
738
739
740 //-----
741 /*{Protheus.doc} STBTaxGrupTrib
742 Armazena valores para Rateio do desconto e acréscimo no Total por alíquota
743 de acordo com o método das impressoras fiscais. (Chamada pela MATXFIS)
744
745 @param aNfCab           Cabeçalho Nota Fiscal
746 @param aNfItem          Itens Nota Fiscal
747 @param nItem            Numero do Item
748 @param cReferencia      Referencia
749 @return Nil
750 @obs
751 @sample
752 */
753 //-----
754 Function STBTaxGrupTrib( aNfCab , aNfItem , nItem , cReferencia )
755
756 Local nPosAliq    := 0 // Posicao aliquota ICMS no array aGrupTrib
757
758 Default aNfCab    := {}
759 Default aNfItem   := {}
760 Default nItem     := 0
761 Default cReferencia := ""
762
763 If Len(aNfCab) > 0 .AND. Len(aNfItem) > 0 .AND. nItem > 0 .AND. !Empty(cReferencia)
764
765     // 1º Passo: Agrupar e armazenar todos os valores por alíquota
766     nPosAliq := aScan( aGrupTrib , { |x| x[1] == aNfItem[nItem][IT_ALIICM] } )
767     If nPosAliq > 0
768         If aNfItem[nItem][IT_TOTAL] == aGrupTrib[nPosAliq][04] //Item de mesmo valor, adiciona item para depois distribuir o residuo entre os itens
769             aAdd( aGrupTrib[nPosAliq][03], nItem )
770         Else
771             If (cReferencia == "IT_DESCTOT")
772                 If aNfItem[nItem][IT_TOTAL] > aGrupTrib[nPosAliq][04]
773                     aGrupTrib[nPosAliq][03] := {nItem}
774                     aGrupTrib[nPosAliq][04] := aNfItem[nItem][IT_TOTAL]
775                 EndIf
776             ElseIf (cReferencia == "IT_ACRESCI")
777                 If aNfItem[nItem][IT_TOTAL] < aGrupTrib[nPosAliq][04]
778                     aGrupTrib[nPosAliq][03] := {nItem}
779                     aGrupTrib[nPosAliq][04] := aNfItem[nItem][IT_TOTAL]

```

```

780         EndIf
781     EndIf
782 EndIf
783 aGrupTrib[nPosAliq][02] += aNfItem[nItem][IT_TOTAL]
784 aAdd( aGrupTrib[nPosAliq][05], { nItem , aNfItem[nItem][IT_TOTAL] , 0 } )
785 Else
786     aAdd( aGrupTrib , {} )
787     nPosAliq := Len(aGrupTrib)
788     aAdd (aGrupTrib[nPosAliq], aNfItem[nItem][IT_ALIQICM] ) //01 - Tipo Tributacao
789     aAdd (aGrupTrib[nPosAliq], aNfItem[nItem][IT_TOTAL] ) //02 - Valor da Soma dos itens desta Tributacao
790     aAdd (aGrupTrib[nPosAliq], {nItem} ) //03 - Posicao do item de Maior/Menor Valor
791     aAdd (aGrupTrib[nPosAliq], aNfItem[nItem][IT_TOTAL] ) //04 - Valor Maior/Menor
792     aAdd (aGrupTrib[nPosAliq], {{nItem,aNfItem[nItem][IT_TOTAL],0}} ) //05 - Posicao do Item ; Valor dos itens desta Tributacao SEM descontos ou acrescimos
    proporcionalizados ; L2_ITEM jah COM descontos ou acrescimos proporcionalizados
793     aAdd (aGrupTrib[nPosAliq], 0 ) //06 - Valor de Desconto ou Acréscimo relacionado a esta Tributacao (de acordo com a regra
    de rateio da impressora)
794 EndIf
795 EndIf
796
797 Return
798
799
800 //-----
801 /*{Protheus.doc} STBTaxRateio
802 Rateia Desconto e Acréscimo pelo mesmo método que as impressoras fiscais
803
804 @param aNfCab           Cabeçalho Nota Fiscal
805 @param aNfItem          Itens Nota Fiscal
806 @param aPosCpo          Posição campo de referência
807 @param cReferencia      Referencia
808 @param nDecimais        Decimais
809 @return Nil
810 @obs
811 @sample
812 */
813 //-----
814 Function STBTaxRateio( aNfCab , aNfItem , aPosCpo , cReferencia ,;
815                      nDecimais )
816
817 Local nAcumula          := 0 // Acumula os descontos/acrescimos rateados
818 Local nResiduo          := 0 // Armazena resíduo do rateio
819 Local nItResiduo        := 0 // Residuo de 0.1 a aplicar no item
820 Local nJ                := 0 // Contador
821 Local nX                := 0 // Contador
822 Local nPercEfet         := 0 // Percentual efetivo do rateio pelo método de impressora fiscal
823 Local nPercProd         := 0 // Percentual do produto em relação ao total da alíquota
824 Local nImpDecimais      := 2 // Decimais utilizadas de acordo com a impressora. Ex: Epson = 15 ; Bematech = 2
825 Local nPosMaiorItem     := 0 // Posicao do maior item do grupo de tributacao em relacao a todo array.
826 Local nPosItArrMaior    := 0 // Posicao do maior item em relacao ao array do grupo de tributacao especifico
827 Local nMaiorAliq        := 1 // Maior aliquota dentre os grupos de tributacao
828 Local nPercProduto      := 0 // Percentual do produto em relacao a seu valor sobre o desconto do grupo de tributacao
829 Local lEmitSAT           := STGetCfg("lUseSAT",.F.) // Verifica se eh venda em estacao SAT

```

```

830
831 Default aNfCab      := {}
832 Default aNfItem     := {}
833 Default aPosCpo      := ""
834 Default cReferencia := ""
835 Default nDecimais    := 2
836
837 nImpDecimais := STBTaxDec('2') // 1 = Grava no ini , 2 = Recupera informacao do ini
838
839 If Len(aNfCab) > 0 .AND. Len(aNfItem) > 0 .AND. Len(aPosCpo) > 0 .AND. Len(aGrupTrib) > 0
840
841 // 2º Passo: Calcula o Percentual efetivo do Desconto ou do Acrescimo (truncado em duas casas decimais)
842 If (cReferencia == "IT_DESCTOT")
843     nPercEfet := NoRound( (( aNfCab[NF_DESCTOT] / aNfCab[NF_TOTAL] ) * 100), nImpDecimais ) / 100
844 ElseIf (cReferencia == "IT_ACRESCI")
845     nPercEfet := NoRound( (( aNfCab[NF_ACRESCI] / aNfCab[NF_TOTAL] ) * 100), nImpDecimais) / 100
846 EndIf
847
848 // Calcula e Acumula os valores aplicando o percentual efetivo no total por alíquotas
849 For nX := 1 To Len(aGrupTrib)
850
851     aGrupTrib[nX][06] := NoRound( aGrupTrib[nX][02] * nPercEfet, 2 )
852
853     nAcumula += aGrupTrib[nX][06]
854
855 Next nX
856
857 // Calculo do residuo
858 If (cReferencia == "IT_DESCTOT")
859     nResiduo := aNfCab[NF_DESCTOT] - nAcumula
860 ElseIf (cReferencia == "IT_ACRESCI")
861     nResiduo := aNfCab[NF_ACRESCI] - nAcumula
862 EndIf
863
864 // Verifica qual a alíquota que tem maior valor total
865 If Abs(nResiduo) > 0
866     aGrupTrib := aSort( aGrupTrib ,,,{|x,y| x[2] > y[2]}) // Ordena por ordem crescente de totais por alíquotas
867     nMaiorAliq := 1
868     //Se os valores dos grupos de aliquota forem iguais, o residuo ira sempre para a maior aliquota.
869     For nX := 1 To Len(aGrupTrib)
870         If Len(aGrupTrib) >= nX + 1
871             If aGrupTrib[nX][2] == aGrupTrib[nX+1][2] // Valores iguais
872                 If aGrupTrib[nX+1][1] > aGrupTrib[nX][1]
873                     nMaiorAliq := nX + 1
874             EndIf
875         Else
876             Exit
877         EndIf
878     EndIf
879 Next nX
880 EndIf
881

```

```

882 // Atribui resíduo para alíquota de maior valor total
883 If nMaiorAliq > 0 .AND. nResiduo <> 0
884     aGrupTrib[nMaiorAliq][6] += nResiduo
885 EndIf
886
887 // Laço de Ajuste dos Itens
888 For nX := 1 To Len(aGrupTrib)
889
890     nAcumula := 0
891     nResiduo := 0
892
893     // Percorre todos os itens deste grupo de Tributacao para acertar os valores de Desconto ou Acrescimo de cada item
894     For nJ := 1 To Len(aGrupTrib[nX][05])
895
896         //Calcula a proporcionalizacao de Acrescimo ou Desconto para este item
897         nPercProduto := aGrupTrib[nX][05][nJ][02] / aGrupTrib[nX][02] // Valor do Item pelo total da alíquota
898
899         //Calcula o valor de Acrescimo ou Desconto para este item
900         If lEmitSAT
901             // Arredondamento conforme a regra do SAT
902             aGrupTrib[nX][05][nJ][03] := LjArredSat( (aGrupTrib[nX][06] * nPercProduto) , nDecimais )
903         Else
904             aGrupTrib[nX][05][nJ][03] := NoRound( (aGrupTrib[nX][06] * nPercProduto) , nDecimais )
905         EndIf
906
907
908         // Atribui à referência do Item
909         aNfItem[aGrupTrib[nX][05][nJ][01]][Val(aPosCpo)] := aGrupTrib[nX][05][nJ][03]
910
911         nAcumula += aNfItem[aGrupTrib[nX][05][nJ][01]][Val(aPosCpo)]
912
913     Next nJ
914
915     // Calculo do residuo
916     If (cReferencia == "IT_DESCTOT")
917         nResiduo := aGrupTrib[nX][6] - nAcumula
918     ElseIf (cReferencia == "IT_ACRESCI")
919         nResiduo := aGrupTrib[nX][6] - nAcumula
920     EndIf
921
922     // Aplica o resíduo no Item de Maior/Menor Valor (posição 3 do aGrupTrib)
923     If Abs(nResiduo) > 0
924         nPosMaiorItem := 0
925         nPosItArrMaior := 0
926         nPosMaiorItem := aGrupTrib[nX][03][1]
927         nPosItArrMaior := aScan( aGrupTrib[nX][5] , { |x| x[1] == nPosMaiorItem } )
928
929         If nPosItArrMaior > 0
930             aGrupTrib[nX][05][nPosItArrMaior][03] += nResiduo
931             aNfItem[aGrupTrib[nX][05][nPosItArrMaior][01]][Val(aPosCpo)] := aGrupTrib[nX][05][nPosItArrMaior][03]
932         EndIf
933

```

```

934      EndIf
935
936      Next nX
937
938 EndIf
939
940 aGrupTrib := {}
941
942 Return Nil
943
944 //-----
945 /*{Protheus.doc} STBTaxDec
946 Grava e recupera no sigaloja.ini o numero de decimais utilizados
947 nos ecfs para os calculos de rateio de desconto
948 @param cTypeOper      Tipo de operacao 1= Grava | 2= Recupera
949
950 @return  nRet - Casas decimais que são usadas na regra do ECF
951 @obs
952 @sample
953 */
954 //-----
955 Function STBTaxDec(cTypeOper)
956
957 Local cDecimais := "2" //Padrao bematech Sweda, de decimais para truncamento de valor no calculo no ECF
958 Local cEcf      := STFGetStation( 'IMPFISC' )
959 Local lAutomato  := If(Type("lAutomatoX")<>"L",.F.,lAutomatoX)
960 Local cGetCliDir := If(lAutomato,"",GetClientDir())
961 Local nRet       := 2 //Retorno Padrao
962
963 Default cTypeOper := "0"
964
965 If cTypeOper == "1"
966     // grava a informação no sigaloja.ini para ser utilizada na regra de desconto e rateio no FRTA271G, igual ao do ECF
967     If 'EPSON' $ cEcf .OR. 'ITAU' $ cEcf
968         cDecimais := "15"
969     EndIf
970
971     WritePProString( "Decimais ECF", "Decimais", cDecimais , cGetCliDir + "SIGALOJA.INI" )
972
973 ElseIf cTypeOper == "2"
974     cDecimais := GetPvProfString("Decimais ECF", "Decimais", " ", cGetCliDir + "SIGALOJA.INI" ) // Ler qtd decimais usada no ECF. Informacao grv na inicializacao do
    sistema
975 EndIf
976
977 If !Empty(cDecimais) .AND. Val(cDecimais) > 0
978     nRet := Val( cDecimais )
979 EndIf
980
981 Return nRet
982
983 /*{Protheus.doc} STBEmiteNF
984 chama a rotina LjNFFimVd() para definir se emite NF-e

```

```

985     @type Function
986     @param cCliCod,      caractere, código do cliente
987     @param cCliLoja,     caractere, código da loja onde cliente está cadastrado
988     @return lEmiteNFe,   lógico, .T. se emite NF-e
989     /*/
990 Function STBEmiteNF(cCliCod, cCliLoja)
991 Local lCliPadrao := .F.
992 Local lMVFISNOTA := SuperGetMV("MV_FISNOTA",.F.,.F.) .AND. !Empty(AllTrim( SuperGetMV("MV_LOJANF",.F., "UNI"))) .AND. SuperGetMv("MV_LJLBNT",.F.,0)>0
993 Local lEmiteNFe := .F.
994
995 Default cCliCod := ""
996 Default cCliLoja := ""
997
998 If !STFGetCfg("lPafEcF") .AND. !STFGetCfg("lPAFNFCE")
999
1000     lCliPadrao:= (AllTrim(cCliCod)+AllTrim(cCliLoja)) == ( AllTrim( SuperGetMV("MV_CLIPAD", .F., "")) + AllTrim( SuperGetMV("MV_LOJAPAD", .F., "")) )
1001
1002     If lMVFISNOTA .and. !lCliPadrao
1003         lEmiteNFe := LjNFFimVd(.T., .F., .F.) //Altera para emissao de Nota Fiscal
1004     EndIf
1005
1006 EndIf
1007
1008 Return lEmiteNFe
1009
1010 /*/{Protheus.doc} STBGetTax
1011 Retorna informacoes dos tributos utilizados no Configurador de tributos
1012
1013 @type function
1014
1015 @param nItem, numérico, Item a ser consultado
1016
1017 @return json, Informacoes dos tributos do Configurador de Tributos
1018 /*/
1019 Function STBGetTax(nItem as numeric) as json
1020
1021 Local lCfgTrib := If(FindFunction("LjCfgTrib"), LjCfgTrib(), .F.) //Verifica se Configurador de Tributos esta habilitado
1022 Local jCfgTaxes := JsonObject():New()
1023
1024 If lCfgTrib
1025     jCfgTaxes := LjCfgTaxes("",nItem)
1026 EndIf
1027
1028 Return jCfgTaxes
1029
1030 /*/{Protheus.doc} STBVerTotTax
1031 Verifica se tributo soma ou subtrai no total da venda
1032
1033 @type function
1034
1035 @param cIdTributo, caracter, Id do tributo
1036 @param nTpVerif, numérico, Soma ou subtrai no total

```

```
1037
1038 @return lógico, Acao realizada no total
1039 /*/
1040 Function STBVerTotTax(cIdTributo as character, nTpVerif as numeric) as logical
1041
1042 Local lCfgTrib as logical
1043 Local lRet as logical
1044
1045 lCfgTrib := If(FindFunction("LjCfgTrib"), LjCfgTrib(), .F.) //Verifica se Configurador de Tributos esta habilitado
1046 lRet := .F.
1047
1048 If lCfgTrib
1049     lRet := LjCfgAgreg(cIdTributo, nTpVerif)
1050 EndIf
1051
1052 Return lRet
```

Componentes/Item/STBCancelItem.prw

278 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBCANCELITEM.CH"
4 #INCLUDE "STPOS.CH"
5
6 //-----
7 /*{Protheus.doc} STBValCnItem
8 Valida o cancelamento do Item
9
10 @param  nItem          Numero do item na venda
11 @param  cItemCode      Codigo do Item
12 @return lRet - Retorna validacao da funcao
13 @obs
14 @sample
15 */
16 //-----
17 Function STBValCnItem( nItem , oModelCesta )
18
19 Local lRet          := .F.           //Retorno
20 Local aLines        := FwSaveRows()  //Linhas do model
21 Local nTotItens     := 0             //Total de Itens
22 Local nI            := 0             //Itens da Cesta
23 Local llJlsPre      := SuperGetMv("MV_LJLSPRE",, .F.) //Funcionalidade de Lista de Presente Ativa
24 Local lItFiscNFi    := .F.           //Verifica se existe o item fiscal e não fiscal para cancelamento no ECF (FieldPos)
25 Local aEstrItNFisc  := {}           //Estrutura do Item Cancelado (FieldPos)
26
27 Default nItem       := 0             // Numero do item na venda
28 Default oModelCesta := Nil
29
30
31 ParamType 0 Var nItem      AS Numeric   Default 0
32 ParamType 1 Var oModelCesta AS Object   Default Nil
33
34
35 //verifica se existe o item fiscal e não fiscal para cancelamento de item no ECF (FieldPos do Model)
36 aEstrItNFisc := STDGetProperty( "L2_ITFISC" )
37
38 lItFiscNFi := Len(aEstrItNFisc) > 0
39
40
41 /*/
42   Verificar se o item existe nas funcoes fiscais
43 /*/
44 If ( STBTaxFoun("IT",nItem) )
45
46   oModelCesta:GoLine(nItem)
47
48   /*/
49   Valida se o item e de um orcamento importado, nao permitindo o seu cancelamento
50   /*/
51   If Empty(oModelCesta:GetValue("L2_NUMORIG")) .OR. oModelCesta:GetValue("L2_VENDIDO") == "N"
52     lRet := .T.

```

```

53 Else
54     lRet := .F.
55     STFMessage("STCancelItem","STOP",STR0001)          //"Atencao, Nao e possivel cancelar itens de orcamentos importados"
56 EndIf
57
58 // Verifica se houve alguma transação tef, impossibilita sua continuação
59 If (STWChkTef("CC") .Or. STWChkTef("PD") .Or. STWChkTef("PX")) .And. STIGetCard() .And. STIGetTotal() > 0
60     lRet := .F.
61     STFMessage("STCancelItem","STOP",STR0010) //"Atenção, não é possível cancelar itens, após pagamento parcial"
62 EndIf
63
64 Else
65     lRet := .F.
66     STFMessage("STCancelItem","STOP",STR0002) //"Atencao, Item nao encontrado nas funcoes fiscais"
67 EndIf
68
69 If lRet
70     If oModelCesta:IsDeleted()
71         lRet := .F.
72         STFMessage("STCancelItem","STOP",STR0007) //"Este item já foi cancelado!"
73     EndIf
74 EndIf
75
76 STFS showMessage("STCancelItem")
77
78 FwRestRows(aLines)
79
80 Return lRet
81
82
83 //-----
84 /*{Protheus.doc} STBCnAllItem
85 Valida se e possivel cancelar todos ou so o ultimo item
86
87 @param  nItem          Numero do item na venda
88 @param  cItemCode      Codigo do Item
89 @return lRet - Retorna validacao da funcao
90 @obs
91 @sample
92 */
93 //-----
94 Function STBCnAllItem()
95
96 Local aLines      := FwSaveRows()          // Linhas do model
97 Local nRet        := 0                    // Retorno
98 Local cRetStatus  := "TODOS"              // Retorno evento
99 Local oModelCesta := STDGPBModel()        // Model da cesta de vendas
100
101 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
102
103 // Inicia Evento
104 STFFireEvent(ProcName(0), "GetStatusECF", { "4", @cRetStatus } )

```

```

105
106 // Caso apenas o ultimo item da impressora possa ser cancelado, entra no IF
107 If Upper(AllTrim(cRetStatus))<>"TODOS"
108
109     oModelCesta:GoLine(oModelCesta:Length())
110     If oModelCesta:IsDeleted()
111         nRet := 1
112     Else
113         nRet := 2
114     EndIf
115 Else
116     nRet := 3
117 EndIF
118
119 FwRestRows(aLines)
120
121 Return nRet
122
123 //-----
124 /*{Protheus.doc} STBCnFindItem
125 Realiza a busca do item que sera cancelado.
126
127 @param   cItemCode      Codigo do Item
128 @param   oModelCesta    Model da cesta de vendas
129 @param   nItem           numero do item
130 @param   nQuantCancel    Quantidade a cancelar
131 @return  lRet - Retorna validacao da funcao
132 @obs
133 @sample
134 */
135 //-----
136 Function STBCnFindItem( cGetProd )
137 Local nRet      := 0 //Retorno com o numero do item na cesta
138 Local nX        := 0 //Variavel para For
139 Local nMvljTpCnc := SuperGetMV("MV_LJTPCNC",,0) //Parametro que determina o tipo da busca no cancelamento onde 0 realiza a busca por codigo do produto e 1 por
140               numero do item na cesta
141 Local cRet      := ''
142
143 cGetProd := AllTrim(cGetProd)
144
145 If !Empty(cGetProd)
146
147     If nMvljTpCnc == 0 //Priorizo a busca por codigo do produto
148         cRet := STBCncPPrd(cGetProd)
149     Else
150         If STBIsDigit(cRet)
151             nRet := Val(cRet)
152         Else
153             nRet := STBPegaIT(cRet)
154         EndIf
155     EndIf

```

```

156      If nRet == 0
157          nRet := STBCncPIT(cGetProd)
158      EndIf
159
160  Else //Priorizo a busca por Numero do item na cesta
161
162      nRet := STBCncPIT(cGetProd)
163
164      If nRet == 0
165          cRet := STBCncPPrd(cGetProd)
166
167          If STBIsDigit(cRet)
168              nRet := Val(cRet)
169          Else
170              nRet := STBPegaIT(cRet)
171          EndIf
172      EndIf
173
174  EndIf
175
176  EndIf
177
178  If nRet == 0
179      STFMessage(ProcName(),"STOP",STR0008) //"Produto não encontrado na lista"
180      STFShowMessage(ProcName())
181  EndIf
182 EndIf
183 Return nRet
184
185 //-----
186 /*{Protheus.doc} STBIsDigit
187 Valida se uma cadeia de caracteres tem somente numeros
188 @param  cStr - Texto a ser avaliado
189 @return lRet - Retorna se a string eh somente numerica
190 @obs
191 @sample
192 */
193 //-----
194 Function STBIsDigit(cStr)
195
196 Local lRet := .T.    //Retorna se a string eh somente numerica
197 Local nY   := 0      //Contador
198
199 Default cStr := ""
200
201 If Empty(cStr)
202     lRet := .F.
203 Else
204     For nY := Len(cStr) To 1 Step -1
205         lRet := If( SubStr(cStr,nY,1) $ "0123456789" , lRet , .F. )
206     Next
207 EndIf

```

```

208
209 Return lRet
210
211 //-----
212 /*{Protheus.doc} STBCncPIt
213 Retorna o numero do item para cancelamento buscando por Item
214 @return lRet - Retorna o numero do item
215 @obs
216 @sample
217 */
218 //-----
219
220 Static Function STBCncPIt(cGetProd)
221 Local nItem      := 0    //Variavel que retorna o numero do item na cesta de produtos
222
223 If STBIsDigit(cGetProd)
224     nItem := Val(cGetProd)
225 Else
226     nItem := STBPegaIT(cGetProd)
227 EndIf
228
229 //Busca o item na MatxFis
230 If !STBTaxFoun("IT",nItem)
231     nItem := 0
232 EndIf
233
234 Return nItem
235
236 //-----
237 /*{Protheus.doc} STBCncPPrd
238 Retorna o numero do item para cancelamento buscando por produto
239 @return lRet - Retorna o numero do item
240 @obs
241 @sample
242 */
243 //-----
244
245 Static Function STBCncPPrd(cGetProd)
246 Local oModelCesta := STDGPBModel()    // Model da cesta
247 Local aInfoItem   := {}               // Retorna as informações sobre os itens
248 Local nX          := 0                // Variavel para For
249 Local cItem       := ''               // Variavel que retorna o numero do item na cesta de produtos
250
251 cGetProd := AllTrim(cGetProd)
252
253 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
254
255 /*/
256     Busca item na base de dados
257 /*/
258 aInfoItem := STWFindItem( cGetProd )
259

```

```
260 // Encontrou o item?
261 If aInfoItem[ITEM_ENCONTRADO]
262     nX := oModelCesta:Length()
263
264     While nX >= 1
265
266         oModelCesta:GoLine(nX)
267
268         If AllTrim(oModelCesta:GetValue("L2_PRODUTO")) == AllTrim(aInfoItem[ITEM_CODIGO]) .AND. !oModelCesta:IsDeleted()
269             cItem := oModelCesta:GetValue("L2_ITEM")
270             Exit
271         EndIf
272
273         nX--
274     EndDo
275
276 EndIf
277
278 Return cItem
```

Componentes/Item/STBFindItem.prw

141 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STBDigItem
6 Procedimento para digitacao de quantidade x item
7 Caso Trabalhe com seperador de digito:
8 Pega o Codigo recebido/digitado e separa em Codigo do Item
9 e quantidade do Item, conforme configuracao do parameto
10
11 @param   cCodeReceived      Codigo recebido/digitado
12 @return  cCodeReceived      Codigo recebido/digitado separado de acordo com parametro
13 @obs
14 @sample
15 */
16 //-----
17
18 //
19 Function STBDigItem(cCodeReceived)
20
21 Local cDigQtde      := Alltrim(SuperGetMV("MV_FRTDIGQ",, ""))      //Digito separador de quantidade x produto
22 Local nI            := 0                                           //Variavel auxiliar
23
24 Default          cCodeReceived      := ""                          // Codigo recebido/digitado
25
26 ParamType 0 Var cCodeReceived As Character Default ""
27
28 /*
29   Caso Trabalhe com seperador de digito:
30   Pega o Codigo recebido/digitado e separa em Codigo do Item
31   e quantidade do Item, conforme configuracao do parameto
32 */
33
34 If !Empty(cDigQtde)
35
36   nI := At(cDigQtde, cCodeReceived)
37   If nI > 0
38     nTmpQuant      := Val(PadL(cCodeReceived, nI - 1))
39     cCodeReceived  := SubStr( cCodeReceived, nI + 1, Len(cCodeReceived))
40   EndIf
41
42   /*
43     Se encontrou Qtde > 1 Altera Qtde
44   */
45   If nTmpQuant > 1
46     STBSetQuant( nTmpQuant )
47   EndIf
48
49 EndIf
50
51
52 Return cCodeReceived

```

```

53
54 //-----
55 /*{Protheus.doc} STIRecProdMFL
56 Prepara a gravacao dos Registros no MFL conforme os parametros recebidos
57 @param aProCode - Resultado de Produtos Consultados
58 @return Nil
59 @obs
60 @sample
61 */
62 //-----
63
64 Function STIRecProdMFL(aProCode)
65 Local lRet      := .F.
66 Local nFor      := 0
67 Local cStation  := STFGetStation("LG_PDV") // PDV atual
68 Local cCaixa    := STDNumCash()
69 Local aMFL      := {}
70
71 For nFor := 1 To Len(aProCode)
72
73     aMFL := {"MFL_FILIAL" , xFilial("MFL")      } , ;
74            {"MFL_PRODUT" , aProCode[nFor][01] } , ;
75            {"MFL_CAIXA"  , cCaixa                } , ;
76            {"MFL_PDV"    , cStation              } , ;
77            {"MFL_DATA"   , dDataBase             } , ;
78            {"MFL_HORA"   , Time()                 } , ;
79            {"MFL_SITUA"  , "00"                  } }
80
81     lRet := STFSaveTab("MFL", aMFL, .T., .T.)
82
83     If lRet
84         STFSLICreate(" ", "MFL", Str(MFL->(Recno()),17,0), "NOVO")
85     EndIf
86
87 Next nFor
88
89 Return Nil
90
91
92 /*{Protheus.doc} STBGetProd
93 Funcao responsavel pela busca de produtos no banco de dados da Retaguarda (Base TOP).
94
95 @param cFil      Filial a ser realizada a busca.
96 @param cDescProd Descrição do Produto a ser pesquisado.
97 @param nLimitRegs Quantidade Limite de registros que se deseja pesquisar.
98 */
99 //-----
100 Function STBGetProd(cFil, cDescProd, nLimitRegs)
101 Local uDados      := {}
102 Local aParam      := {}
103 Local uResult     := Nil
104 Local aFields     := {}

```

```

105 Local aTables          := {}
106 Local cWhere           := ""
107 Local cOrderBy         := ""
108
109 cDescProd := AllTrim(cDescProd)
110 cDescProd := Replace(cDescProd,"*","%")
111
112 //Monta os campos da query
113 aFields := {"B1_COD", "B1_DESC"}
114
115 //Tabela da Query
116 aTables := {"SB1"}
117
118 //Monta a Cláusula Where da query
119 cWhere := "B1_FILIAL = '"+cFil+"' AND "
120 cWhere += "B1_DESC LIKE '" + cDescProd + "' AND "
121 cWhere += "D_E_L_E_T_ = ' '"
122
123 //Monta a Cláusula Order By da query
124 cOrderBy := "B1_DESC"
125
126 // Busca de Clientes via STBRemoteExecute
127 aParam := { aFields,;
128             aTables,;
129             cWhere,;
130             cOrderBy,;
131             nLimitRegs;
132           }
133
134 If !STBRemoteExecute("STDQueryDB", aParam,,, @uResult)
135     // Tratamento do erro de conexao
136     uDados := .F.
137 Else
138     uDados := uResult
139 EndIf
140
141 Return uDados

```

Componentes/Item/STBItemCheck.prw

234 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "FWCSS.CH"
5 #INCLUDE "STPOS.CH"
6 #INCLUDE "STBITEMCHECK.CH"
7
8 //-----
9 /*{Protheus.doc} STBItLstCk
10 Função para montar itens para Dialog de conferencia
11 @param      aSL2
12 @return      aRet
13 */
14 //-----
15 Function STBItLstCk( aSL2 )
16 Local aRet      := {}
17 Local aStItemChk := {}
18 Local nX        := 0
19 Local nTamL2ITEM := TamSX3("L2_ITEM")[1]
20 Local cItem      := Replicate("0",TamSX3("L2_ITEM")[1]) //tratamento do sequencial do item
21 Local aItem      := {}
22 Local cDescItem  := ""
23 Local nPosQUANT  := AScan( aSL2[1] , { |x| x[1] == "L2_QUANT" } )
24 Local nPosPRODUTO := AScan( aSL2[1] , { |x| x[1] == "L2_PRODUTO" } )
25 Local nPosENTREGA := AScan( aSL2[1] , { |x| x[1] == "L2_ENTREGA" } )
26 Local lStItemChk := ExistBlock("STITEMCHK")
27
28 For nX := 1 to Len(aSL2)
29
30     cItem := Soma1(cItem,nTamL2ITEM)
31     aItem := STWFindItem( aSL2[nX][nPosPRODUTO][2] )
32
33     If !aItem[ITEM_ENCONTRADO]
34         LjGrvLog("Importa_Orcamento:STBItemCheck","Item Nao encontrado na base, chama rotina: STBUpdProducts")
35         STBUpdProducts( aSL2[nX][nPosPRODUTO][2] )
36         aItem := STWFindItem( aSL2[nX][nPosPRODUTO][2] ) //Chama novamente para atualizar descricao
37     EndIf
38
39     cDescItem := IIF(Empty(aItem[ITEM_DESCRICA0]),STR0001 + aSL2[nX][nPosPRODUTO][2],aItem[ITEM_DESCRICA0]) //para não parar o processo de venda, sinaliza qual código não
40     lCheck    := .F. //Código de Produto não encontrado na base do PDV:"
41
42     Do Case
43         Case aSL2[nX][nPosENTREGA][2] == "1"
44             cEntrega := STR0002 //Retira Posterior"
45         Case aSL2[nX][nPosENTREGA][2] == "2"
46             cEntrega := STR0003 //Retira"
47             lCheck    := .T.
48         Case aSL2[nX][nPosENTREGA][2] == "3"
49             cEntrega := STR0004 //Entrega"
50         Case aSL2[nX][nPosENTREGA][2] == "4"
51             cEntrega := STR0005 //Retira Posterior"

```

```

52      Otherwise
53          cEntrega := "ND"
54      EndCase
55
56      //PE para permitir alterar a obrigatoriedade de conferência do item (lCheck)
57      If lStItemChk
58          LjGrvLog("Conferencia de Item","Antes da Chamada do Ponto de Entrada:STITEMCHK",{lCheck, aSL2[nX]} )
59          aStItemChk := ExecBlock( "STITEMCHK",.F.,.F.,{lCheck, @aSL2[nX]} )
60          lCheck := aStItemChk[1]
61          cEntrega := aStItemChk[2]
62          LjGrvLog("Conferencia de Item","Apos a Chamada do Ponto de Entrada:STITEMCHK. Retorno:", lCheck )
63      EndIf
64      AAdd(aRet, {cItem, cDescItem, aSL2[nX][nPosQUANT][2],0,cEntrega,lCheck, AllTrim(aSL2[nX][nPosPRODUTO][2]), .T. } )
65  Next nX
66
67  Return aRet
68
69
70  //-----
71  /*{Protheus.doc} STBItPesq
72  Função para pesquisar o item na lista
73  @param      cGetCodProd, oGetCodProd, oListConf, oMsgErro
74  @return      lRet
75  */
76  //-----
77  Function STBItPesq( cGetCodProd, oGetCodProd, oListConf, oMsgErro, oBtnOk )
78
79  Local nX      := 0
80  Local lRet     := .T.
81  Local lPendIt  := .F.
82  Local aItem    := {}
83  Local nPosQuant := 0
84  Local nQuant   := 1
85  Local cCodItem := ""
86  Local aMVLJITOP := StrToKarr(SuperGetMv("MV_LJITOP", ,"*,-,%,?" ) , ",") //Parametro com os codigos dos caracteres de atalho de operacoes do item no registro do item
87
88  nPosQuant := AT(AllTrim(aMVLJITOP[1]),cGetCodProd) //utilizar regra de quantidade para permitir conferir quantidade de Item > 1
89  If nPosQuant > 0
90      nQuant := STIComma(SubStr(cGetCodProd,1,nPosQuant-1))
91  EndIf
92
93  cCodItem := SubStr(cGetCodProd,nPosQuant+1)
94  aItem := STWFindItem( cCodItem )
95  lRet := aItem[ITEM_ENCONTRADO]
96
97  If lRet
98      lRet := .F.
99
100  For nX:=1 to Len(oListConf:AARRAY)
101      If oListConf:AARRAY[nX][6] .AND. oListConf:AARRAY[nX][8] .AND. ((oListConf:AARRAY[nX][3] - oListConf:AARRAY[nX][4] ) >= nQuant) .AND. oListConf:AARRAY[nX][7] ==
102          AllTrim(aItem[ITEM_CODIGO])
103          oListConf:AARRAY[nX][4] := oListConf:AARRAY[nX][4] + nQuant

```

```

103         oListConf:Refresh()
104         lRet := .T.
105         Exit
106     EndIf
107 Next nX
108
109 If lRet
110     cGetCodProd := Space(TamSX3("L1_PRODUTO"))[1])
111 EndIf
112
113 oMsgErro:lVisible := .F.
114 oMsgErro:Refresh()
115
116 For nX:=1 to Len(oListConf:AARRAY)
117     If oListConf:AARRAY[nX][6] .AND. oListConf:AARRAY[nX][8] .AND. ((oListConf:AARRAY[nX][3] - oListConf:AARRAY[nX][4] ) > 0)
118         lPendIt := .T.
119         Exit
120     EndIf
121 Next nX
122
123 If lPendIt
124     oGetCodProd:SetFocus()
125 Else
126     oBtnOk:SetFocus()
127 EndIf
128 EndIf
129
130 If !lRet
131     oMsgErro:cCaption := STR0006    //"Produto/quantidade não encontrado!"
132     oMsgErro:lVisible := .T.
133     oMsgErro:Refresh()
134
135     STFMessage(ProcName(),"ALERT",oMsgErro:cCaption )
136     STFShowMessage(ProcName())
137 EndIf
138
139 Return lRet
140
141
142 //-----
143 /*{Protheus.doc} STBitCkOk
144 Função para pesquisar o item na lista
145 @param      oListConf, oMsgErro, aSL2
146 @return     lRet
147 */
148 //-----
149 Function STBitCkOk( oListConf, oMsgErro, aSL2, lZeraPay )
150 Local lRet      := .T.
151 Local cItPend   := ""
152 Local nQtdDel   := 0
153 Local nX
154 Local aSL2BKP   := {}    //Backup da Tabela aSL2

```

```

155
156 Default aSL2      := {}
157 Default lZeraPay   := .F.
158
159 If !Empty(aSL2)
160
161     aSL2BKP := aClone(aSL2)
162
163     For nX:=1 to Len(oListConf:AARRAY)
164         If oListConf:AARRAY[nX][6] .AND. oListConf:AARRAY[nX][8] .AND. ((oListConf:AARRAY[nX][3] - oListConf:AARRAY[nX][4] ) > 0)
165             lRet      := .F.
166             cItPend    := oListConf:AARRAY[nX][1]
167             Exit
168         EndIf
169     Next nX
170
171     If lRet
172         //Atualiza aSL2 removendo itens excluídos
173         For nX:=1 to Len(oListConf:AARRAY)
174             If !oListConf:AARRAY[nX][8]
175                 ADel(aSL2,nX-nQtdDel)
176                 nQtdDel += 1
177             EndIf
178         Next nX
179
180         If nQtdDel > 0
181             ASize(aSL2,len(aSL2)-nQtdDel)
182         EndIf
183     Else
184         oMsgErro:cCaption := STR0007 + cItPend      //"Existe item pendente para conferência: "
185         oMsgErro:lVisible := .T.
186         oMsgErro:Refresh()
187
188         STFMessage(ProcName(),oMsgErro:cCaption )
189         STFShowMessage(ProcName())
190     EndIf
191     If nQtdDel > 0
192         STFMessage("STBItCkOk","YESNO",STR0012) //"Itens do orçamento alterados, por esse motivo a negociação de Formas de Pagamento e Descontos no total serão desfeitas,
deseja continuar?" //"Itens do orçamento alterado, por esse motivo a negociação será desfeita,  deseja continuar?"
193         If STFShowMessage("STBItCkOk")
194             STISetZrPg(.T.)
195             lZeraPay := .T.
196         Else
197             aSL2 := aSL2BKP
198             lRet := .F.
199         EndIf
200     EndIf
201 Else
202     lRet := .F.
203 EndIf
204
205 Return lRet

```

```

206
207 //-----
208 /*{Protheus.doc} STBitCkDel
209 Função para pesquisar o item na lista
210 @param      oListConf, oMsgErro
211 @return      lRet
212 */
213 //-----
214 Function STBitCkDel( oListConf, oMsgErro )
215 Local nItem := oListConf:nAt
216 Local lRet := .T.
217
218 If !oListConf:AARRAY[nItem][6]
219     oMsgErro:cCaption := STR0008 + oListConf:AARRAY[nItem][1] + STR0009    //"Item:"    //" não possui controle de conferência."
220     oMsgErro:lVisible := .T.
221     oMsgErro:Refresh()
222
223     STFMessage(ProcName(),"ALERT",oMsgErro:cCaption )
224     STFS showMessage(ProcName())
225     lRet := .F.
226 EndIf
227
228 If lRet .AND. oListConf:AARRAY[nItem][8] .AND. ApMsgYesNo(STR0010 + oListConf:AARRAY[nItem][1] + " - " + AllTrim(oListConf:AARRAY[nItem][2]))    //"Confirmar cancelar o Item:"
229     oListConf:AARRAY[nItem][8] := .F.
230 ElseIf lRet .AND. !oListConf:AARRAY[nItem][8] .AND. ApMsgYesNo(STR0011 + oListConf:AARRAY[nItem][1] + " - " + AllTrim(oListConf:AARRAY[nItem][2]))    //"Deseja remover o status
    de cancelado do Item:"
231     oListConf:AARRAY[nItem][8] := .T.
232 EndIf
233
234 Return lRet

```

Componentes/Item/STBItemRegistry.prw

984 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBITEMREGISTRY.CH"
5
6 Static lSaleActive := .F.
7 Static lCpoImpEnt := SL2->(ColumnPos("L2_TOTFED")) > 0 .AND. ;
8                     SL2->(ColumnPos("L2_TOTEST")) > 0 .AND. ;
9                     SL2->(ColumnPos("L2_TOTMUN")) > 0 // Verifica a existencia do campo de Total de Imposto NCM/NBS
10
11
12 // Situacoes do _SITUA - SL1 ou SL2
13 //-----
14 /*
15 " " - Base Errada, Registro Ignorado.
16 "00" - Venda Efetuada com Sucesso
17 "01" - Abertura do Cupom Nao Impressa
18 "02" - Impresso a Abertura do Cupom
19 "03" - Item Nao Impresso
20 "04" - Impresso o Item
21 "05" - Solicitado o Cancelamento do Item
22 "06" - Item Cancelado
23 "07" - Solicitado o Cancelamento do Cupom
24 "08" - Cupom Cancelado
25 "09" - Encerrado SL1 (Nao gerado SL4)
26 "10" - Encerrado a Venda
27 Pode nao ter sido impresso o cupom
28 "TX" - Foi Enviado ao Server
29 "RX" - Foi Recebido Pelo Server
30 "OK" - Foi Processado no Server
31 */
32 //Enviar um OK ao Client que foi Processado
33 //-----
34
35
36 //-----
37 /*{Protheus.doc} STBSaveSaleBasket
38 Salva Cabecalho da venda na cesta de vendas
39 @param
40 @return lRet - Retorna se Salvou Item
41 @obs
42 @sample
43 */
44 //-----
45 Function STBSaveSaleBasket()
46 Local lRet := .T. // Retorno
47 Local lUsaFecha := SuperGetMV("MV_LJCONFF",.T.,.F.) // Utilizar conf. de fechamento
48 Local cTransNat := SuperGetMV("MV_LJTRNAT",.F.,"") // Natureza financeira da transferencia
49 Local lUsaTransf := SuperGetMV("MV_LJTRANS",.T.,.F.) .AND. !Empty(cTransNat) .AND. lUsaFecha // Utiliza Transferencia de caixa
50 Local cStation := STFGetStation("CODIGO") // Estacao atual
51 Local lEmitNfce := LjEmitNFce() // Sinaliza se utiliza NFC-e
52 Local cNumSerieSAT := STFGetStat("SERSAT") // Retorna a série do equipamento

```

```

53 Local lUseSat      := STFGetCfg("lUseSAT",.F.) //Usa SAT?
54 Local aPrgInfo     := {}
55 Local lNovoNum      := .F.
56 Local aCartCRD      := IIF(ExistFunc('STDFindCart'),STDFindCart(),{}) //Carrega informacoes do cartão utilizado no SIGACRD
57 Local oCliModel     := STDGCLiModel()          // Model com as informacoes do cliente
58
59 /*/
60   OBS:
61   "L1_CLIENTE" , "L1_LOJA" , "L1_VEND" sao carregados na abertura do sistema e
62   Alterados na funcao de selecao de clientes nao precisam ser atualizados aqui
63 /*/
64
65 //Starta o Tempo de atendimento
66 If SL1->(ColumnPos("L1_TIMEATE")) > 0
67   STDSPBasket( "SL1" , "L1_TIMEATE"          , Seconds()          )
68 EndIf
69
70 STDSPBasket( "SL1" , "L1_EMISSAO"           , dDataBase              )
71 STDSPBasket( "SL1" , "L1_DTLIM"             , dDataBase              )
72 STDSPBasket( "SL1" , "L1_EMISNF"           , dDataBase              )
73 STDSPBasket( "SL1" , "L1_ESTACAO"           , cStation               )
74 STDSPBasket( "SL1" , "L1_SITUA"            , "01"                   ) // "01" - Abertura do Cupom Nao Impressa
75 STDSPBasket( "SL1" , "L1_TRCXGER"          , lUsaTransf             )
76 STDSPBasket( "SL1" , "L1_TREFETI"          , .F.                     )
77
78 If lEmitNfce
79   STDSPBasket( "SL1" , "L1_TPORC"           , "E"                     )
80 EndIf
81
82 If STFGetCfg("lUseSAT",.F.) .AND. SL1->(ColumnPos("L1_SERSAT")) > 0
83   STDSPBasket( "SL1" , "L1_SERSAT"          , cNumSerieSAT            )
84 EndIf
85
86 /* Segue o padrao do FrontLoja, na L1_TABELA grava o valor que esta no parametro MV_TABPAD
87   Ja na L2_TABELA nao grava, pois no FrontLoja gravava antes o valor da BI_TABELA, onde ambos os
88   campos sao do tamanho 1 */
89 If !Empty(oCliModel:GetValue("SAIMASTER","A1_TABELA"))
90   STDSPBasket( "SL1" , "L1_TABELA"          , oCliModel:GetValue("SAIMASTER","A1_TABELA") )
91 Else
92   STDSPBasket( "SL1" , "L1_TABELA"          , AllTrim(SuperGetMv("MV_TABPAD")) )
93 EndIf
94
95 If STFGetCfg("lMultCoin") // Se usa Multimoeda
96   STDSPBasket( "SL1" , "L1_MOEDA"           , STBGetCurrency()        )
97   STDSPBasket( "SL1" , "L1_TXMOEDA"         , 1                        )
98 EndIf
99
100 STDSPBasket( "SL1" , "L1_NUMMOV"            , AllTrim(STDNumMov()) )
101
102 STDSPBasket( "SL1" , "L1_GCCART"           , iif(Len(aCartCrd)>=1,aCartCRD[1],"") ) //Cartão do SIGACRD
103
104 If !STFGetCfg("lUseECF")

```

```

105
106     If !lUseSat
107         aPrgInfo := GetAPOInfo("STBPAYMENT.PRW")
108         lNovoNum := aPrgInfo[4] >= Ctod("16/01/2018")
109     EndIf
110
111     If lEmitNfce .AND. (lUseSat .OR. !lNovoNum)
112         STDGPBasket( "SL1" , "L1_D0C" , STDGPBasket( "SL1" , "L1_NUM" ) )
113     Else
114         STDGPBasket( "SL1" , "L1_D0C" , "NFCe" )
115     EndIf
116 EndIf
117 // Informar ao server o Status Atual do Check-Out.
118 // Add funcao de totais na venda
119 STFSLICreate( AllTrim(cStation) , "CON" , "V|" + Alltrim(STDGPBasket("SL1","L1_PDV")) +
120 ;
121     "|" + AllTrim(STDGPBasket("SL1","L1_D0C")) + "|" + AllTrim( STR(STDGPBasket( "SL1" , "L1_VLRTOT" ) ) ) + ;
122     "|" + AllTrim(Str(0)) , "SOBREPOE" ) // Total de Itens da Venda
123
124 Return lRet
125
126
127 //-----
128 /*{Protheus.doc} STBSaveItBasket
129 Salva Item na cesta de vendas
130
131 @param aInfoItem      Array contendo as informacoes do Produto
132 @param nItemLine      Linha do Item
133 @return lRet          Retorna se atualizou cesta
134 @obs
135 @sample
136 */
137 //-----
138 Function STBSaveItBasket( aInfoItem , nItemLine, cIdItRel )
139
140 Local lRet          := .T.                // Retorno da funcao
141 Local aSaveLines    := FWSaveRows()      // aSaveLines
142 Local aMostruario    := {"N",""}        // Array de Mostruario
143 Local lCAT83        := SF4->(ColumnPos("F4_CODLAN")) > 0 .AND. SL2->(ColumnPos("L2_CODLAN")) > 0 .AND. ExistFunc("FISA023") .AND. SuperGetMV("MV_CAT8309",,.F.) // Verifica
CAT/83
144 Local lLjcFid       := SuperGetMv("MV_LJCFID",,.F.) .AND. CrdxInt() // Indica se a recarga de cartao fidelidade esta ativa
145 Local cItemSitTrib   := ""                // Situacao tributaria do item
146 Local lEmitNfce     := LjEmitNFCe() // Sinaliza se utiliza NFC-e
147 Local lLjNfPafEcf   := STBIsPaf()
148 Local lL2Modbc       := SL2->(ColumnPos( "L2_MODBC" )) > 0 //Verifica se possui campo utilizado para otimizar processo da NFCe
149 Local lL2CEST        := IIF((lEmitNfce .Or. lLjNfPafEcf) .AND. SL2->(ColumnPos( "L2_CEST" )) > 0, .T.,.F.)
150 Local cModBC         := ""
151 Local aCodVps        := IIF(ExistFunc("STBGetCodVP"), STBGetCodVP(), {} ) //array codigo dos vale presentes
152 Local nCount         := 0                //contador
153 Local aModBC         := {}
154 Local lSaveOrc       := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento

```

```

155 Local lL2FECP      := SL2->(ColumnPos("L2_ALQFECP") > 0 .AND. ColumnPos("L2_VALFECP") > 0)
156 Local lL2FECPST    := SL2->(ColumnPos("L2_ALQFCST") > 0 .AND. ColumnPos("L2_VFECPST") > 0)
157 Local lDate        := .T.
158 Local oModelCesta  := STDGPBModel()           //Model da cesta de produtos
159 Local cLF_MOTICMS  := ""
160 Local lUseSat       := STFGetCfg("lUseSAT",.F.) //Usa SAT?
161 Local lExistGetTriSat := lUseSat .AND. (ExistFunc("LjGetTriSat") .AND. ExistFunc("LjSetTriSat")) //Armazenamento para XML SAT
162 Local aSATtrib     := {}                      //Tributação produto SAT
163
164 Default aInfoItem   := {}                    // Array do Item
165 Default nItemLine   := 0                    // Linha do Item na Venda
166 Default cIdItRel    := ""                   // ID do Item relacionado
167
168 // Salva Item na cesta de vendas
169 If (DATE() <> dDataBase)
170     lDate := .F.
171 Endif
172
173 If nItemLine > 0 .AND. lDate
174
175     If nItemLine > 1
176         /*
177         Para o primeiro item nao realizar addline, pois o grid já traz linha em branco
178         */
179         STDPBAddLine("SL2")
180     EndIf
181
182     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_FILIAL" , xFilial("SL2") , nItemLine ) ,)
183     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_NUM" , STDGPBasket("SL1","L1_NUM") , nItemLine ) ,)
184
185     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_DOC" , AllTrim(STDGPBasket("SL1","L1_DOC")) , nItemLine ) ,)
186     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_SERIE" , AllTrim(STDGPBasket("SL1","L1_SERIE")) , nItemLine ) ,)
187     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_PDV" , STDGPBasket("SL1","L1_PDV") , nItemLine ) ,)
188     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_EMISSAO" , STDGPBasket("SL1","L1_EMISSAO") , nItemLine ) ,)
189     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VEND" , STDGPBasket("SL1","L1_VEND") , nItemLine ) ,)
190
191     If nItemLine <= 99
192         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_ITEM" , StrZero(nItemLine,TamSx3("L2_ITEM")[1]), nItemLine ) ,)
193     Else
194         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_ITEM" , STBPegaIT(nItemLine), nItemLine ) ,)
195     EndIf
196
197     If lRet .AND. SL2->(ColumnPos("L2_IDITREL")) > 0
198         lRet := STDSPBasket( "SL2" , "L2_IDITREL" , cIdItRel, nItemLine )
199     Endif
200
201     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_PRODUTO" , aInfoItem[ITEM_CODIGO] , nItemLine ) ,)
202
203     If lLjNfPafEcF .And. !Empty(Alltrim(STBTaxRet(nItemLine,"IT_CEST"))) .And. !Empty(AllTrim(aInfoItem[ITEM_POSIPI]))
204         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_DESCRI",;
205             "#" + AllTrim(STBTaxRet(nItemLine,"IT_CEST")) + "#" + AllTrim(aInfoItem[ITEM_POSIPI]) + "#" + AllTrim(aInfoItem[ITEM_DESCRICA0]), nItemLine ) ,)
206     Else

```

```

207     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_DESCRI" , aInfoItem[ITEM_DESCRICA0] , nItemLine ) ,)
208 EndIf
209
210 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_UM" , aInfoItem[ITEM_UNID_MEDIDA] , nItemLine ) ,)
211 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_LOCAL" , aInfoItem[ITEM_ARMAZEMPAD] , nItemLine ) ,)
212 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_CODBAR" , aInfoItem[ITEM_CODBAR] , nItemLine ) ,)
213 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_ORIGEM" , aInfoItem[ITEM_ORIGEM] , nItemLine ) ,)
214 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_CODISS" , aInfoItem[ITEM_CODISS] , nItemLine ) ,)
215 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_POSIPI" , aInfoItem[ITEM_POSIPI] , nItemLine ) ,)
216
217 If lUseSat
218     STBTaxAlt("IT_VALICM", LjArredSat(STBTaxRet(nItemLine,"IT_ALIQICM")/100 * STBTaxRet(nItemLine,"IT_BASEICM"),TamSX3("L2_VALICM")[2]),nItemLine)
219     If !Empty(STBTaxRet(nItemLine,"IT_ALIQPS2"))
220         STBTaxAlt("IT_VALPS2", LjArredSat(STBTaxRet(nItemLine,"IT_ALIQPS2")/100 * STBTaxRet(nItemLine,"IT_BASEPS2"),TamSX3("L2_VALPS2")[2]),nItemLine)
221     Endif
222     If !Empty(STBTaxRet(nItemLine,"IT_ALIQCF2"))
223         STBTaxAlt("IT_VALCF2", LjArredSat(STBTaxRet(nItemLine,"IT_ALIQCF2")/100 * STBTaxRet(nItemLine,"IT_BASECF2"),TamSX3("L2_VALCF2")[2]),nItemLine)
224     Endif
225 Endif
226
227 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VALICM" , STBTaxRet(nItemLine,"IT_VALICM") , nItemLine ) ,)
228 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_BASEICM" , STBTaxRet(nItemLine,"IT_BASEICM") , nItemLine ) ,)
229
230 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
231 If ExistFunc("STBGetTblPr") .AND. !Empty(STBGetTblPr()) .AND. oModelCesta:HasField("L2_CODTAB")
232     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_CODTAB" , STBGetTblPr() , nItemLine ) ,)
233 EndIf
234
235 If Len(aCodVps) > 0 .AND. IIF(ExistFunc("STDIsvP"),STDIsvP(aInfoItem[ITEM_CODIGO]),.F.)
236     FOR nCount := 1 To Len(aCodVps)
237         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VALEPRE", aCodVps[nCount] , nItemLine ) ,) //Codigo do vale presente
238     NEXT nCount
239 EndIf
240
241 If lExistGetTriSat
242     //TRIBUTACAO SAT
243     aSATtrib := LjGetTriSat()
244
245     STBIItSatTr(@aSATtrib, nItemLine, "IT_BASEPS2")
246     STBIItSatTr(@aSATtrib, nItemLine, "IT_ALIQPS2")
247     STBIItSatTr(@aSATtrib, nItemLine, "IT_BASECF2")
248     STBIItSatTr(@aSATtrib, nItemLine, "IT_ALIQCF2")
249     STBIItSatTr(@aSATtrib, nItemLine, "IT_PAUTPIS")
250     STBIItSatTr(@aSATtrib, nItemLine, "IT_PAUTCOF")
251     STBIItSatTr(@aSATtrib, nItemLine, "IT_CEST")
252
253     //Grava os impostos no Array de tributacoes do SAT
254     LjSetTriSat(aSATtrib)
255 EndIf
256
257 //Regra utilizada no MatxFis para verificar a modalidade da base de calculo
258 If lEmitNfce

```

```

259     aModBc := STBTaxRet(nItemLine,"IT_SPED")
260     If lL2Modbc
261         If Len(aModBc) > 0
262             cModBC := aModBc[1][6]
263         EndIf
264
265         IIf(lRet , lRet := STDSPBasket( "SL2", "L2_MODBC", cModBC, nItemLine ) , )
266     EndIf
267
268     If SL2->(ColumnPos( "L2_DESCICM" )) > 0 .And. SL2->(ColumnPos( "L2_MOTDICM" )) > 0
269         /*
270         -- Validação conforme a MatxFis --
271
272         Conforme o link https://centraldeatendimento.totvs.com/hc/pt-br/articles/360025687693
273         -MP-NFE-934-Rejeição-Não-informado-valor-do-ICMS-desonerado-ou-o-Motivo-de-desoneração
274
275         o campo D2_DESCICM será gravado dependendo da forma
276         que a TES é configurada e na NF-e o valor da TAG vICMSDeson é enviada
277         pois a tabela CD2 é gravada nesse momento
278
279         Porem para a NFC-e, envio somente usamos o campo L2_DESCICM com isso
280         o campo deve ser gravado a partir somente de qualquer configuração de TES
281
282         Por isso o campo D2_DESCICM não teve sua gravação alterada no LOJXFUNC
283         */
284         cLF_MOTICMS := Alltrim(STBTaxRet(nItemLine,"LF_MOTICMS"))
285         If !Empty(cLF_MOTICMS) .And. (Len(aModBC) > 0)
286             IIf( lRet, lRet := STDSPBasket( "SL2", "L2_DESCICM", aModBc[1][26] , nItemLine ) , )
287             IIf( lRet, lRet := STDSPBasket( "SL2", "L2_MOTDICM", cLF_MOTICMS , nItemLine ) , )
288         EndIf
289     EndIf
290 EndIf
291
292 If lRet .AND. lL2CEST
293     lRet := STDSPBasket( "SL2", "L2_CEST", AllTrim(STBTaxRet(nItemLine,"IT_CEST")), nItemLine )
294 EndIf
295
296 If lL2FECP
297     IIf( lRet, lRet := STDSPBasket( "SL2", "L2_VALFECP", STBTaxRet(nItemLine,"IT_VALFECP") , nItemLine ) , )
298     IIf( lRet, lRet := STDSPBasket( "SL2", "L2_ALQFECP", STBTaxRet(nItemLine,"IT_ALIQFECP") , nItemLine ) , )
299 EndIf
300
301 If lL2FECST
302     IIf( lRet, lRet := STDSPBasket( "SL2", "L2_VFECST", STBTaxRet(nItemLine,"IT_VFECST") , nItemLine ) , )
303     IIf( lRet, lRet := STDSPBasket( "SL2", "L2_ALQFCST", STBTaxRet(nItemLine,"IT_ALFCST") , nItemLine ) , )
304 EndIf
305
306 If lRet .And. lLjNfPafEcf
307     If SL2->(ColumnPos("L2_IAT")) > 0
308         lRet := STDSPBasket( "SL2" , "L2_IAT", IIf( STBRuleArred() == "R", "A", "T" ) , nItemLine)
309     EndIf
310

```

```

311     If lRet .And. SL2->(ColumnPos("L2_IPPT")) > 0
312         lRet := STDSPBasket( "SL2" , "L2_IPPT", AllTrim(aInfoItem[ITEM_IPPT]) , nItemLine)
313     EndIf
314
315     If lRet .And. SL2->(ColumnPos("L2_DECQTD")) > 0
316         lRet := STDSPBasket( "SL2" , "L2_DECQTD", TamSX3("L2_QUANT")[2] , nItemLine)
317     EndIf
318
319     If lRet .And. SL2->(ColumnPos("L2_DECVLU")) > 0
320         lRet := STDSPBasket( "SL2" , "L2_DECVLU", TamSX3("L2_VRUNIT")[2] , nItemLine)
321     EndIf
322
323     lRet := lRet .And. STDSPBasket( "SL2" , "L2_CONTD0C", STDGPBasket( "SL1" , "L1_CONTD0C" ) , nItemLine)
324 EndIf
325
326 //Serie do PDV
327 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_SERPDV" , STFGetStat("SERPDV"), nItemLine) ,)
328
329 /*/
330     Atualiza valores MATXFIS
331 /*/
332 If lRet
333     STBRefshItBasket( nItemLine, aInfoItem)
334 EndIf
335
336 If lRet .AND. lCAT83
337     // grava o campo da tabela CDZ gravado na TES q serve para apuracao do Credito pelo FISCO
338     lRet := STDSPBasket( "SL2" , "L2_CODLAN" , STBGetTESInfo( "F4_CODLAN" , AllTrim(STBTaxRet(nItemLine,"IT_TES"))) ) , nItemLine )
339 EndIf
340
341 // Ajusta situacao tributaria para gravacao
342 If lRet
343     cItemSitTrib := STBTaxSit(nItemLine)
344     lRet := STDSPBasket( "SL2" , "L2_SITTRIB" , STBStAdjust( cItemSitTrib ) , nItemLine )
345 EndIf
346
347 /*/
348     Mostruário Saldão
349 /*/
350 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VDMOST" , aMostruario[1] , nItemLine ) ,)
351 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VD0BS" , aMostruario[2] , nItemLine ) ,)
352 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_ITEMSD1" , "000000" , nItemLine ) ,)
353 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VENDIDO" , "S" , nItemLine ) ,)
354 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_GRADE" , "N" , nItemLine ) ,)
355
356 //Recarga do cartao fidelidade processada: //TOD0: precisa ser Imprementado : Leandro
357 If lLjcFid
358     If FA271aGrcf()
359         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_PRODUTO" , "cCodProd" , nItemLine ) ,)
360         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_NUMCFID" , LaFunhGet(1) , nItemLine ) ,)
361         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_DTSDFID" , LaFunhGet(2) , nItemLine ) ,)
362         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_DTSDFID" , LaFunhGet(3) , nItemLine ) ,)

```

```

363
364     If Ca280CkWs ()
365         //W - Via WebService(LJCCARFID)
366         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_PROCFID" , "W" , nItemLine ) ,)
367         Fa271aSpfw (.T.)
368     Else
369         //B - Via processo batch (LJGRVBATCH)
370         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_PROCFID" , "B" , nItemLine ) ,)
371         Fa271aSpfw (.F.)
372     EndIf
373 EndIf
374 EndIf
375
376 IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_SITUA" , "04" , nItemLine ) ,) // "04" - Impresso o Item
377
378 // Atualizar dados no cabeçalho - SL1
379 IIf(lRet , lRet := STDSPBasket( "SL1" , "L1_SITUA" , "04" ) ,) // "04" - Impresso o Item
380
381 // Salva venda como orcamento
382 If lSaveOrc
383
384     If !SuperGetMv("MV_LJCNVDA",, .F.)
385         IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_TABELA" , Padr( AllTrim(SuperGetMv("MV_TABPAD")) , SL2->(TamSx3("L2_TABELA")[1])) , nItemLine ) ,)
386     EndIf
387
388     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_ENTREGA" , "2" , nItemLine ) ,)
389     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_VENDIDO" , "" , nItemLine ) ,)
390     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_DOC" , "" , nItemLine ) ,)
391     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_SERIE" , "" , nItemLine ) ,)
392     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_PDV" , "" , nItemLine ) ,)
393     IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_SITUA" , "" , nItemLine ) ,)
394 EndIf
395
396 Else
397     If !lDate
398         STFMessage("ItemRegistered","STOP",STR0002)//"A data do sistema esta diferente da data do sistema operacional. Atenção! Favor efetuar o logoff do sistema"
399     Endif
400     lRet := .F.
401 EndIf
402
403 //-----
404 // Restaura Rows, Atualiza View
405 //-----
406 FWRestRows( aSaveLines )
407
408 Return lRet
409
410
411 //-----
412 /*{Protheus.doc} STBSearchPrice
413 Busca o preco do produto para exibi-lo na tela.
414

```

```

415 @param cItemCode    , caractere, Código do Item
416 @param cCliCode     , caractere, Código do Cliente
417 @param cCliLoja     , caractere, Código da Loja
418
419 @return nPrice , numérico, Preço do item
420 @obs
421 @sample
422 /*/
423 //-----
424 Function STBSearchPrice(cItemCode , cCliCode, cCliLoja )
425
426 Local aFindItem      := {}           // Array da busca de itens
427 Local nPrice         := 0           // PReco
428
429 Default cItemCode    := ""
430 Default cCliCode     := ""
431 Default cCliLoja     := ""
432
433
434 aFindItem := STWFindItem( cItemCode )
435
436 // Encontrou o item?
437 If aFindItem[ITEM_ENCONTRADO]
438
439     /*/
440     2 - Busca preco do item caso nao tenha sido informada por parametro
441     /*/
442     If nPrice <= 0
443
444         nPrice := STWFormPr( aFindItem[ITEM_CODIGO], cCliCode, /*filial*/, cCliLoja,/*nMoeda*/, STBGetQuant() )
445
446         // Se a tabela de preço utilizada esta fora de vigência
447         If nPrice == -999 // Verifica se tabela de preço esta dentro da vigencia
448             LjGrvLog(aFindItem[ITEM_CODIGO],"STBSearchPrice | Item não poderá ser registrado, motivo: Tabela de preço fora de vigência.")
449             lRet := .F.
450             STFMessage("ItemRegistered","STOP", STR0003 + CHR(13)+CHR(10) + STR0004) //"Tabela de preço fora de vigência." "Verifique o código da tabela contido no parâmetro
MV_TABPAD"
451
452         ElseIf nPrice <= 0 //Se nao achou preco
453             lRet := .F.
454             STFMessage("ItemRegistered","STOP",STR0001) //"Preço não encontrado"
455             STFS showMessage("ItemRegistered")
456         EndIf
457     EndIf
458 EndIf
459 EndIf
460 EndIf
461
462 Return nPrice
463
464
465 //-----

```

```

466 /*/{Protheus.doc} STBSearchPrice
467 Busca o preco do produto para exibi-lo na tela.
468
469 @param  nItemLine      Linha do Item
470 @return  lRet - Retorna se encontrou o preco
471 @obs
472 @sample
473 */
474 //-----
475 Function STBRefshItBasket( nItemLine, aInfoItem )
476 Local lRet           := .T.      // Retorno função
477 Local aTesImpInf      := {}       // Array de Aliquotas de Imposto por TES
478 Local cIndImp         := 0        // Posição do imposto variável
479 Local nX              := 0        // Contador de for
480 Local aAlqLeiTr       := {0,0,0,0} //array utilizado para pegar as aliquotas das lei dos impostos
481 Local cMvFisCTrb      := SuperGetMV("MV_FISCTRB",,"1") // Informar metodo de consulta do percentual de carga tributaria.Informe 1 para tratamento legado ou 2
    aliquotas da Nota Fiscal e CGA/CGB.
482 Local cMvFisAlCT      := SuperGetMV("MV_FISALCT",,"3") // Informar qual o metodo de composicao da carga tributaria quando MV_FISCTRB = 2.
483 Local nIt_Total       := 0        // valor total somado ao acrescimo, já que a MATXFIS nao retorna o valor do acrescimo somado ao IT_TOTAL
484 Local nFrtSegDes      := 0        // 0 valor do Frete Seguro e Despesa tem que ser subtraido do valor do item
485 Local nItAcresci      := 0        // Valor do acrescimo financeiro para a condição de pagamento
486 Local nItDedICMS      := 0        // Valor da desoneração do icms
487 Local lUseSat         := STFGGetCfg("lUseSAT",.F.) //Usa SAT?
488 Local lExistGetTriSat := lUseSat .AND. (ExistFunc("LjGetTriSat") .AND. ExistFunc("LjSetTriSat")) //Armazenamento para XML SAT
489 Local aSATtrib        := {}       //Tributação produto SAT
490 Local lCfgTrib        as logical //Verifica se Configurador de Tributos esta habilitado
491 Local jCfgTaxes       as json //Configurador de Tributos
492
493 Default nItemLine := 0
494 Default aInfoItem := {}
495
496 lCfgTrib := If(FindFunction("LjCfgTrib"), LjCfgTrib(), .F.)
497 jCfgTaxes := JsonObject():New()
498
499 If nItemLine <= STDPBLength("SL2")
500     lRet := .T.
501 EndIf
502
503 If lRet
504     If lCfgTrib
505         jCfgTaxes := STBGetTax(nItemLine)
506     EndIf
507
508     If SL2->(ColumnPos( "L2_DESCICM" )) > 0
509         nItDedICMS := STBTaxRet(nItemLine,"IT_DEDICM")
510     EndIf
511     nItAcresci := STBTaxRet(nItemLine, "IT_ACRESCI")
512     nFrtSegDes := STBTaxRet(nItemLine,"IT_FRETE") + STBTaxRet(nItemLine,"IT_SEGURO") + STBTaxRet(nItemLine,"IT_DESPESA")
513     nIt_Total := STBTaxRet(nItemLine, "IT_TOTAL") - nFrtSegDes + nItDedICMS
514
515     If Len(jCfgTaxes) > 0 .And. jCfgTaxes[1]:getJsonObject("ident_trib") == "000022" //Configurador de Tributos - Id IPI = 000022
516         If STBVerTotTax(jCfgTaxes[1]:getJsonObject("ident_trib"), 1)

```

```

517         nIt_Total -= STBTaxRet(nItemLine,"IT_VALIPI")
518     EndIf
519 Else //Legado TES
520     nIt_Total -= STBTaxRet(nItemLine,"IT_VALIPI")
521 EndIf
522
523 // Quando ICM solidário NÃO agrega valor no total, não deve subtrair
524 If Len(jCfgTaxes) > 0 .And. jCfgTaxes[1]:getJsonObject("ident_trib") == "000056" //Configurador de Tributos - Id ICMS-ST = 000056
525     If STBVerTotTax(jCfgTaxes[1]:getJsonObject("ident_trib"), 1)
526         nIt_Total -= STBTaxRet(nItemLine, "IT_VALSOL") // Subtrai valor do ICMS ST
527     EndIf
528 ElseIf AllTrim(STBGetTESInfo( "F4_INCSOL" , AllTrim(STBTaxRet(nItemLine,"IT_TES")))) <> "N" //Legado TES
529     nIt_Total -= STBTaxRet(nItemLine, "IT_VALSOL") // Subtrai valor do ICMS ST
530 EndIf
531
532 STDSPBasket( "SL2" , "L2_QUANT" , STBTaxRet(nItemLine,"IT_QUANT" ) , nItemLine )
533
534 If Empty(STBTaxRet(nItemLine,"IT_DESCONTO")) .AND. Empty(nItAcresci) .AND. Empty(STBTaxRet(nItemLine,"IT_DESCOT"))
535     STDSPBasket( "SL2" , "L2_VRUNIT" , STBTaxRet(nItemLine,"IT_PRCUNI"), nItemLine)
536 Else
537     STDSPBasket( "SL2" , "L2_VRUNIT" , nIt_Total / STBTaxRet(nItemLine,"IT_QUANT"), nItemLine)
538 EndIf
539
540 STDSPBasket( "SL2" , "L2_VLRITEM" , nIt_Total , nItemLine )
541
542 STDSPBasket( "SL2" , "L2_DESCPRO" , STBTaxRet(nItemLine,"IT_DESCOT" ) , nItemLine )
543 STDSPBasket( "SL2" , "L2_VALDESC" , STBTaxRet(nItemLine,"IT_DESCONTO" ) , nItemLine ) // Valor do Desconto
544 STDSPBasket( "SL2" , "L2_DESC" , STBDiscConvert( STBTaxRet(nItemLine,"IT_DESCONTO") , "V" , STBTaxRet(nItemLine,"IT_VALMERC") ) [2] , nItemLine )
545 // Percentual de desconto
546 STDSPBasket( "SL2" , "L2_TES" , AllTrim(STBTaxRet(nItemLine,"IT_TES" )) , nItemLine )
547 STDSPBasket( "SL2" , "L2_CF" , AllTrim(STBTaxRet(nItemLine,"IT_CF" )) , nItemLine ) // Código Fiscal de Operação
548 STDSPBasket( "SL2" , "L2_VALACRS" , nItAcresci , nItemLine ) // Acréscimo financeiro
549
550 //Impostos
551 STDSPBasket( "SL2" , "L2_VALIPI" , STBTaxRet(nItemLine,"IT_VALIPI" ) , nItemLine )
552 STDSPBasket( "SL2" , "L2_VALISS" , STBTaxRet(nItemLine,"IT_VALISS" ) , nItemLine )
553 STDSPBasket( "SL2" , "L2_ALIQISS" , STBTaxRet(nItemLine,"IT_ALIQISS" ) , nItemLine )
554 STDSPBasket( "SL2" , "L2_BASEISS" , STBTaxRet(nItemLine,"IT_BASEISS" ) , nItemLine )
555 STDSPBasket( "SL2" , "L2_PRCUNI" , STBTaxRet(nItemLine,"IT_PRCUNI" ) , nItemLine )
556 STDSPBasket( "SL2" , "L2_VALPIS" , STBTaxRet(nItemLine,"IT_VALPIS" ) , nItemLine )
557 STDSPBasket( "SL2" , "L2_VALCOFI" , STBTaxRet(nItemLine,"IT_VALCOF" ) , nItemLine )
558 STDSPBasket( "SL2" , "L2_VALCSLL" , STBTaxRet(nItemLine,"IT_VALCSL" ) , nItemLine )
559 STDSPBasket( "SL2" , "L2_BASEPS2" , STBTaxRet(nItemLine,"IT_BASEPS2" ) , nItemLine )
560 STDSPBasket( "SL2" , "L2_VALPS2" , STBTaxRet(nItemLine,"IT_VALPS2" ) , nItemLine )
561 STDSPBasket( "SL2" , "L2_ALIQPS2" , STBTaxRet(nItemLine,"IT_ALIQPS2" ) , nItemLine )
562 STDSPBasket( "SL2" , "L2_BASECF2" , STBTaxRet(nItemLine,"IT_BASECF2" ) , nItemLine )
563 STDSPBasket( "SL2" , "L2_VALCF2" , STBTaxRet(nItemLine,"IT_VALCF2" ) , nItemLine )
564 STDSPBasket( "SL2" , "L2_ALIQCF2" , STBTaxRet(nItemLine,"IT_ALIQCF2" ) , nItemLine )
565 STDSPBasket( "SL2" , "L2_BRICMS" , STBTaxRet(nItemLine,"IT_BASESOL" ) , nItemLine )
566 STDSPBasket( "SL2" , "L2_ICMSRET" , STBTaxRet(nItemLine,"IT_VALSOL" ) , nItemLine )
567

```

```

568 If lExistGetTriSat
569     //TRIBUTACAO SAT
570     aSATtrib      := LjGetTriSat()
571
572     STBItSatTr(@aSATtrib, nItemLine, "IT_BASEPS2")
573     STBItSatTr(@aSATtrib, nItemLine, "IT_ALIQPS2")
574     STBItSatTr(@aSATtrib, nItemLine, "IT_BASECF2")
575     STBItSatTr(@aSATtrib, nItemLine, "IT_ALIQCF2")
576     STBItSatTr(@aSATtrib, nItemLine, "IT_PAUTPIS")
577     STBItSatTr(@aSATtrib, nItemLine, "IT_PAUTCOF")
578     STBItSatTr(@aSATtrib, nItemLine, "IT_CEST")
579
580     //Array de tributacoes do SAT
581     LjSetTriSat(aSATtrib)
582 EndIf
583
584 //valores de impostos por ente tributario da lei dos impostos, na atribuicao de total na venda
585 If ValType( aInfoItem ) == "A"
586
587     If Len(aInfoItem) == 0 //caso não esteja preenchido, passara aqui. Estara vazio quando vem chamado do total da venda (F7)
588         //busca as informacoes do item para passar a funcao de lei dos impostos
589         aInfoItem := STWFindItem( STDGPBasket( "SL2" , "L2_PRODUTO" , nItemLine ) )
590         If ExistFunc("STBImpItem") .AND. (Len(aInfoItem) >= 23)
591             aAlqLeiTr := STBImpItem(aInfoItem[ITEM_POSIPI] ,aInfoItem[ITEM_EX_NCM] ,aInfoItem[ITEM_CODISS] ,aInfoItem[ITEM_CODIGO] )
592
593             If Len(aAlqLeiTr) == 4 .OR. !(cMvFisAlCT == "3" .AND. cMvFisCTrb == "2")
594                 //carrega as aliquotas
595                 aInfoItem[ITEM_TOTIMP] := aAlqLeiTr[1]
596                 aInfoItem[ITEM_TOTFED] := aAlqLeiTr[2]
597                 aInfoItem[ITEM_TOTEST] := aAlqLeiTr[3]
598                 aInfoItem[ITEM_TOTMUN] := aAlqLeiTr[4]
599             ElseIf Len(aAlqLeiTr) == 8
600                 //carrega os valores
601                 aInfoItem[ITEM_TOTIMP] := aAlqLeiTr[5]
602                 aInfoItem[ITEM_TOTFED] := aAlqLeiTr[6]
603                 aInfoItem[ITEM_TOTEST] := aAlqLeiTr[7]
604                 aInfoItem[ITEM_TOTMUN] := aAlqLeiTr[8]
605             EndIf
606         EndIf
607     EndIf
608
609 If aInfoItem[ITEM_TOTIMP] > 0 // se tem algum indice de lei dos impostos carregado, faz os calculos
610     If Len(aAlqLeiTr) == 4 .OR. !(cMvFisAlCT == "3" .AND. cMvFisCTrb == "2")
611         nItTotal := STBTaxRet(nItemLine,"IT_TOTAL")
612         STDSPBasket( "SL2" , "L2_TOTIMP" , STBRound((nItTotal * aInfoItem[ITEM_TOTIMP])/100) , nItemLine )
613
614         //Lei dos impostos por Ente Tributario
615         If lCpoImpEnt .AND. Len(aInfoItem) >= 23 //garantir que o define ITEM_TOTFED exista
616             STDSPBasket( "SL2" , "L2_TOTFED" , STBRound((nItTotal * aInfoItem[ITEM_TOTFED])/100) , nItemLine )
617             STDSPBasket( "SL2" , "L2_TOTEST" , STBRound((nItTotal * aInfoItem[ITEM_TOTEST])/100) , nItemLine )
618             STDSPBasket( "SL2" , "L2_TOTMUN" , STBRound((nItTotal * aInfoItem[ITEM_TOTMUN])/100) , nItemLine )
619         EndIf

```

```

620     ElseIf Len(aAlqLeiTr) == 8
621         STDSPBasket( "SL2" , "L2_TOTIMP" , aInfoItem[ITEM_TOTIMP], nItemLine )
622         //Lei dos impostos por Ente Tributario
623         If lCpoImpEnt .AND. Len(aInfoItem) >=23 //garantir que o define ITEM_TOTFED exista
624             STDSPBasket( "SL2" , "L2_TOTFED" , aInfoItem[ITEM_TOTFED], nItemLine )
625             STDSPBasket( "SL2" , "L2_TOTEST" , aInfoItem[ITEM_TOTEST], nItemLine )
626             STDSPBasket( "SL2" , "L2_TOTMUN" , aInfoItem[ITEM_TOTMUN], nItemLine )
627         EndIf
628     EndIf
629 EndIf
630 EndIf
631
632 // Controle de impostos IVA
633 aTesImpInf := TesImpInf( AllTrim(STBTaxRet(nItemLine,"IT_TES" )))
634
635 /* Estrutura do aTesImpInf
636 [n][1]-> Codigo do Imposto
637 [n][2]-> Campo no SD1 ou SD2 onde e gravado o valor imposto
638 [n][3]-> Se o valor do imposto incide na Nota
639 [n][4]-> Se o valor do imposto incide na Duplicata
640 [n][5]-> Se o valor do imposto deve ser Creditado
641 [n][6]-> Campo no SF1 ou SF2 onde e gravado o valor imposto
642 [n][7]-> Campo no SD1 ou SD2 onde e gravada a base do imposto
643 [n][8]-> Campo no SF1 ou SF2 onde e gravada a base do imposto
644 [n][9]-> Aliquota do imposto
645 */
646
647 If Len(aTesImpInf) > 0
648     For nX := 1 To Len(aTesImpInf)
649         //Guarda posicao do imposto corrente
650         cIndImp := Substr("L2_"+Substr(aTesImpInf[nX][2],4,7),Len("L2_"+Substr(aTesImpInf[nX][2],4,7)),1)
651
652         STDSPBasket( "SL2" , "L2_BASIMP"+cIndImp , STBTaxRet( nItemLine,"IT_BASEIV"+ cIndImp ) , nItemLine )
653         STDSPBasket( "SL2" , "L2_VALIMP"+cIndImp , STBTaxRet( nItemLine,"IT_VALIV"+ cIndImp ) , nItemLine )
654         STDSPBasket( "SL2" , "L2_ALQIMP"+cIndImp , STBTaxRet( nItemLine,"IT_ALIQIV"+ cIndImp ) , nItemLine )
655     Next nX
656 EndIf
657 EndIf
658 EndIf
659
660 STBSetDefQuant() // Seta quantidade padrao apos o registro de item
661 STBDefItDiscount() // Seta desconto padrao apos registro de item
662
663 FreeObj(jCfgTaxes)
664 jCfgTaxes := Nil
665
666 Return lRet
667
668
669
670
671

```

```

672
673 //-----
674 /*{Protoheus.doc} STBDataBar
675 Faz a leitura a interpretacao do codigo de barras
676 GS1 Data Bar
677 @param  Codigo Capturado pelo leitor
678 @return  aReturn - Retorna codbar e data validade
679 @obs
680 @sample
681 */
682 //-----
683 Function STBDataBar(cCodDtaBar)
684
685 Local cSymbolId := "" // Simbolo identificador do tipo de cod bar
686 Local cAI      := "" // Application Identifiers
687 Local aReturn  := {} // Retorno da funcao em 2 dimensoes
688 Local cCodBar  := "" // Codigo de barras no padrao EAN 13
689 Local dDtValid := "" // Data de validade do produto
690
691 DEFAULT cCodDtaBar := ""
692
693 //Separa os simbolos do databar, verifique o documento
694 //Databar_GS1_General_Specifications_v13_Identificador_de_Simbologia.pdf (deve ser solicitado a GS1)
695 cSymbolId := Substr(cCodDtaBar,1,3)
696 cCodDtaBar := Substr(cCodDtaBar,4,Len(cCodDtaBar))
697 cCodBar    := cCodDtaBar
698
699 If UPPER(cSymbolId) == "[E0" //GS1 DataBar
700
701   While Len(cCodDtaBar) > 0
702     cAI := Substr(cCodDtaBar,1,2)
703
704     If cAI == "01" // Tradicional EAN 13
705       cCodBar := Substr(cCodDtaBar,4,13)
706       cCodDtaBar := Substr(cCodDtaBar,17,Len(cCodDtaBar))// Remove o trecho capturado
707
708     ElseIf cAI == "17" // Data de Validade YYMMDD
709       dDtValid := STOD("20"+Substr(cCodDtaBar,3,6) )
710       cCodDtaBar := Substr(cCodDtaBar,9,Len(cCodDtaBar)) // Remove o trecho capturado
711     Else
712       //Para esse primeiro modelo nao foram desenvolvidos todos os codigos AIs
713       cCodDtaBar := ""
714     EndIf
715   End
716 Else
717   cCodBar := cSymbolId + cCodDtaBar // Caso nao encontre retorna o mesmo cod
718 EndIf
719
720 AAdd( aReturn, { cCodBar } ) // Tradicional EAN 13
721 AAdd( aReturn, { dDtValid } ) // Data de Validade
722
723 Return(aReturn)

```

```

724
725
726 //-----
727 /*{Protheus.doc} STBSaleAct
728 Funcao responsavel em verificar se uma venda esta em andamento ou nao
729 @param lActive
730 @return
731 @obs
732 @sample
733 */
734 //-----
735 Function STBSaleAct(lActive)
736 Default lActive := .F.
737 lSaleActive := lActive
738 Return .T.
739
740 //-----
741 /*{Protheus.doc} STBGetAct
742 Funcao para retornar o valor da lSaleActive
743 @param
744 @return lSaleActive
745 @obs
746 @sample
747 */
748 //-----
749 Function STBGetAct()
750 Return lSaleActive
751
752
753 //-----
754 /*{Protheus.doc} STBPegaIT
755 Transforma o Numero Item Para 2 Bytes.
756 @param
757 @return lSaleActive
758 @obs
759 @sample
760 */
761 //-----
762 Function STBPegaIT(uItem)
763
764 Local nX          := 0              // Contador de For
765 Local nY          := 0              // Contador de For
766 Local aTemp       := {}             // Array temporario
767 Local nTamSd2It   := TamSX3("D2_ITEM")[1] //Tamanho do campo D2_ITEM
768 Local aItems      := {} //Guarda o item para verificar o L2_ITEM
769
770 //ÚÀAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
771 //³Se o campo for caracter transformar em maiusculo por segurança³
772 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
773 If ValType(uItem) == "C"
774     uItem := Upper( Right(uItem, nTamSd2It) )
775 EndIf
```

[illegible]

```

828 @return nil
829 @obs
830 @sample
831 /*/
832 //-----
833 Function STBSumItem(lItemFisc, nItemLine)
834 Local nItem := 0 //Numero do Contador
835 Local cItem := "" //Codigo do Item
836
837 Default lItemFisc := .T.
838 Default nItemLine := 0
839
840 If lItemFisc
841     nItem := STDGPBasket("SL1","L1_TOTFISC")+1
842     STDSPBasket("SL1","L1_TOTFISC", nItem)
843     cItem := STBPegaIT(nItem)
844     STDSPBasket("SL2", "L2_ITFISC",cItem, nItemLine)
845
846 Else
847     nItem := STDGPBasket("SL1","L1_TOTNFIS")+1
848     STDSPBasket("SL1","L1_TOTNFIS", nItem)
849     cItem := STBPegaIT(nItem)
850     STDSPBasket("SL2", "L2_ITNFISC", cItem, nItemLine)
851 EndIf
852
853 Return
854
855
856 //-----
857 /*/{Protheus.doc} STBImpItem
858 Lei dos impostos para o cupom
859 @param lItemFisc - Item Fiscal
860 @return nil
861 @obs
862 @sample
863 /*/
864 //-----
865 Function STBImpItem(cPOSIPI,cEX_NCM,cCODISS,cCODIGO)
866
867 Local xB1ImpNcm := Nil // variavel que controla o retorno da função AlqLei2741
868 Local lAlqLei2741 := FindFunction("AlqLei2741") // Funcao desenvolvida pelo fiscal para tratar a lei dos impostos
869 Local aAlqLeiTr := {} // array com valores da aliquota retornado pela função fiscal (Lei transparência) 1-Aliq Total|2-Aliq
    Federal|3-Aliq Estadual|4-Aliq Municipal
870 Local lCpoTotImp := (SL2->(ColumnPos("L2_TOTIMP"))) > 0) .And. (SB1->(ColumnPos("B1_IMPNCM"))) > 0)
871 Local nIMPNCM := 0
872 Local cB1CodISS := ""
873 Local aArea := {}
874
875 xB1ImpNcm := 0
876
877 // nova funcionalidade disponibilizada pelo Fiscal
878 If lAlqLei2741

```

```

879     xBlImpNcm := 0
880     aArea      := GetArea()
881
882     xBlImpNcm := AlqLei2741(cPOSIPI      ,cEX_NCM      ,cCODISS      ,SA1->A1_EST, ;
883                           SA1->A1_COD_MUN ,cCODIGO      ,1 /*Ref Item prod Matxfis*/ ," "      , ;
884                           /*cLoteCtl*/    ,/*cMvFisCTrb */ ,/*cMvFisAlCT*/    ,/*lMvFisFRas*/ )
885
886     RestArea(aArea)
887 Else
888
889     If lCpoTotImp
890         nIMPNCM := RetFldProd(SB1->B1_COD,"B1_IMPNCM")
891         xBlImpNcm := Iif(Empty(nIMPNCM ), SB1->B1_IMPNCM, nIMPNCM )
892     EndIf
893
894     If Findfunction("Lj7BuscaImp")
895         If xBlImpNcm ==0 .AND. !Empty(SB1->B1_POSIPI)
896             cBlCodISS := RetFldProd(SB1->B1_COD,"B1_CODISS")
897             cBlCodISS := Iif(Empty(cBlCodISS), SB1->B1_CODISS, cBlCodISS)
898             xBlImpNcm := Lj7BuscaImp(SB1->B1_POSIPI,!Empty(cBlCodISS),.F.)
899         EndIf
900     EndIf
901
902 EndIf
903
904 If ValType(xBlImpNcm) == "N"
905     aAlqLeiTr := {xBlImpNcm,0,0,0}
906 ElseIf Len(xBlImpNcm) == 2
907     aAlqLeiTr := {xBlImpNcm[1],0,0,0,xBlImpNcm[2],0,0,0}
908 Else
909     aAlqLeiTr := aClone(xBlImpNcm)
910 EndIf
911
912 Return aAlqLeiTr
913
914
915 //-----
916 /*/{Protheus.doc} STBAjustIT
917 Essa função tem o objetivo realizar os ajustes nos itens:
918 ajuste no ICMS ao aplicar o desconto.
919
920 @param
921 @return
922 @obs
923 @sample
924 */
925 //-----
926 Function STBAjusICM()
927 Local nI      := 0
928 Local lRet    := .T.
929
930 For nI := 1 To STDPBLength("SL2")

```

```

931
932     If !STDPBIsDeleted( "SL2", nI )
933
934         IIf( lRet , lRet := STDSPBasket( "SL2" , "L2_BASEICM" , STBTaxRet(nI ,"IT_BASEICM") , nI ) , )
935         IIf( lRet , lRet := STDSPBasket( "SL2" , "L2_VALICM" , STBTaxRet(nI ,"IT_VALICM") , nI ) , )
936         IIf( lRet, lRet := STDSPBasket( "SL2" , "L2_VALFECP" , STBTaxRet(nI,"IT_VALFECP") , nI ) , )
937
938     EndIf
939
940 Next nI
941
942 If lRet
943     LjGrvLog(SL1->L1_NUM ,"STBAjusICM - Ajustado da base e valor do ICMS")
944 Else
945     LjGrvLog(SL1->L1_NUM ,"STBAjusICM - Não foi possível o Ajustate da base e valor do ICMS")
946 EndIf
947
948 Return
949
950
951 //-----
952 /*{Protheus.doc} STBItSatTr
953 Essa função tem o objetivo armazenar base de cálculo e valor dos
954 impostos Pis, Cofins, CST, etc...
955
956
957 @param aSatTrib, Array , Referência criada para impostos PIS/COFINS/CST etc e transmitido pelo XML do SAT
958 @param nItem , Numérico , número do item do TOTVS PDV
959 @param cCpo , Carácter , qual campo do MATXFIS deverá ser lido
960 @return nil
961
962 @obs
963 @sample
964 */
965 //-----
966 Function STBItSatTr(aSatTrib, nItem, cCpo)
967
968 Local nPos := 0
969
970 Default aSatTrib := {} //Parâmetro de Referência
971 Default nItem := 0
972 Default cCpo := ""
973
974 nPos := aScan(aSATtrib, {|x| x[1] == nItem .And. x[2] == cCpo } )
975 If nPos > 0 .AND. Len(aSatTrib[nPos]) > 3
976     aSatTrib[nPos][3] := STBTaxRet(nItem, cCpo) //Valor do imposto
977     If Len(aSatTrib[nPos]) >= 4 .AND. aSatTrib[nPos][4] <= 0
978         aSatTrib[nPos][4] := nItem
979     EndIf
980 Else
981     aAdd(aSATtrib, {nItem,cCpo,STBTaxRet(nItem, cCpo),nItem})
982 EndIf

```

983
984 Return nil

Componentes/Item/STBQuantity.prw

188 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBQUANTITY.CH"
4
5 Static nItemQuant := 1          // Quantidade do item default = 1
6 Static nQtdOther  := 0          // Quantidade do item default = 0
7
8 //-----
9 /*{Protheus.doc} STBQuantity
10 Seta quantidade do item
11
12 @param  nItemQuant    Quantidade do Item
13 @return Nil
14 @obs
15 @sample
16 */
17 //-----
18 Function STBSetQuant( nQuant, nTpQtd )
19
20 Default  nQuant := 1          //Quantidade do item
21 Default  nTpQtd := 1          //Tipo da quantidade a ser armazenada
22
23 If  nTpQtd == 1
24     nItemQuant := nQuant
25 Else
26     nQtdOther  := nQuant
27 EndIf
28
29 Return Nil
30
31
32 //-----
33 /*{Protheus.doc} STBSetDefQuant
34 Seta quantidade default do item
35
36 @param
37 @return Nil
38 @obs
39 @sample
40 */
41 //-----
42 Function STBSetDefQuant()
43
44 // seta quantidade default
45 nItemQuant := 1
46 nQtdOther  := 0
47
48 Return Nil
49
50
51 //-----
52 /*{Protheus.doc} STBGetQuant

```

```

53 Pega quantidade do item
54
55 @param
56 @return nItemQuant      Quantidade do Item
57 @obs
58 @sample
59 /*/
60 //-----
61 Function STBGetQuant(nTpQtd)
62 Local nQtdRet := 0      //Retorno da função
63
64 Default nTpQtd := 1      //Tipo da quantidade a ser resgatada
65
66 If nTpQtd == 1
67     nQtdRet := nItemQuant
68 Else
69     nQtdRet := nQtdOther
70 EndIf
71
72 Return nQtdRet
73
74
75 //-----
76 /*/{Protheus.doc} STBBalQuant
77 Pega quantidade do item na balanca
78
79 @param
80 @return Nil
81 @obs
82 @sample
83 /*/
84 //-----
85 Function STBBalQuant()
86
87 Local nTmpQtd      := 0          // Guarda quantidade da balanca
88 Local aRet         := {}         // Array de Retorno
89 Local aData        := {}         // Array data
90
91 /*/
92     Verifica se usa balanca para pegar peso - ( Quantidade = Peso )
93 /*/
94
95 aRet :=      STFFireEvent(  ProcName(0)      ,;      // Nome do processo
96                           "STBBalQuant"    ,;      // Nome do evento
97                           {.T.}            )      // 06 - Define se usa a balaça
98
99
100 If ValType(aRet) == "U" .OR. Len(aRet) == 0 .OR. Valtype(aRet[1]) <> "L" .OR. ! aRet[1]
101     STFMessage("STBBalQuant","POPUP",STR0001) //"Por favor colocar o produto na balança ou informar a quantidade!"
102     STFShowMessage("STBBalQuant")
103
104     STBSetQuant(0)

```

```

105 Else
106     aRet := STFFireEvent( ProcName(0)    ;;      // Nome do processo
107                          "STBalanceGet"  ;;      // Nome do evento
108                          aData           )      //
109
110     If ValType(aRet) == "U" .OR. ValType(aData) == "U" .OR. Len(aData) == 0 .OR. Valtype(aData[1]) <> "N"
111         STFMessage("STBBalQuant","POPUP",STR0001) //"Por favor colocar o produto na balança ou informar a quantidade!"
112         STFShowMessage("STBBalQuant")
113
114         STBSetQuant(0)
115     Else
116         nTmpQtd := aData[1]
117         If nTmpQtd > 0
118             STBSetQuant( nTmpQtd )
119         Else
120             STFMessage("STBBalQuant","POPUP",STR0001) //"Por favor colocar o produto na balança ou informar a quantidade!"
121             STFShowMessage("STBBalQuant")
122
123             STBSetQuant(0)
124         EndIf
125     EndIf
126
127 EndIf
128
129 Return Nil
130
131
132 //-----
133 /*{Protheus.doc} STBQtdEtq
134 Seta quantidade de acordo com a etiqueta de codigos de barras
135 e o preco
136
137 @param  cCodeReceived      Codigo recebido/digitado
138 @param  nPrice             Preco do item
139 @param  cBalanca           Tipo produto de balanca
140
141 @return Nil
142 @obs
143 @sample
144 */
145 //-----
146
147 Function STBQtdEtq( cCodeReceived , nPrice , cBalanca )
148
149 Local cCodProBal      := ""          // Codigo do Item da balanca
150 Local cInfProBal      := ""          // Peso/quantidade do Item da balanca
151 Local nValBal         := 0
152 Local nQtdBal         := 0
153 Local nDifBal         := 0
154
155 Default cCodeReceived := ""          // Codigo do Item
156 Default nPrice         := 0          // Preco do item

```

```

157 Default cBalanca      := ""          // Codigo do Cliente
158
159 ParamType 0  Var cCodeReceived      AS Character      Default ""
160 ParamType 1  Var nPrice              AS Numeric        Default 0
161 ParamType 2  Var cBalanca           AS Character      Default ""
162
163 //Separa o codigo recebido para fazer validacao da balanca
164 cCodProBal := Substr(cCodeReceived,2,6) //Codigo do Item da balanca
165 cInfProBal := SubStr(cCodeReceived,8,5) //Peso/quantidade do Item da balanca
166
167 // Verifica se o produto esta no padrao EAN-13
168 // e se o primeiro digito eh 2 (padrao balanca)
169 If cBalanca == "1" .AND. Len(Alltrim(cCodeReceived)) == 13 .AND. Substr(cCodeReceived,1,1) == "2"
170     //comparar o valor da etiqueta com o valor calculado
171     nValBal := Val(cInfProBal)/100
172     nQtdBal := Round(nValBal / nPrice ,3 )
173     nDifBal := nValBal - (A410Arred((nPrice * nQtdBal),"D2_TOTAL"))
174
175     //A Balanca Utiliza 3 Decimais!!!
176     //Esse tratamento é realizado quando há diferebça do valor enviado na etiqueta e o valor calculado pelo sistema
177     //Para o valor total ficar igual ao valor enviado pela etiqueta, o parâmetro MV_ARREFAT deve ser igual a 'N' (trunca o valor)
178     If nDifBal > 0
179         nQtdBal += 0.001
180     ElseIf nQtdBal < 0
181         nQtdBal -= 0.001
182     EndIf
183
184     STBSetQuant (nQtdBal)
185
186 EndIf
187
188 Return Nil

```

Componentes/Item/STBSetItDiscount.prw

69 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static nItemDiscount := 0           // Valor do desconto
5 Static cTypeDiscount := ""         // Tipo de desconto. "V" - Valor , "P" - Percentual
6
7
8 //-----
9 /*{Protheus.doc} STBQuantity
10 Set desconto do item
11
12 @param  nItemQuant      Quantidade do Item
13 @param  cTypeDiscount   Tipo de desconto. "V" - Valor , "P" - Percentual
14 @return Nil
15 @obs
16 @sample
17 */
18 //-----
19 Function STBSetItDiscount( nValDiscount , cType )
20
21 Default nValDiscount := 0
22 Default cType         := ""
23
24 ParamType 0 Var nValDiscount AS Numeric      Default 0
25 ParamType 1 Var cType       AS Character    Default ""
26
27 nItemDiscount := nValDiscount
28 cTypeDiscount := cType
29
30 Return Nil
31
32
33 //-----
34 /*{Protheus.doc} STBGetItDiscount
35 Get desconto do item
36
37 @param
38 @return aRet[1]      nItemDiscount - Valor de desconto
39 @return aRet[2]      cTypeDiscount - Tipo de desconto
40 @obs
41 @sample
42 */
43 //-----
44 Function STBGetItDiscount()
45
46 Local aRet := {}           // Retorno Função
47
48 AADD( aRet , nItemDiscount )
49 AADD( aRet , cTypeDiscount )
50
51 Return aRet
52

```

```
53
54 //-----
55 /*{Protheus.doc} STBDefItDiscount
56 Set default desconto do item
57
58 @param
59 @return Nil
60 @obs
61 @sample
62 */
63 //-----
64 Function STBDefItDiscount()
65
66 nItemDiscount := 0           // Valor do desconto
67 cTypeDiscount := ""         // Tipo de desconto. "V" - Valor , "P" - Percentual
68
69 Return Nil
```